



TEAMCENTER

Workflow Designer on Active Workspace

Software Version 2606

Unpublished work. © 2026 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: <https://www.sw.siemens.com/en-US/trademarks/>. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center

Log in to Support Center to access software downloads and updates, product and release documentation, Knowledge Base articles, videos, other self-service resources, and to open support cases: <https://support.sw.siemens.com>.

If you do not have a Support Center login, register here: <https://support.sw.siemens.com/register>.

To send feedback on documentation: https://support.sw.siemens.com/doc_feedback_form.

Table of Contents

- Overview of Workflow Designer..... 1-1**
 - What is Workflow Designer?..... 1-1
 - Before you begin..... 1-3
 - What can I do with Workflow Designer? 1-4
 - Workflow templates..... 1-5
 - Access templates..... 1-5
 - Workflow elements..... 1-6
 - The Workflow Designer interface..... 1-10
- Creating workflow templates..... 2-1**
 - Workflow templates overview..... 2-1
 - Basic workflow template process..... 2-1
 - Core templates..... 2-3
 - Create a workflow template..... 2-3
 - Add tasks to a template..... 2-4
 - Add paths between tasks..... 2-4
 - Delete paths between tasks..... 2-5
 - Failure paths..... 2-6
 - Task template types..... 2-8
 - Customize task paths..... 2-15
 - Subworkflows..... 2-15
 - What are subworkflows?..... 2-15
 - Create a subworkflow..... 2-17
 - Creating subworkflows for assemblies..... 2-19
 - Creating subworkflows for related objects..... 2-20
- Creating task templates..... 3-1**
 - Create a task template..... 3-1
 - Create custom task templates..... 3-1
 - What are custom task templates?..... 3-1
 - Create new business types for custom task templates..... 3-2
 - Create a custom task template..... 3-2
 - Configure a custom task template..... 3-3
- Editing workflow templates..... 4-1**
 - Determining which editing options to use..... 4-1
 - Offline versus online editing..... 4-2
 - Identify templates as non-modifiable..... 4-2
 - Edit workflow templates..... 4-3
 - Edit a template..... 4-3
 - Apply template edits to active processes..... 4-4
 - Edit the template layout..... 4-5

Add filter conditions to templates.....	4-6
Filtering conditions.....	4-6
Adding targets after submitting a workflow.....	4-7
Configure a default workflow template.....	4-7
Preference considerations.....	4-7
Configure a default workflow template for an object.....	4-8
Configure a default workflow template specifically for groups and subgroups.....	4-9
Managing templates in Workflow Designer.....	5-1
Importing and exporting templates in Workflow Designer.....	5-1
Import workflow templates.....	5-1
Export workflow templates.....	5-2
Select and compare workflow templates.....	5-2
Delete templates.....	5-3
Distributing workflow templates using Multi-Site Collaboration.....	5-3
Replicate a workflow template.....	5-3
Synchronize replicated templates.....	5-4
Managing Task Behavior.....	6-1
Delete a task.....	6-1
Modifying tasks.....	6-1
Modify tasks in Workflow Designer.....	6-1
Access Control Lists.....	6-2
Work with task handlers.....	6-9
Assign users to tasks.....	6-15
Add notifications to tasks.....	6-21
Retain approvals.....	6-23
Work with forms.....	6-28
Modify task properties.....	6-29
Using an assignment matrix.....	7-1
What is an assignment matrix?.....	7-1
The assignment matrix process.....	7-2
Understanding assignment matrices using a mail-sorting example.....	7-2
Administrative tasks.....	7-6
The assignment matrix discovery process.....	7-10
Create an assignment matrix definition.....	7-24
Create and populate an assignment matrix.....	7-26
Add entries to an assignment matrix.....	7-27
Update an assignment matrix.....	7-36
Add and modify the AMX handler.....	7-38

1. Overview of Workflow Designer

What is Workflow Designer?

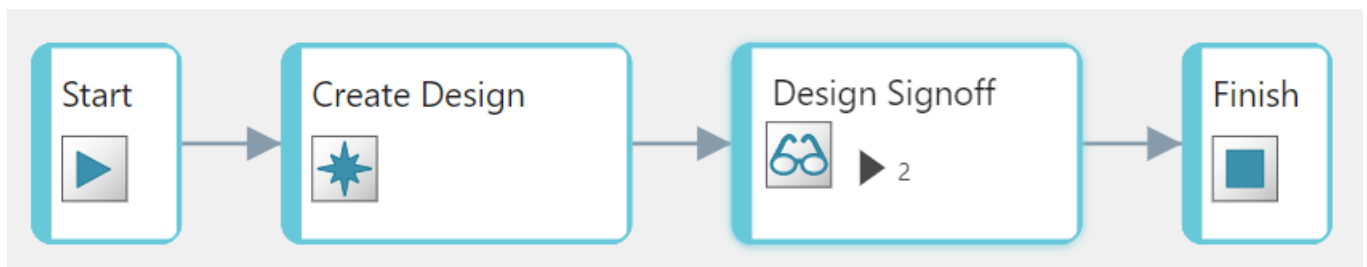
Workflow Designer lets you design workflow templates that can incorporate and automate your company's business practices and procedures. Business users then use these templates to initiate workflow processes.

The concept of a workflow is that all work goes through one or more business processes to accomplish an objective, flowing from one task to the next. Workflow is the automation of these business processes.

A *workflow* contains a series of tasks to complete in order to reach a repeatable business goal. Customer business objects (such as documents and parts) are passed between participants during a workflow process.

Workflow templates define the blueprint of a workflow process to be performed at your site that can enforce best practices and eliminate errors. Templates are a good way to embed process knowledge in the project to ensure everyone on the team complies with the process, even though they may not know all its details.

In this example, the administrator created a template to request a design from a system designer (shown here by the **Do** task, **Create Design**), which is then followed by a review from other team members (the **Design Signoff** task) before it can be marked as complete (the **Finish** task). Users can reuse this template to standardize best practices and ensure that a continuous process is used throughout a project.



Workflow Designer's multiple features work together in various ways to improve your product's efficiency and productivity.



Where do I go from here?

The following table shows the high-level roles engaged with Workflow Designer.

 Administrator	An Administrator, or a workflow template designer, in the default DBA group is the only user who can create templates based on existing templates or their own design. Administrators can also modify, delete, import, or export existing templates. Business users outside the DBA group can be added to this workflow designer group by modifying the Awp0IsWorkflowTemplateAuthor condition. This condition is available in BMIDE and defines who has edit privileges. For details, see Access templates.
Get me started	Take a look at the prerequisites and information on how you can enable, configure, and start Workflow Designer before you begin .
Working with templates in Workflow Designer	• Create a workflow template • Import workflow templates

	• Export workflow templates
Modify an existing task to create a new one	Create a task template
Manage task behavior	Modify tasks in Workflow Designer
Learn more about workflow handlers	Work with task handlers
Create privileged users	Allow non-DBA users to create and edit templates
 Business User	All non-administrative users (or those who do not design templates) can access the Workflow Designer to view the list of templates and what they contain in read-only mode. If non-DBA users select a workflow template, they can view its task data but can neither modify nor import or export the template. Non-DBA users also cannot see templates that are under construction.
Get me started with applying templates to workflows.	See Workflows and Tasks to learn more about applying templates to a workflow.

Before you begin

This section provides essential information to ensure you are prepared to start using the Workflow Designer effectively.

Prerequisites

Ensure that you have administrator privileges to use the Workflow Designer in **Edit**  mode.

Enable Workflow Designer

1. Log on to Deployment Center and select the environment to which you want to add Workflow.
2. Go to the **Applications** tab. Click **Add or Remove Selected Applications** .
3. In the **Available Applications** panel, use the web browser search to find the Workflow application. Select the application, and then click **Update Selected Applications**.

Deployment Center automatically selects any additional dependent applications.

4. Go to the **Deploy** tab. Click **Generate Install Scripts** to generate deployment scripts you will use to update affected machines.
When script generation is complete, note any special instructions in the **Deploy Instructions** panel.
5. Locate deployment scripts, copy each script to its target machine, and then run each script on its target machine.

For more information about running deployment scripts, see *Deployment Center — Usage*.

Or, if you see Teamcenter Environment Manager (TEM), add the following features in the **Features** panel:

- **Workflow** (Client)
Installs the user interface elements for using Workflow.
Select **Active Workspace**→**Client**→**Workflow**.
- **Workflow** (Server)
Installs the server-side definitions and templates for **Workflow**.
Select **Active Workspace**→**Server Extensions**→**Workflow**

To use the Workflow Designer, install the **Active Admin** workspace.

If you have trouble accessing Workflow Designer, contact your system administrator.

Configure Workflow Designer

You can use Workflow Designer's default configuration settings or modify them using workflow preferences.

Start Workflow Designer

From the **Launcher**, click **Workflow Designer**.

If this option is unavailable, search for it. You can pin it from the search results for future access.

What can I do with Workflow Designer?

Workflow Designer provides a graphical editor to perform the following capabilities:

- Create a workflow template, either new or based on an existing template.
- Update the workflow template to add tasks.
- Provide visual cues for process and task templates as well as online and offline templates.
- Import and export process templates, including those created in the Teamcenter rich client.
- Create custom workflow templates, update template properties, and so on, using workflow handlers. Hints are provided for the workflow handler arguments and their respective values to help users identify mandatory and optional parameters. These hints can avoid runtime errors because validation is performed during creation. A JSON file must be created for each handler to enable this feature.

Workflow templates

A *workflow template* is a predefined workflow structure that you use as a pattern for your own workflow processes. Each instance of a workflow process uses a workflow template. A workflow template sets up the rules for a workflow and is used as a blueprint for creating multiple workflow processes.

Using Workflow Designer, you can do the following:

- Create both sequential (serially) or asynchronous (in parallel) workflow process templates.
- Build new workflow process templates based on core templates.
- Define a specific workflow process by placing workflow tasks in the required performance order.
- Define additional workflow process requirements (such as placing a status on targets and creating subprocesses) in the template using workflow handlers.

Access templates

In Workflow Designer, user access to templates differs according to predefined groups, as determined by the system administrator.

- **Workflow Designers**

System administrators in the default **DBA** group are the only users who can create templates based on existing templates or their own design. Administrators can also modify existing templates and import and export templates.

Non-**DBA** users can be added to this **Workflow Designers** group by modifying the **Awp0IsWorkflowTemplateAuthor** condition. This condition is available in BMIDE and defines who has edit privileges. These workflow commands use server-side conditions; you can change this behavior by overriding the commands.

The **Awp0IsPrivilegedWorkflowTemplateAuthor** condition has the same privileges as **Awp0IsWorkflowTemplateAuthor**, but also allows for modifications to templates that are marked as read-only (**Require admin privilege for modification**, found in a workflow's properties).

The Workflow Designer is available in the following workspaces for DBA users: **Active Admin**, **Active Architect**, and the **Default** workspace.

- **Non-Workflow Designers**

All non-Workflow Designer users can access the Workflow Designer via the **Launcher** to view the list of templates and what they contain in read-only mode. If non-DBA users select a workflow template, they can view its task data but can neither modify nor import or export the template. Non-DBA users also cannot see templates that are under construction.

Allow non-DBA users to create, edit, import, and export templates

If an administrator wants to include a group of users that are granted Workflow Designer privileges, non-DBA users can be identified to create, import, export, and edit templates.

Note
Non-DBA users can add existing ACLs to a template but cannot create new ones.

To identify non-DBA users, administrators must do the following.

Procedure

- 1. Modify the BMIDE condition **Awp0IsWorkflowTemplateAuthor** to specify a group that includes users with template privileges. This example allows users in the **AuthorGroup** to create and modify templates.

```
Awp0IsWorkflowTemplateAuthor ( UserSession u)
((u.fnd0ConditionHelper.awp0GetPropertyvalue(u.group,"privilege","1",false)) OR
((u.fnd0ConditionHelper.awp0IsPlatformVersionAtLeast( "14.3","14.3" ) = true) AND
(Condition::Awp0IsPrivilegedWorkflowTemplateAuthor(u) = true)) OR u.group_name="AuthorGroup"
```

- 2. Create a new group using the name you specified in the condition, then add users who should be granted template privileges. Or, apply that specified name to an existing group to provide template privileges.

Workflow elements

In Teamcenter, workflows are *processes* based on process templates that are composed of *tasks*.

Term/Concept	Description
Workflow process	A <i>workflow process</i> is the automation of business procedures in which documents, information or tasks are passed from one participant to another in a way that is governed by rules or procedures. Teamcenter workflows allow you to manage your product data processes.
Parent process	<i>Parent workflow processes</i> are workflows that contain child workflow processes. When child workflow processes (also known as subprocesses) are created, the parent workflow processes can be dependent or independent of subprocesses.
Subprocess	A <i>subprocess</i> is the child workflow of a parent workflow process.
Workflow template	<i>Workflow templates</i> are blueprints of workflow processes. Your administrator creates process templates. A specific process is defined by placing tasks in the template in the required order of performance.

Term/Concept	Description
	Additional requirements, such as quorums and duration times, may also be included in the template.
Tasks	<i>Tasks</i> are the fundamental building block used to construct a process. Each task defines a set of actions, rules, and resources used to accomplish that task. User actions cause tasks to move from one state to another, and as a result, the overall process moves forward or backward.
Root task	The top-level task of every workflow is referred to as the <i>root task</i> . The root task is the top-level parent task that contains all the other tasks as subtasks.
Subtask	A <i>subtask</i> is the child task of a parent task.
Container tasks	<i>Container tasks</i> are tasks that contain other tasks: • Review Contains select-signoff-team and perform-signoffs tasks. The Decision options are Approve, Reject, and No Decision. • Acknowledge Contains select-signoff-team and perform-signoffs tasks. The Decision options are Acknowledged and Not Acknowledged. • Route Contains Review, Acknowledge and Notify tasks. • Task Use it as a starting point for creating your own custom tasks, such as tasks to carry your custom forms or other site-specific tasks that the users must complete.
Interactive tasks	<i>Interactive tasks</i> are tasks that require user interaction and display in the affected user's worklists. Different types of tasks have different interactive requirements. Typical tasks include: • select-signoff-team The assigned user is required to select a signoff team to sign off the target object of the task. • perform-signoffs Assigned users are required to review and sign off the target object of the task. • Do The assigned user is required to review and perform the task instructions, then mark the task complete. • Route

Term/Concept	Description
	Uses the Review, Acknowledge, and Notify subtasks, each of which has its own dialog box. • Task Use it as a starting point for creating your own custom tasks, such as tasks to carry your custom forms or other site-specific tasks that the users must complete.
Workflow handlers	<i>Workflow handlers</i> are small ITK programs used to extend and customize workflow tasks. Action handlers perform actions, such as attaching objects and sending email; rule handlers can identify whether a rule has been satisfied.
Task attributes	<i>Task attributes</i> are attributes that further configure task behavior. You can set security attributes, customize task symbols, and define condition results.
Quorum requirements	<i>Quorum requirements</i> are values that specify the number of approvals required before perform-signoffs tasks can complete and workflows can proceed.

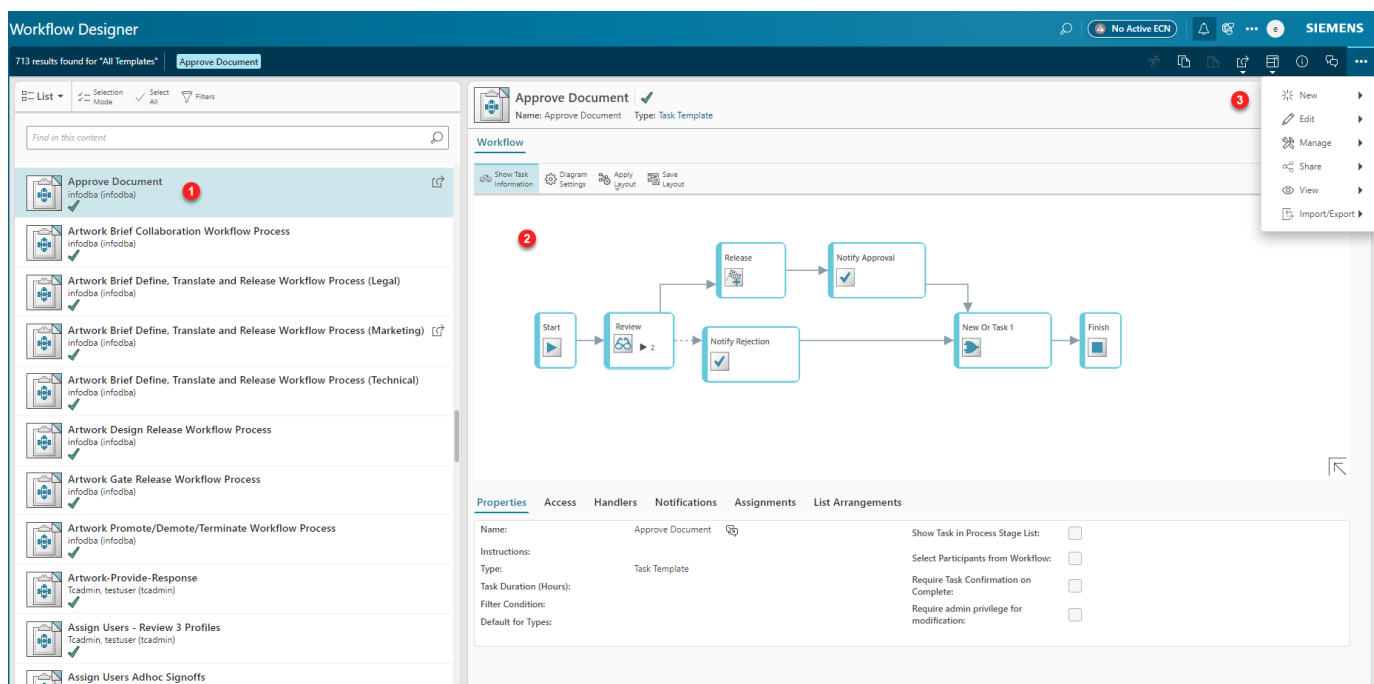
In a workflow, actions are assigned or allowed depending on the type of user.

User	Description
Responsible party	A <i>responsible party</i> is the user responsible for performing a particular task within a workflow process. While performing the task, the responsible party can reassign responsibility of the task to another user.
Privileged user	A <i>privileged user</i> is the responsible party, the process owner, or a member of a system administration group.
Current user	The <i>current user</i> is the user that completed the most recent task action and is independent of the responsible party. When using the "\$User" keyword in an argument, the "current user" is determined by evaluating the most recent task at the same level as the task containing the argument using the "\$User" keyword. For example, if the most recent task is in a sub-process and not on the same "process level" as the task using the argument with the "\$User" keyword, the argument will pull the current user from the most recent task of the parent process instead of the sub-task
Process owner	A <i>process owner</i> is the user who initiated the workflow process. The process owner is also known as the process initiator. When the workflow process is initiated, the process owner becomes the responsible party for the workflow process; the root task of the workflow process is placed in the process owner's worklist.

User	Description
	Whenever any task in the workflow process is not explicitly assigned to another user, person or resource pool, the responsible party for the task defaults to the process owner.
Task owner	A <i>task owner</i> is a user who is granted privileges for the task's target data.

The Workflow Designer interface

The Workflow Designer interface consists of multiple panels.



1

Once open, a filter results window displays the number of online and offline templates available.

The **Templates** panel is on the left. A navigation bar contains commands specific to the **Templates** panel.

Command	Description
View	Provides various display modes for the available templates: List with Summary, List, Table, Table with Summary, or Images.
Selection Mode	Lets you select a specific template(s).
Select All	Lets you select all available templates.
Filters	Lets you apply various criteria to restrict the list of available templates.
Search	Lets you search for a template by entering the name in the search field and clicking the search icon.
Open	Opens the template.

2 Once a workflow template is selected or opens, additional functionality is available. Additional icons appear when in edit mode.

Command	Description
Show Task Information	Displays details for the selected template under the Properties, Access, Handlers, Assignments, List Arrangements, Notifications, and Forms tabs.
Diagram Settings	Click the Show Grid check box to display major or minor grid lines to assist with symmetrical layout.
Apply Layout	Provides various options to display the task order.
Task Palette	Displays the various task types to add to the workflow template, as well the success or fail path branches, when the template is in edit mode.
Remove	Deletes an object from the graphic editor when the object is selected in edit mode.
Save Layout	Saves the current layout of the workflow and overwrites the position values displayed in either the fixed layout mode or in the rich client. This command is only shown in the auto layout mode. This is because the fixed layout mode automatically saves the task node positions as they are moved.
Online	The preference WRKFLW_save_node_positions must be set to true for this option to display. Makes the template online and available for users. Once the edits are saved, and you place the template online, the newly edited template overwrites the previous version.
Start Edit	Allows the user to edit the selected template. You can choose whether the template should be kept online or offline while it is being edited.
End Edit	Stops the template editing process and saves any updates.
Full Screen	Takes over the full browser window for the display.

3 Additional Workflow Designer functionality is found under **More Commands ...**.

Command	Description
New	Provides the options to create or import a new workflow template. The menu options under New are not displayed if a template is already open.
Create Workflow Templates	Creates a template, providing the option to enter a name, description, and whether the template is based on one that already exists.
Import	Imports an existing template from a file outside Workflow Designer.
Export	The Export option exports a template to an .XML file outside of Workflow Designer. You can use multi-select to export multiple templates at once.
Note The Export option is available under the Share menu.	
Edit	Displays the editing options.
Start Edit	Allows the user to edit the selected template. You can choose whether the template should be kept online or offline while it is being edited.
Template edits are saved automatically as you make them.	
Offline	Provides the user the option of keeping the template offline while being edited. In this case, you see only one instance of the template, which shows the under construction check mark icon  . The template is not available to other users while you are editing it.
Online	Makes the template online and available for users.
Once the edits are saved, and you place the template online, the newly edited template overwrites the previous version.	
End Edit	Stops the template editing process and saves any updates.
Delete	Deletes the workflow process template.

See [Creating Workflow Templates](#) for more information.

2. Creating workflow templates

Workflow templates overview

Basic workflow template process

A *workflow* describes the individual tasks and the task sequence required to model the workflow process.

A *workflow template* is a predefined workflow structure that you can use as a pattern for your own workflows. You can define a specific workflow by placing workflow tasks in the order required. You can define additional workflow process requirements, such as placing a status on targets, and creating subworkflows (also known as subprocesses).

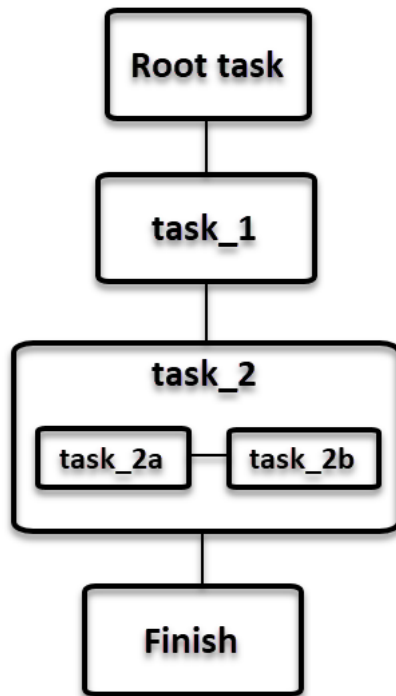
Workflow templates are created from scratch or based on existing templates. Existing templates include the out-of-the-box default templates included with Workflow Designer or those created by your organization.

You can use the out-of-the-box templates as a basis for creating your own templates, configured to include specific assignments, automatic notifications, or a unique purpose of the workflow.

Note

The out-of-the-box templates will always override during upgrades, so if you want to isolate your configuration changes, create Saved As copies of the templates. You can always view the latest changes on the out-of-the-box templates and then decide which changes you want to adopt at your own pace.

Each workflow contains a group of nested tasks. The top-level task of every workflow is referred to as the *root task*, as shown here in an example workflow structure.



The root task is the top-level parent task that contains all the other tasks as subtasks. It is the first task run when a workflow process is initiated and the last task to complete before the workflow itself is completed.

In a relatively simple workflow, as shown in the following example, the root task (Start) leads to a **Do** task (Complete the work). The **Do** task leads to a **Review** task (Review the work, which contains the **select-signoff-team** and **perform-signoffs** subtasks), which then leads to an **Acknowledge** task (Acknowledge the work, which also contains the **select-signoff-team** and **perform-signoffs** subtasks) and finally to a **Finish** step.



To design and maintain workflow processes in Workflow Designer, you can perform the following actions:

- Create templates
- View templates
- Add tasks to templates
- Link tasks
- Modify task behavior

- Import and export workflow templates

Core templates

The following table lists the templates and their associated types included with the rich client.

Template name	Task template definition type	Task type value specified in task template	Executing task's real type	Executing task's task type
Process	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Review Process	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Task	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Review Task	EPMTaskDefinition	EPMReviewTask	EPMTask	EPMReviewTask
Do Task	EPMDoTaskDefinition	EPMDoTask	EPMTask	EPMDoTask
Or Task	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Add Status Task	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Change Management Procedure	EPMTaskDefinition	EPMTask	EPMTask	EPMTask
Change Management Item	EPMTaskDefinition	EPMTask	EPMTask	EPMTask

Create a workflow template

Procedure

1. Go to **More Commands** *** > **New** ✨ > **Create Workflow Template** from the **Page level** toolbar.
2. Click **Workflow** as the **Template Type**.
3. Enter a **Name** and **Description**. A unique name is required. Entering a duplicate name will display an error message.
4. Under **Based On**, select an existing template on which to base the new task, or select **None** to create your own template.
5. Click **Add** to create the new template.

A basic template contains the start and end tasks. Add the individual tasks and paths to customize the template.

Add tasks to a template

After creating a template, you can add tasks.

Procedure

1. Select your newly created template from the list.
The initial template only contains the start and end tasks.
2. Note that the initial creation of a template automatically puts it in edit mode. However, if you had created a template, left it, and then came back to it, click **Start Edit**  to make any changes.
3. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
4. (Optional) Click **Settings**  to show the layout grid for aligning your tasks.
5. Click **Task Palette**  to add tasks to the template.
6. Select a task and drag to the template. The task name is highlighted for editing.
Continue adding tasks in the same manner.
7. Click **End Edit**  to save your new task(s) to the template.

Add paths between tasks

After adding one or more tasks, add paths connecting the tasks to promote their progression. A path establishes the sequence by which peer-level tasks are run, indicating that the task on the arrow end of the path cannot start until the task on the start end is completed.

Manually created paths, drawn from the predecessor task to the successor task, are explicit paths. Assumed paths are automatically created by the system if no explicit paths have been created from the **Start** node by the time the template is set to the **Available** stage. When you put a workflow template in **Edit** mode and draw a single path from the **Start** node to another task node, assumed path behavior is disabled. The system does not draw assumed paths, even if you leave tasks unlinked and change the workflow template to the **Available** stage. Any unlinked tasks are skipped when a workflow process based on the workflow template is initiated, and no error messages appear.

There are two basic path types, **Success** or **Fail**.

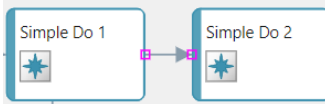
Procedure

1. Click **Task Palette**  to add paths to the template.
2. On the **Task Palette**, select a **Success** or **Fail** path.
3. Move your cursor over a task. A cross-hair indicates the starting point for the path. Click and hold the cursor to draw a path between tasks. Release the cursor to view the new path.

Tip

If you select a path, you can draw all the same path types at one time. For instance, select **Success**, and draw all the **Success** paths followed by all the **Fail** paths.

4. To change the established path type, click the path to display the pink line indicators, and click **Remove** . Then select **Success** or **Fail** from the **Task Palette**.



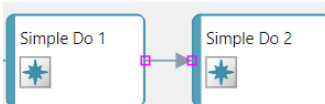
5. Go to **More Commands** *** > **Edit**  > **Online**  from the **Page level** toolbar. The template is now available to the organization.

Delete paths between tasks

When you delete a task from a template, the system deletes its path along with the task. If you do not reestablish explicit paths among the remaining tasks, the system creates assumed paths.

Procedure

1. With the template open, click **Task Palette**  to delete paths in the template.
2. To delete the established path type, click the path to display the pink line indicators, and click **Remove** .



The system deletes the selected path.

3. Go to **More Commands** *** > **Edit**  > **Online**  from the **Page level** toolbar. The template is now available to the organization.

Failure paths

A failure path gives an alternate course that a workflow can follow in any of the following scenarios:

- A task is rejected.
- The user determines that the task cannot be completed.
- There is an error.

When creating a workflow, each path is configured as either a *success path* or a *failure path*. A failure path must be configured into the workflow process template at design time. A task follows the appropriate path based on the task's outcome. A success path is traversed when a task's state transitions to **Complete** or when a task is promoted and it transitions to a **Skipped** state. A task completes upon the successful execution of the task's handlers on the **Complete** action.

It is important to note that a handler will not execute on a fail path. When a task traverses the failure path, the task state changes to **Failed** (not completed). The complete handlers will only execute when a success path is taken and the task state changes to **Completed**.

Note

An available option to still execute handlers when a task state changes to failed is to connect the task's failure path to a non-interactive task. The sole purpose of this non-interactive task would be to act as a place where handlers can be added along the failure path, if you want those handlers to execute, such as when a task fails. Subsequently, the non-interactive task's path can then flow into the next task that you want to execute in the workflow.

For example, if a Review task in a workflow gets rejected, including a non-interactive task along the failure path can execute a notify handler (EPM-notify) to communicate that the task was rejected with a failure email.

Backward branching allows a path to be routed backward to some previous task in the workflow process flow, including the **Start** node. Both success and failure paths are capable of branching in a backward direction. Backward branching allows the re-execution of a task with a **Complete** or **Skipped** task state.

To create a failure path, on the **Task Palette**, select the **Fail** path, then click and hold the cursor to draw a path between tasks. Failure path options display differently for different tasks.

Task	Failure option
Do	Set to Unable to Complete
Review	Set to Reject
Route	Set to Reject
Condition	Set to Unable to Complete

Task	Failure option
Validate	Set to Error Path
EPM	Set to Complete

Task template types

A *task template* is a blueprint of a workflow task. A task is a fundamental building block used to construct a workflow process. Each task defines a set of actions, rules, and resources used to accomplish that task.

Task templates can be both **designed by yourself** to apply and reuse in workflow templates or provided by the Workflow Designer, as shown below.

Symbol	Task	Definition
	Acknowledge Task	Uses the Acknowledged and Not Acknowledged subtasks, each of which has its own dialog.
	Add Status Task	Creates and adds a release status to the target objects of the workflow process. It is a visual milestone in a workflow process. No dialog box is associated with this type of task.
	Condition Task	Branches a workflow according to the defined query criteria. It requires that the succeeding task containing an EPM-check-condition handler that accepts a Boolean value of either True or False .
	Do Task	Allows the user to choose from two options if at least one failure path is configured. Complete confirms the completion of a task and triggers the branching to a success path. Unable to Complete indicates the task could not be completed, for varying reasons.
	Or Task	Continues the workflow process when any one of its multiple task predecessors is completed or promoted. There is no limit to the number of predecessors an Or task may have.
	Review Task	Uses the select-signoff-team and perform-signoffs subtasks, each of which has its own dialog. Full Participation Required is an option that allows the user to set the Review task to wait for all reviewers to submit their decisions before completing and following the appropriate path.
	Route Task	Uses the Review , Acknowledge , and Notify subtasks, each of which has its own dialog box.

Symbol	Task	Definition
	Task	Use it as a starting point for creating your own custom tasks, such as tasks to carry your custom forms or other site-specific tasks that the users must complete.
	Validate Task	Branches a workflow along two or more paths. Active paths flowing out of the task are determined by whether specified workflow errors occur. Use this task to design workflows around anticipated errors.

Custom tasks

A custom task template is a task that you can apply and reuse in workflow templates. These are based on a pre-existing Workflow Designer task. You can then edit that task to offer new capabilities. Use it as a starting point for creating your own custom tasks, such as tasks to carry your custom forms or other site-specific tasks for users to complete.

See [What are custom task templates](#) for more detailed information.

Do tasks

Use the **Do** task to define actions for a user to complete. When this task is performed in a workflow, it displays the required actions to the user in the **Overview** tab of the task.

Note

Configure the **WRKFLW_task_complete** preference to use single click functionality for a **Do** task. Setting the value to **true**, the **Complete** value is selected by default for the **Done** button.

If you require user authentication before this **Do** task is performed, add the EPM-require-authentication handler to the **Perform** action of the task. When you implement user authentication for this task, a password box appears below the **Comments** box. Users must type their user password in this box before they can click **Apply** and complete the task.

After completing the instructions, the user must select the **Complete** check box. The task does not complete until the user selects the check box. (This task is automatically configured with the EPM-hold handler to stop the task from completing until the check box is selected.) When the user selects the check box, the task sets the handler's argument to **False** and changes the status to **Complete**.

If the task is configured with a failure path the user can select one of the following check boxes:

- **Complete** confirms the completion of the task and continues the workflow down the success path.

- **Unable to Complete** indicates the user is unable to complete the instructions and continues the workflow down the failure path.

Review tasks

Use the **Review** task to route workflow targets (documents, parts, designs, and so on) for review.

Note

Configure the **WRKFLW_task_complete** preference to use single click functionality for a **Select Signoff Team** review task. Setting the value to **true**, the **Ad-hoc Done** check box is selected by default. This also sets the associated **task_result** property to **Complete** by default.

The task includes two subtasks:

- The **select-signoff-team** subtask requires the workflow initiator to select the users who will perform the review (the signoff team). You can configure this subtask with predefined group/role profiles that the workflow initiator must select or allow the workflow initiator to selector users of his choice in an ad hoc manner.

This subtask uses selection functionality from the Organization application, allowing the selector to search by group/role/user and to select signoff members individually or by project teams or address lists.

- The **perform-signoffs** subtask is then distributed to the selected signoff team, prompting them to review the target objects and signoff.

Caution

Do not add or delete subtasks from the **Review** task. It may cause an error that prevents the task from executing.

When this task is performed in a workflow, the **perform-signoffs** task displays three options to each signoff team member: **Approve**, **Reject**, and **No Decision**. Selecting either **Approve** or **Reject** performs the task. **No Decision** is the default selection, selecting this option does not perform the task.

If you require user authentication before this **Review** task can be performed, add the EPM-require-authentication handler to the **Perform** action of the task. When you implement user authentication for this task, a password box appears below the **Comments** box. Users must type their user password in this box before they can click **Apply** and complete the task.

If a user manually promotes a **Review** task that has both an **Approve** path and **Reject** path using **Manage**  > **Promote Task** on the primary toolbar in the Inbox, then they must select which path the workflow is to follow at that time.

Add Status tasks

Use the **Add Status** task template to create and add a **Release** status to the target objects of the workflow process.

This template is a visual milestone in the workflow process. There is no action for the user to perform, and therefore, no dialog box associated with the **Add Status** task.

Or tasks

Use an **Or** task template to continue the workflow process when any one of its multiple task predecessors is completed or promoted. There is no limit to the number of predecessors an **Or** task may have. Typically, **Or** tasks are used to unite parallel paths create by:

- True/false condition paths branching from **Condition** tasks.
- Parallel links branching from a single task.

This template is a visual milestone in the workflow process. There is no dialog box associated with the **Or** task.

Acknowledge tasks

Use the **Acknowledge** task to define the **Signoff Team** profiles with which a user complies to assign acknowledgment responsibilities to other users. This template also provides the **perform-signoffs** task for the **Signoff Team** members to complete.

Caution

- Do not add or delete subtasks from the **Acknowledge** task. It may cause an error that prevents the task from executing.
- Signoff profiles are unavailable for the **Acknowledge** task if it is a subtask within the **Route** task template. The **Route** task does not function properly if signoff profiles are defined for the subtasks. The **Route** task template is designed to be used as an electronic routing sheet, and the workflow initiator assigns specific signoff members.

When this task is performed in a workflow, the **Acknowledge** task displays two decision commands to members of the selected signoff team: **Acknowledged** and **No Decision**. Signoff team members choose one of the above commands to perform the signoff.

If you require user authentication before this **Acknowledge** task is performed, add the EPM-require-authentication handler to the **Perform** action of the task. When you implement user authentication for this task, a password box appears below the **Comments** box. Users must type their user password in this box before they can click **Apply** and complete the task.

Condition tasks

Use the **Condition Task** template to branch your workflow process according to defined criteria. Because this task template is used to branch workflow process flow, you must always create at least two paths branching off from the task. The paths can be either success paths, failure paths, or a combination of the two.

- Success paths can be either true paths, false paths, or paths with a customized result.
- Failure paths can only be generated from manual **Condition** tasks. They allow an alternate course when a specified task is rejected, a user determines the path cannot be completed, or an error occurs.

Tip

If you use a **Condition** task to branch your workflow process, you can use one or more **Or** tasks later in the workflow process to resolve the paths into a single path.

The system determines which of the branches flowing from a **Condition** task to perform based on the *task result*. The task result is stored in the **Condition** task. The successor tasks have a handler configured with a value that may match the task result. After the task result is set, the successor tasks are examined and any successor tasks containing a value matching the task result are started. Use any of the following methods to set the task results:

- Create a query against the target (automatic only).
 - Create a query against the task (automatic only).
 - Create a query against subprocesses (automatic only).
- If there are multiple subprocesses, a query runs on the associated subprocesses and the results are used to branch accordingly. The query is typically configured to look at the root task's **result** attribute for all the subprocesses.
- If there is only one subprocess, and it is configured to set the result on the **Condition** task, no query is needed, and the workflow follows the branch based on the result.
- Configure the task result from the manual **Condition** task's dialog box.

A **Condition** task can be configured to complete either automatically or manually. You need to determine which configuration is best suited for the workflow process template you are defining. Typically, if a handler can determine the criteria, it is best to configure the task as automatic.

Task	Description
Automatic Condition task	Add an action handler that sets the task's result to true, false, or a customized value. The simplest way to achieve this is to use the task template's interface to define a condition query at

Task	Description
Manual Condition task	design time; this automatically inserts the action handler. Alternatively, you can create a custom action handler that uses ITK to verify criteria. During design, you do not define a query or add an action handler to the task template. Because no query is defined, and no action handler is configured to set the task result, when the workflow process is run, the end user must manually indicate a value using an interactive dialog box. The value chosen by the end user is used to set the task result.

Caution

To ensure desired results, condition tasks that run queries in workflows should always have at least one target object when a condition query is run against workflow targets.

- When a condition task runs a condition query against workflow targets, the system searches the database for that query class and filters the results based on the workflow target objects.
- Because handlers can move objects between targets and references in a workflow, the workflow may have objects in the references folder, but no objects in the targets folder. The condition query will not search in the database if the workflow does not have any targets. This will set a false path of the condition task.

Route tasks

Use the **Route** task as a router sheet with which a user assigns review, acknowledge and notification responsibilities to specified users.

Note

Configure the **WRKFLW_task_complete** preference to use single click functionality for a **Select Signoff Team** route task. Setting the value to **true**, the **Ad-hoc Done** check box is selected by default. This also sets the associated **task_result** property to **Complete** by default.

When this task is performed in a workflow, the **Route** task displays three subtasks: **Review**, **Acknowledge**, and **Notify**. The workflow initiator can then assign other users to perform these tasks. The selected users are the signoff team.

Caution

- Do not add or delete subtasks from the **Route** task. It may cause an error that prevents the task from running.

- Signoff profiles are unavailable for the **Acknowledge** subtask within the **Route** task template. The **Route** task does not function properly if a signoff profile is defined for the **Acknowledge** subtask. The **Route** task template is designed to be used as an electronic routing sheet, and the workflow initiator assigns specific signoff members.

After the **Route** task is performed, the selected signoff team is prompted to perform the **Review** or **Acknowledge** tasks or simply notified of the review through program mail. Notified users do not need to perform any task.

If you want to require user authentication before the **Review** or **Acknowledge** subtasks can be performed, add the EPM-require-authentication handler to the **Perform** action of the subtask (the **perform-signoffs** task of either the **Review** or **Acknowledge** subtasks). When you implement user authentication for either of these subtasks, a password box appears below the **Comments** box. Users must type their user password in this box before they can click **Apply** and complete the task.

If a user manually promotes a **Route** task that has both an **Approve** path and **Reject** path using **Manage**  > **Promote Task** on the primary toolbar in the Inbox, then they must select which path the workflow is to follow at that time.

You can also route or reassign tasks to another user from your inbox, in the same manner as selecting a signoff task. You can select options and designate specific users to notify, acknowledge, or review tasks.

To set up the display of the **Task View** pane, configure the **WORKFLOW_new_route_task_panel** preference. Display choices are **ON** for list box view or **OFF** for option button view.

Validate tasks

The **Validate** task branches a workflow along two or more paths. The path followed is determined by whether specified errors occur during a workflow. Use this task to design workflows around anticipated errors (such as checked out targets), unexpected errors (such as failed scripts or failure of custom handlers), or to track any and all workflow errors.

Configure the **Validate** task by defining one or more *success* and *failure* paths flowing from the task. The success path is followed if no error occurs. The failure path is followed when errors occur.

Note that handlers should be placed on either the **Started** action or the **Completed** action of the Validate task.

When errors occur, you determine if the failure path is followed when:

- Any error occurs.
- Only when an error you specify on a list of error codes occurs.

Note

In the context of the **Validate** task, *workflow error* means any error generated by a workflow handler.

Configure the task to follow a failure path by **pairing** a workflow handler and an error code. Place a handler to be validated on the **Validate** task and then add the respective error code to the path's error list (or set the path to fail on *any* error).

Customize task paths

There are two basic path types between tasks **Success** or **Fail**.

You can modify **Success** paths for **Condition** tasks adding a customized response, instead of the default **True** or **False** values. You cannot modify fail paths.

Procedure

1. Click **Task Palette**  to add a **Success** path from a **Condition** task.
2. Close the **Task Palette** and select the path.
The **Properties** tab displays the specific options relative to the selected item. For **Success** paths from a **Condition** task, the default Properties are **true** and **false**.
3. Enter a new value in the **Add Custom Result** box and click **Add**.
The new result is added to the **Results List**.
4. In the **Results List**, click the list and clear the value that is no longer desired, for instance, **True**.
The new path value is reflected in the workflow template.

Subworkflows

What are subworkflows?

Sometimes you want a workflow process to generate additional workflows as it proceeds. For example, you may want a workflow to generate additional workflows for each target of the parent workflow. This is useful if you want each target to undergo a separate review and signoff process.

Subworkflows (or subprocesses) are child workflows of a parent workflow. For example, users might want to create a subworkflow after receiving a task in their worklist dependent upon the completion of one or more tasks not tracked by the existing workflow. They create a subworkflow to track the additional tasks. In other cases, when working on a task, a user might discover that the opinion of another user would be useful. Creating an ad hoc subworkflow includes that second user as part of the existing workflow.

Subworkflows are created in two locations:

Parent workflow templates Administrators can configure workflow templates to create subworkflows. For example, a parent workflow template can be configured to automatically launch subworkflows for each target of the parent workflow via the **EPM-create-sub-process** handler (see *Start an ad hoc subworkflow* for details).

My Worklist Generally, any user can create an ad hoc subworkflow from a task within their worklist. This functionality is not limited to privileged users.

When you create a subworkflow from an in-process task in your worklist, you create a dependency between the selected task in the parent process and the newly created subworkflow. If a subworkflow is created from an in-process task, the task cannot be completed until the subworkflow is completed.

Regardless of how preferences are set to control the inheritance of target objects from the parent workflow, users can always manually remove targets from subworkflows.

Consider the following when working with subworkflows:

If the parent workflow is dependent on the subworkflow, the parent workflow cannot be completed until the subworkflow is completed.

For example, if the **EPM-create-sub-process** action handler is used to create subworkflows for multiple targets from a parent workflow, the parent workflows are dependent on the subworkflows.

Access to create subworkflows is governed by the Access Manager **Has Class (Task)** rule and the **Task Named** access control list (ACL). The same permissions allowing you to perform the task allow you to create a subworkflow from the task.

This pseudo folder **Independent Sub-Workflows** displays subworkflows identified from the property **epm_subprocesses**. Subworkflows are presented in no particular order, nor can their order be controlled.

Workflow subprocesses can be dependent or independent of parent processes, depending on the preference. Preferences are set by administrators.

Independent of parent process

The **WRKFLW_skip_abort_on_sub_process** preference is honored only for independent subprocesses.

Set the **WRKFLW_skip_abort_on_sub_process** preference to true to skip abort of subprocess when a parent process is aborted.

Dependent upon parent process

If there is a dependency from a parent process to its subprocesses, aborting the parent will abort the dependent subprocesses, irrespective of the value of the **WRKFLW_skip_abort_on_sub_process** preference.

The default value of **WRKFLW_skip_abort_on_sub_process** is false which will abort the subprocesses along with parent process.

The **WRKFLW_parent_behavior_on_sub_process_abort** preference ensures the parent process's task proceeds after the dependent subprocess is aborted.

Related Concepts

[Creating subworkflows for assemblies](#)

[Creating subworkflows for related objects](#)

Related Tasks

[Create a subworkflow](#)

Create a subworkflow

[What are subworkflows?](#)

[Creating subworkflows for assemblies](#)

[Creating subworkflows for related objects](#)

The **EPM-create-sub-process** handler starts subworkflows (also known as subprocesses) from a workflow process. The new subworkflow can take on attachments of the parent process, and these attachments can be grouped by property.

You can add the **EPM-create-sub-process** handler multiple times to a single task, allowing you to use different workflow templates per target object type. Use the handler to:

- Set dependencies between the parent workflow and its subworkflows.
- Define targets and attachments for the subworkflows.
- Transfer attachments from the parent workflow to a subworkflow.
- Create subworkflows for multiple targets.
- Create subworkflows for assemblies.
- Create subworkflows for related objects.

The handler accepts numerous arguments, allowing you to create a wide variety of instances for generating subworkflows. For example:

- The following argument settings create a subworkflow based on the **Clinical Trials Phase I** template, which inherits all the targets and reference attachments from the parent workflow. Because the workflow name is not defined, a workflow name for the child workflow is automatically generated in the format `<parentprocess:count>`.

Argument	Value
-template	Clinical Trials Phase I
-from_attach	ALL
-to_attach	ALL

- The following argument settings launch a subworkflow based on the **Clinical Trials Phase I** workflow template. All item revisions from the parent workflow are excluded as targets for the new workflow.

Argument	Value
-template	Clinical Trials Phase I
-from_attach	ALL
-to_attach	TARGET
-exclude_type	ItemRevision

- The following argument settings launch multiple subworkflows based on the **Clinical Trials Phase I** workflow template. Each item revision that was a target or reference attachment of the parent workflow launches a new subworkflow with that item revision as the target.

For example, if the parent workflow contained three item revisions as targets, three different subworkflows are launched.

Argument	Value
-template	Clinical Trials Phase I
-from_attach	ALL
-to_attach	TARGET
-include_type	ItemRevision
-multiple_processes	

Add the EPM-create-sub-process handler

Add the **EPM-create-sub-process** handler to a task in a workflow template to start a subworkflow.

Procedure

- Select a template from the list.
- Click **Start Edit** .
- Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.

4. Select the task to contain the subworkflow.
5. Click the **Handlers** tab.
6. In the handlers table on the left, select the trigger group to add the handler to: **Started** or **Completed**.
7. Under **New Handler** on the right, click **Action** as the handler type.
8. Select the **EPM-create-sub-process** handler from the drop-down list.
9. Select the name of the template to contain the subworkflow from the drop-down list.
10. Click **Add**.
11. Click **End Edit**  to save your new subworkflow to the template.

Creating subworkflows for assemblies

In workflows that contain assemblies, there are various arguments you can use with the EPM-create-sub-process action handler to create subworkflows for components of the assemblies.

Argument	Behavior
-process_assembly	Searches for assemblies in the target, reference, or all (as specified by the -from_attach argument) and creates subworkflows for each component.
-depth	Specifies the depth to which the assembly is traversed.
-rev_rule	Specifies the revision rule applied to the assembly.
-include_related_type	Creates subworkflows only for assembly components of the types specified in this argument.
-exclude_related_type	Does not create subworkflows for assembly components of the types specified in this argument.

Note

The **-include_related_type** and **-exclude_related_type** arguments can be used in conjunction with each other. If used in conjunction, the **-include_related_type** argument takes precedence; first the objects are processed against **-include_related_type** and then processed against **-exclude_related_type**.

Related Concepts

What are subworkflows?
Creating subworkflows for related objects

Related Tasks

Create a subworkflow

Creating subworkflows for related objects

There are various arguments you can use with the EPM-create-sub-process action handler to create subworkflows for related objects of target and reference data.

Argument	Behavior
-relation	Creates subworkflows for each object attached by the specified relation to the target or reference object. (Specify a particular target, or reference object, or all, using the -from_attach argument.)
-include_related_type	Creates subworkflows only for related objects of the type(s) specified in this argument.
-exclude_related_type	Does not create subworkflows for related objects of the type(s) specified in this argument.

Note

The **-include_related_type** and **-exclude_related_type** arguments can be used in conjunction with each other. If used in conjunction, the **-include_related_type** argument takes precedence; first the objects are processed against **-include_related_type**, and then **-exclude_related_type**.

Related Concepts

What are subworkflows?
Creating subworkflows for assemblies

Related Tasks

Create a subworkflow

3. Creating task templates

Create a task template

A *task template* is a task that you can apply and reuse in workflow templates. These are based on a pre-existing Workflow Designer task. You can then edit that task to offer new capabilities.

The **Task** template is the default template of the EPMTaskTemplate type. Use it as a starting point for creating your own tasks, such as a review task to include a specific set of key roles, or other site-specific tasks for users to complete.

Procedure

1. In the Workflow Designer, go to **More Commands ***** > **New**  > **Create Workflow Template**  from the **Page level** toolbar.
2. Click **Task** as the **Template Type**.
3. Enter a unique **Name** and **Description** (optional).
4. Under **Based On**, select an existing template that the new task must be based on. The **Type** list automatically populates with the task type for your new template.
5. Click **Add**.
The new task is added to the **Templates** panel. You can open it and edit the task properties.
The new task is also available in the **Task Palette** and can be included in workflow templates.

Create custom task templates

What are custom task templates?

You can create your own specific task requirements with a *custom task template*. This is a custom-designed task that you can apply and reuse in workflow templates. It is based on a pre-existing task that you can then edit to create your own.

For example, you may want a task to perform an external third-party authorization. Such a custom task type can be created in BMIDE and then associated with a custom task template. Modifications to the task can be made using the **Handlers** tab, such as adding a custom authentication handler. The new custom task template can then be made available.

Note

Directly modifying the out-of-the-box tasks and paths is not recommended; instead, create custom tasks.

Configuration requirements

Creating a custom task template requires BMIDE configuration before you can start the creation process in the Workflow Designer. Additional configuration is also required to designate an associated icon to the template as well as for configuring the task behavior and adding the custom task template to the Task Palette.

Create new business types for custom task templates

Administrators must add two new business types for the custom task template in BMIDE. Create one for **EPMTaskTemplate** and one for **EPMTask** to be referenced at runtime.

The two new types are:

- **EPM<YOUR_CUSTOM_TASK>TaskTemplate**
Include a subtype of **EPMTaskTemplate**.
- **EPM<YOUR_CUSTOM_TASK>Task**
Include a subtype of **EPMTask**.

Create a custom task template

Procedure

1. In the Workflow Designer, go to **More Commands > New**  **> Create Workflow Template**  from the **Page level** toolbar.
2. Click **Task** as the **Template Type**.
3. Enter a unique **Name** and **Description** (optional).
4. Under **Type**, select the name of the custom type added in BMIDE.
5. Click **Add**.
The newly created task template is added to the list of templates in the **Task Palette**.

Additional configuration is needed to create the task icon and to customize the task behavior in the Workflow Designer.

Configure a custom task template

Once you create the new custom task template in Workflow Designer, you can add an icon and customize its behavior.

Add an icon to a task

Custom icons are a good way to offer better visual representation about where a certain task is in a workflow or what its custom function is. To designate an icon for the new template, see Adding custom type icons in the documentation.

Configure the task behavior

Once the icon has been associated to the new task template, you can specify the task behavior in the Workflow Designer.

Configure Perform UI for the custom task

The file **performTaskPanelConfiguration.json** is instrumental in building your task behavior. This file provides the binding for the out-of-the-box **Perform Task** panels that are displayed in the secondary area binding for the secondary area (in the **Overview** tab) or in the right-side **Tool and Information** command area.

The out-of-the-box configuration JSON file is as follows:

```
{
  "secondaryAreaContribution":
  {
    "Awp0EPMConditionTaskPerform" : {
      "id": "Awp0EPMConditionTaskPerform",
      "priority": 1,
      "performPanelId": "Awp0EPMConditionTaskPerform",
      "condition": "modelObject.type === 'EPMConditionTask'"
    },
    "Awp0EPMDoTaskPerform" : {
      "id": "Awp0EPMTaskPerform",
      "priority": 1,
      "performPanelId": "Awp0EPMTaskPerform",
      "condition": "modelObject.type === 'EPMDoTask'"
    },
    "Awp0EPMSSTTaskPerformSecondary" : {
      "id": "Awp0EPMSSTTaskPerformSecondary",
      "priority": 1,
      "performPanelId": "Awp0EPMSSTTaskPerformSecondary",
      "condition": "modelObject.type === 'EPMSselectSignoffTask'"
    },
    "Awp0EPMRouteTaskPerformSecondary" : {
      "id": "Awp0EPMRouteTaskPerformSecondary",
```

```

        "priority": 1,
        "performPanelId": "Awp0EPMRouteTaskPerformSecondary",
        "condition": "modelObject.type === 'EPMRouteTask'"
    },
    "Awp0SignoffPerform" : {
        "id": "Awp0SignoffPerform",
        "priority": 1,
        "performPanelId": "Awp0SignoffPerform",
        "condition": "modelObject.type === 'EPMPerformSignoffTask' ||
        modelObject.type === 'Signoff'"
    },
    "Awp0EPMTaskPerform" : {
        "id": "Awp0EPMTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMTaskPerform",
        "condition": "( (modelObject.type !== 'EPMConditionTask'
&& modelObject.type !== 'EPMSelectSignoffTask' && modelObject.type !==
'EPMRouteTask' && modelObject.type !== 'EPMDoTask' && modelObject.type !==
'Signoff' && modelObject.type !== 'EPMPerformSignoffTask')) &&
modelObject.modelType.typeHierarchyArray.indexOf('EPMTask') >= 0) "
    }
},
"perfromTaskToolAreaContribution":
{
    "Awp0EPMConditionTaskPerform" : {
        "id": "Awp0EPMConditionTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMConditionTaskPerform",
        "condition": "modelObject.type === 'EPMConditionTask'"
    },
    "Awp0EPMDoTaskPerform" : {
        "id": "Awp0EPMTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMTaskPerform",
        "condition": "modelObject.type === 'EPMDoTask'"
    },
    "Awp0EPMSelectSignoffTaskPerform" : {
        "id": "Awp0EPMSelectSignoffTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMSelectSignoffTaskPerform",
        "condition": "modelObject.type === 'EPMSelectSignoffTask'"
    },
    "Awp0EPMRouteTaskPerform" : {
        "id": "Awp0EPMRouteTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMRouteTaskPerform",
        "condition": "modelObject.type === 'EPMRouteTask'"
    },
    "Awp0SignoffPerform" : {
        "id": "Awp0SignoffPerform",
        "priority": 1,
        "performPanelId": "Awp0SignoffPerform",
        "condition": "modelObject.type === 'EPMPerformSignoffTask' ||
modelObject.type === 'Signoff'"
    },
    "Awp0EPMTaskPerform" : {

```



```

        "id": "Awp0EPMTaskPerform",
        "priority": 1,
        "performPanelId": "Awp0EPMTaskPerform",
        "condition": "( (modelObject.type != 'EPMConditionTask'
&& modelObject.type != 'EPMSelectSignoffTask' && modelObject.type !=
'EPMRouteTask' && modelObject.type != 'EPMDoTask' && modelObject.type !=
'Signoff' && modelObject.type != 'EPMPPerformSignoffTask') &&
modelObject.modelType.typeHierarchyArray.indexOf('EPMTask') >= 0) "
    }
}

```

The correct panel ID is chosen based on the object type as defined in the **Condition** expression of each panel ID. The selected panel ID is set in context, and the respective view is loaded.

Tool and Information area - Perform Task Panel View file

The configuration of the **Perform Task Panel View** file (Awp0PerformTaskPanelView.html, in the Tool and Information area) is as follows:

```

<aw-perform-command-panel caption="i18n.performTask">
  <aw-perform-panel-include
name="{ctx.performTaskCtx.activePerformTaskPanelId}"
exist-when="ctx.performTaskCtx.activePerformTaskPanelId"></aw-perform-panel-include>
</aw-perform-command-panel>

```

Secondary area - Perform Task Panel View file

The configuration of the **Perform Task Panel View** file (Awp0PerformTaskView.html, in the secondary area) is as follows:

```

<aw-panel id="Awp0PerformTask" exist-when="data.isTaskPerformable">
  <aw-perform-panel-include
name="{ctx.performTaskCtx.activePerformTaskPanelId}"
exist-when="ctx.performTaskCtx.activePerformTaskPanelId"></aw-perform-panel-include>
</aw-panel>

```

User customization

Adding a custom perform UI requires:

- Creation of a custom module.

- Addition of the file **performTaskPanelConfiguration.json** to the **viewModel** folder of the custom module

Use case 1: Include a specific panel for a custom type in a custom task

You can include a specific panel for a custom type in a custom task. For this, you must include the **performTaskPanelConfiguration.json** file in your custom module.

Example of a custom **performTaskPanelConfiguration.json** file:

```
{
  "secondaryAreaContribution":
  {
    "Awp0Fnd2CustomTaskPerform" : {
      "id": "Awp0Fnd2CustomTaskPerform",
      "priority": 2,
      "performPanelId": "Awp0Fnd2CustomTaskPerform",
      "condition": "modelObject.type === 'Fnd2CustomTask'"
    }
  },
  "performTaskToolAreaContribution":
  {
    "Awp0Fnd2CustomTaskPerform" : {
      "id": "Awp0Fnd2CustomTaskPerform",
      "priority": 2,
      "performPanelId": "Awp0Fnd2CustomTaskPerform",
      "condition": "modelObject.type === 'Fnd2CustomTask'"
    }
  }
}
```

In this example, the **Awp0Fnd2CustomTaskPerform** panel is added for the custom type **Fnd2CustomTask**. The custom panel ID can be any value, and the corresponding panel is loaded if the condition matches the type of the custom task (**Fnd2CustomTask** in this example). If no custom panel ID is present, then the default view displays as the generic EPM TaskPerform panel.

Use case 2: Display your custom panel for an object

This example explains how to display your own custom panel for the **EPMDoTask** object. In this case, you may require an additional field to display information about the task. You can create this customization by passing the higher priority in your custom .json configuration file as shown here:

```
{
  "secondaryAreaContribution":
  {
    "Awp0EPMDoTaskPerform1" : {
      "id": "Awp0EPMTaskPerform",
      "priority": 2,
```

```

        "performPanelId": "Awp0EPMConditionTaskPerform",
        "condition": "modelObject.type === 'EPMDoTask'"
    },
    "perfromTaskToolAreaContribution":
    {
        "Awp0EPMDoTaskPerform1" : {
            "id": "Awp0EPMTaskPerform",
            "priority": 2,
            "performPanelId": "Awp0EPMConditionTaskPerform",
            "condition": "modelObject.type === 'EPMDoTask'"
        }
    }
}

```

The provided task type has a default priority of 1, but greater integer values have a higher priority. You can override this by providing a higher priority to handle that case. In this example, **Awp0EPMConditionTaskPerform** is displayed for a Do task where the registered type is **Awp0EPMTaskPerform**.

Use the custom task created in Active Workspace in the rich client

If you create a custom task in Active Workspace and plan on using that task in the rich client, additional steps are required. See *Creating custom templates* in the Teamcenter help on <https://support.sw.siemens.com> for instructions.

4. Editing workflow templates

Determining which editing options to use

Perform edits on existing workflow process templates by selecting the template to be edited and clicking the **Edit Mode**  button.

Consider the following questions before editing a workflow template.

Editing task	Description
Edit offline or online?	Offline editing prevents users from accessing the workflow template while you edit. Use this option when you do not want the old version of the workflow template available for use until your edits are complete. Online editing allows users to initiate workflows based on the old version of the workflow template, while you edit a copy of the same template. When you switch the edited version to the Available stage, the older copy is overwritten; only the edited copy remains available from the interface.
Apply edits to running workflow processes?	After editing a workflow template, you can apply the edits to all active processes that are based on the template. When you select the Set Stage to Available check box, the Apply Template Changes dialog box asks whether to apply the edits to all active workflow processes based on the template. Select the Apply template changes to all active workflow processes check box to update each active workflow process based on the workflow template as follows: • If the edits in the workflow template occur <i>later</i> in the workflow than the active workflow process has reached, the edits are applied to the workflow. • If the edits in the workflow template occur <i>earlier</i>, and the active workflow has already passed the place where the edits were made, the edits do not take effect unless the task or path is re-run (using backward branching or loops) or the task is demoted. • If the edits in the workflow template impact an active task, the edits are applied after the task completes and take effect only if the task is re-run.

Editing task	Description
Which workflow components can be edited?	• If the edits delete the currently active task, the next task is started. You can edit any aspect of the workflow process template, including: • Changing the template name • Adding and removing tasks • Adding, deleting, redrawing, and resetting flow paths • Adding, deleting, and resetting handlers, attributes, task attributes, and attachments

Offline versus online editing

Deciding whether to edit a workflow template online or offline is determined by whether you want to grant users access to the existing version of the workflow template while you edit it.

- Offline editing prevents users from accessing the workflow template while you edit it. Use this option when you do not want the old version of the template available until your edits are complete.

With this option, there is only one instance of the template. The system sets the workflow template to the **Availability: Offline** stage. The template is not available to users initiating workflow processes against objects; it does not appear in the **Process Template** list.

- Online editing allows users to initiate workflows based on the existing version of the workflow template while you edit a copy of the same template.

The system makes a copy of the workflow template and sets it to the **Availability: Offline** stage; this is the version you edit. Both versions of the workflow template appear in the **Process Template** list. The **Availability: Offline** symbol appears next to the version being edited.

Users can continue to use the existing version of the workflow template. When you switch the edited version to the **Available** stage, the existing copy is overwritten; only the edited copy remains available.

Note

Users who do not have the appropriate rights or role will only see **Online** templates.

Identify templates as non-modifiable

Users identified by the **Awp0IsPrivilegedWorkflowTemplateAuthor** BMIDE condition as privileged can specify templates as requiring administrative privileges for modification so that other users cannot modify them in the Workflow Designer.

Using the **Edit** mode in Workflow Designer, you can modify existing templates or templates you are actively developing to be read-only.

Procedure

1. [Create a new template](#), or select a workflow template from the list and click **Start Edit** .
2. Ensure that no task is selected in the template. On the **Properties** tab, click the **Require admin privilege for modification** check box.

Show Task in Process Stage List:	☒
Process in Background:	☐
Select Participants from Workflow:	☐
Require Task Confirmation on Complete:	☐
Require admin privilege for modification:	☐

Once saved, templates are non-modifiable and can only be edited by privileged users who are identified based on the BMIDE condition **Awp0IsPrivilegedWorkflowTemplateAuthor**. All other users cannot access the **Start Edit** button once the read-only template is open in the Workflow Designer.

Edit workflow templates

Edit a template

Using the **Edit** mode in Workflow Designer, you can modify existing templates or templates you are actively developing.

Procedure

1. Select a workflow template from the list and click **Start Edit** . Offline templates are indicated by the **Availability: Offline** icon .
2. The template is offline and ready for editing. In the **Properties** tab, you can change the template name, add or modify instructions, apply filter conditions, and other parameters.

Note

You cannot rename a template with an identical name already in use. If a name is in use, an error message appears.

3. When you finish editing, click **End Edit** . This saves your changes, but keeps the template offline.

Click **Online**  to make the template available.

4. If the Workflow Designer is configured to accommodate applying template changes to active processes, additional prompts display; see [Apply Templates to Active Processes](#) for more information.

Apply template edits to active processes

After editing a workflow template, you can make the template available to users and apply the edits to active processes based on the template. This action modifies all the template's active processes simultaneously.

The setting configured in the **EPM_enable_apply_template_changes** option determines how and when the processes are applied. Only the **OPTIONAL** or **AUTOMATIC** values apply edits to active processes.

Once you click **Online**  to make the template available, you are prompted based on your preferences.

NONE

Any changes are automatically applied to the template but not to any active workflows. This is the default value.

OPTIONAL

Applies workflow template edits to active workflows based on each selected workflow template.

This setting lets you choose on a case-by-case basis whether to apply workflow template edits to active workflows based on the workflow template.

Yes:

At the prompt, if you click **Yes**, your edits are applied to all active workflows based on that workflow template. The request is processed in the background if Dispatcher is configured. Otherwise, changes are applied to the foreground. A confirmation message provides details on the number of workflows that were successfully updated and the number of workflows that failed.

No:

If you click **No** at the prompt, the changes are not applied to active workflows.

AUTOMATIC

All changes are automatically applied to active workflows.

If Dispatcher is configured, the template changes are automatically applied in the background, without any prompting.

If Dispatcher is not configured, the template changes are applied directly to the foreground and a confirmation message displays how many workflows (if any) include the changes to the template.

Note

Dispatcher must be enabled and configured for asynchronous processing.

Edit the template layout

There are multiple layout options for a template.

The preference **WRKFLW_preferred_diagram_layout** controls the preferred layout. This setting determines which layout appears when you first display a template. It applies this layout to both the Workflow Designer and Workflow Viewer.

To edit the layout of a workflow template, do the following:

Procedure

1. Open the template that you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click **Apply Layout** and select a layout option.

Auto Layout includes:

- Top-to-Bottom
- Right-to-Left
- Left-to-Right

Fixed Layout

This layout option displays the task (and subtask) node size and location as they were last saved in the rich client or Active Workspace. This also lets you view legacy template layouts as they were originally designed in the rich client. While editing a template, you can manually set the location of the template nodes. Fixed Layout node positions are automatically saved as they are added or moved.

Switching from Auto Layout to Fixed Layout

When switching from Auto Layout to Fixed Layout, the previously saved task node locations are displayed. For templates that have just been created, the nodes in the Fixed Layout are initially the same as what they were in Auto Layout.

Switching from Fixed Layout to Auto Layout

When switching from Fixed Layout to Auto Layout, the task node locations generated by the automatic positioning display. The layout changes made to the nodes are not saved unless you click **Save Layout**. Doing this overwrites all pre-existing location data for the template.

Add filter conditions to templates

Filter conditions set on a template control who can see and use specific templates. Administrators define the conditions and set their parameters. Siemens Digital Industries Software recommends that you use Business Modeler IDE conditions to associate templates with a target object type and a user group. Conditions offer greater versatility, with criteria such as session group, role, and user; target project and target release status; and custom criteria, both session-specific and target-specific, that an administrator can create.

See Use conditions to filter workflow template availability in the BMIDE guide.

To add filter conditions to a template:

Procedure

1. Select a template and click **Start Edit** .
2. To access the **Filter Conditions**, either select the template background or the **Start** task.

Note

Filter conditions are not displayed in the **Properties** tab if a task is selected.

3. Select one of the predefined conditions from the **Filter Conditions** list.
4. Click **End Edit**  when all your template changes are complete.

Filtering conditions

The preference **CR_allow_alternate_procedures** manages the visibility of workflow templates in the **Submit to Workflow** panel.

Values include:

Any: The **All** option is pre-selected and all templates are listed when a user submits a workflow.

Assigned: This is the default value. The **Assigned** option is pre-selected when a user submits a workflow. Only templates that match the condition criteria (as defined in the BMIDE condition) are displayed.

None: Only valid templates that match the condition criteria (as specified in the BMIDE condition) are displayed. The options **All** and **Assigned** are not visible when a user submits a workflow.

Example

For example, define the following expression to filter workflow templates while initiating a workflow for a problem report.

```
((o !=null) AND
u.fnd0ConditionHelper.fnd0isSubTypeOf(o, "ProblemReportRevision")) OR ((t!
=null) AND u.fnd0ConditionHelper.fnd0isSubTypeOf (t, "ProblemReportRevision
```

The condition expression validates when the object submitted is not null and the object type is **ProblemReportRevision**.

Adding targets after submitting a workflow

It may be necessary to add additional targets after a workflow has started. This is still governed by ACL access on the task first. If you have permission then you can add additional targets.

- If the **CR_allow_alterate_procedures** preference is set to **any** or **Assigned**, there is no restriction on what type of target can be added.
- If the **CR_allow_alterate_procedures** preference is set to **none** and **WRKFLW_allow_adding_target_behavior** is set to **1** (default value), there is no restriction on what type of target can be added.
- If the **CR_allow_alterate_procedures** preference is set to **none** and **WRKFLW_allow_adding_target_behavior** (explained as follows) is set to **0**, targets can be added that satisfy the condition criteria of the template.

To allow targets to be manually added or pasted at a specific task level, use the preference **WRKFLW_allow_adding_target_behavior**. This is useful when adding a target after a workflow is initiated. Values include:

- **0:** Allows specific target(s) that satisfy the **fnd0FilterCondition** condition when the value of **CR_allow_alterate_procedures** preference is set to **none**.
- **1:** Allows any target(s). This is the default value.

Configure a default workflow template

Preference considerations

You can create a default template for a select set of object types in Workflow Designer. This template is then automatically selected when a user submits an existing object to a new workflow.

Workflow Designer automatically creates a preference, **WorkspaceObject_default_workflow_template**, that associates the template with that class. This preference includes the value **WorkspaceObjectDefaultWorkflowTemplate**. This means that all users can see the default workflow template as **WorkspaceObjectDefaultWorkflowTemplate** for all workspace objects.

Note the following:

- You can associate multiple classes with a template. A preference is created for each class type.
- If an existing preference already exists, and you choose that same class on a new template, the preference is updated with the new template name.
- If you manually add or update a preference, the template reflects the change in Workflow Designer.
- If you delete a preference that the template was associated with, the preference no longer shows the association in Workflow Designer.

Configure a default workflow template for an object

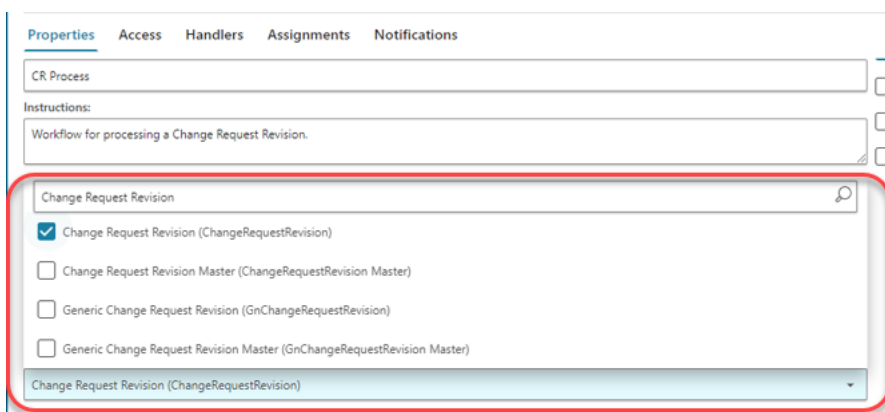
Procedure

1. Select a template and click **Start Edit** .
2. To access the default types for the template, either select the template background or the **Start** task.

Note

Types are not displayed in the **Properties** tab if a task is selected.

3. Click the **Default for Types** list and select a class to associate with the default template.



4. Click **End Edit**  when all your template changes are complete. When the **Submit to Workflow** panel is brought up for an object, the default template is displayed at the top of the list. The rest of the templates are sorted alphabetically.

Configure a default workflow template specifically for groups and subgroups

In case users from different groups want to see a different default template for an object, you can define groups/subgroups for a template.

Note

If a preference specifies a group, all of its subgroups are also included. However, you can identify specific subgroups for a default template using this method.

Two methods are available to support this feature.

- You can use the existing preference **WorkspaceObject_default_workflow_template**. This preference can be enhanced to support multiple values with the format **GroupName::Template Name** or **Subgroup.Group::TemplateName**.
- If it is required to specify default Workflow templates for groups without a default preference (for example, ItemRevision), then you need to create a new preference.

Enhance an existing preference

If it is required that all users belonging to a group or subgroup (such as the **Engineering** group and subgroups of **Engineering**) see a different value than the default template, do the following:

1. From the Launcher, click **Preferences**.
If this option is unavailable, search for it. You can pin it from the search results for future access.
2. Locate the preference associated with the class type.
3. Click **Edit** .
4. Click **Yes** under **Multiple Values**.

The screenshot shows a configuration window titled "WorkspaceObject_default_workflow_template". It has a toolbar with "Cancel", "Save", and other icons. The window is divided into two main sections: "Definition" and "Values".

Definition Section:

- Name:** WorkspaceObject_default_workflow_template
- Product Area:** General (dropdown menu)
- Description:** Defines the default workflow template to be used for submitting Workspace Objects to the workflow. (text area)
- Protection Scope:** User (dropdown menu)
- Environment:**
 - ☐ Enabled
 - Location:** Site
 - Type:** String
 - Multiple Values:** ☒ Yes

Values Section:

- Values:** WorkspaceObjectDefaultWorkflowTemplate (text input with a clear button)
- Click to add a value** (placeholder text)

- Under **Values**, add a new entry to the preference as **GroupName::TemplateName** or **Subgroup.Group::TemplateName**.

The screenshot shows the "Values" section of the configuration window. It displays three entries in a list:

- Workflow Template
- Engineering: Workflow Template
- DBA.Engineering: Workflow Template

Each entry has a small "X" icon to its right. Below the list is a light blue area with the text "Click to add a value".

- Click **Save** .

Create a new preference

If it is required to specify default Workflow templates for groups without a default preference (for example, **ItemRevision**), then you must create a new preference. If such a preference exists, then see the previous section.

1. From the **Launcher**, click **Preferences**.

If this option is unavailable, search for it. You can pin it from the search results for future access.

2. Go to **More Commands ***** > **New** ✨ > **New Preference** from the **Page level** toolbar.

3. Enter the internal class name under **Name**. For example, **ItemRevision_default_workflow_template**.

4. Enter the required fields, and click **Yes** under **Multiple Values**.

5. Enter the default template name first, then press **Enter**.

6. To add additional templates for groups or subgroups, enter the values individually.

For example, **Engineering::Workflow Template** includes the Engineering group and all of its subgroups.

dba.Engineering::Workflow Template would only include the **DBA** subgroup.

7. Click **Add**.

In the Workflow Designer, when you select the specific template, the class name will be populated under the **Default for Types** list in the **Properties** tab.

When the **Submit to Workflow** panel is brought up for an object, the default template is displayed at the top of the list for those specific groups/subgroups. The rest of the templates are sorted alphabetically.

5. Managing templates in Workflow Designer

Importing and exporting templates in Workflow Designer

Import workflow templates

You can import workflow templates into the Teamcenter database from an exported workflow template file. Importing templates is useful for transferring workflow templates between different Teamcenter sites. The templates must first be exported from a Teamcenter database into an export file, after which you can import the file into the Teamcenter database at another site.

Procedure

1. Go to **More Commands *** > Import/Export**  > **Import**  from the **Page level** toolbar. The system displays the **Import** panel.
2. Select **Choose File** and navigate to the directory containing the exported template.
3. (Optional) If you want the system to continue the transfer if one or more workflow templates fail to transfer, select the **Continue On Error** check box. If one or more workflow templates fail to transfer, the system records transfer errors in its log files, bypasses the failed workflow templates, and transfers the remaining workflow templates.

If you do not select this option, the system stops the transfer process if one workflow template fails to transfer and only includes in the transfer those workflow templates that transferred successfully.

4. (Optional) If you want the system to overwrite any workflow template of the same name that already exists in the database, select the **Overwrite Duplicate Templates** check box. The system does not display or log any errors.

Select this option when the imported workflow template contains changes that you want applied to the database.

For example, you have added two custom tasks to a workflow template and thoroughly tested the revised template in your test database. Now you are ready to import the changes to the production database. By choosing to overwrite duplicate templates when importing the workflow template to the production database, you are effectively editing the workflow template. On import, the original workflow template is overwritten by the importing workflow template; it now contains the two custom tasks.

If you do not select this option, any importing template with the same name as an existing template is ignored and the import process continues. A message is logged that a workflow template of the same name exists.

5. (Optional) If you chose to overwrite duplicate templates, you can also choose to ignore the origin ID of the template you are importing by selecting the **Ignore origin ID** check box.

Select this option if you get the following error when attempting to import workflow templates:

```
The importing template(s) do not match with the existing template(s).  
The import of template(s) in overwrite mode failed.
```

6. Click **Import** to import the templates contained within the file you selected into the Teamcenter database.
The imported template names now exist in the database and appear in the template list.

Export workflow templates

You can export workflow templates from the Teamcenter database in XML format, storing the templates in a single export file. After exporting the templates, you can import the file into the Teamcenter database at another site. You can also search the XML to determine handler and argument usage.

A template that is offline and under construction can be exported and then imported into another Teamcenter environment. The offline state is retained during the import process.

Procedure

1. Select one or more templates to export.
2. Go to **More Commands *** > Import/Export**  **> Export**  from the **Page level** toolbar.
The **Export Workflow Templates** panel appears.
3. Specify the name of the export file in the **File Name** box, for example, **template_export**.

Note

The export file location is your default downloads folder of your browser.

4. (Optional) If you want the system to continue the transfer if one or more templates fail to transfer, select **Continue on error**. If one or more templates fail to transfer, the system records transfer errors in its log files, bypasses the failed templates, and transfers the remaining templates.

If you do not choose this option, the system stops the transfer process if one template fails to transfer and only includes in the transfer those templates that transferred successfully.
5. Click **Export** to export the templates.
The selected templates are exported in XML format to the file name you defined in step 3.

Select and compare workflow templates

You can view, select, and compare templates from the Workflow Designer dashboard.

Procedure

1. Select any template in the list to display the workflow properties.
2. Select multiple templates by holding down the **Control** key to select specific templates, or use the **Shift** key to select a range of templates.
You can compare them for specifics such as **Owner** or **Date Modified**.
3. Select **Settings**  > **Arrange** to customize the workflow criteria available for comparison.
4. To remove the selections, deselect each template.

Delete templates

If a template is no longer used or becomes obsolete you can permanently delete it from the system.

Caution

Deleting templates is permanent and cannot be reversed.

Procedure

1. Select one or more templates to remove from the system.
2. Go to **More Commands ***** > **Edit** > **Delete**  from the **Page level** toolbar.
3. Choose **Delete** from the dialog box to permanently remove the template.

Distributing workflow templates using Multi-Site Collaboration

Replicate a workflow template

You can distribute your workflow templates to different Teamcenter sites by replicating templates using **Multi-Site Collaboration**. You can replicate your workflow templates, including those under construction, on several Teamcenter sites by using the **data_share** utility and update them with the **data_sync** utility. You cannot edit the replicas, only the template at the owning site. Also, handlers attached to the templates must exist at all sites where the templates are replicated.

Procedure

1. If necessary, [create the template](#) that you want to replicate.
2. Run the **data_share** utility with the following arguments:

```
data_share -u=<user-id> -p=<password> -g=<group> -f=send -site=<remote-site-name1>
-name=<workspace-object>-class=<class-name>
```

For example, if you want to replicate the **demotemplate** workflow template at the **teamcentersite2** site, run the following utility command (the required logon information is omitted from the example):

```
data_share -f=send -site=teamcentersite2 -name=demotemplate  
-class=EPMTaskTemplate
```

Note

- If you want to transfer ownership to the specified site, add the **-transfer** argument to the command.
- If you want to import the template at another site to the current site, change the **-f** argument to **-f=remote_import**.
- If you want to replicate the template at more than one site, add more **-site** arguments to the command.
- If you want to replicate several templates, type the template names in a text file and replace the **-name** and **-class** arguments with the **-filename** and **-classoffile** arguments, respectively.

Results

The replicate template appears at the new site with the  symbol.

Synchronize replicated templates

You can use Multi-Site Collaboration to synchronize replicated templates at remote sites.

Procedure

1. Update the template at the owning site that is replicated at another site.

Note

If you want *active* workflows based on the synchronized template to be updated at the replica site, set the **WRKFLW_multisite_apply_template_changes** preference to **true**.

2. Run the **data_sync** utility with the following arguments:

```
data_sync -u=<user-id> -p=<password> -g=<group> -f=sync -site=<remote-site-name1>  
-class=<class-name> -update
```

For example, if you changed the **demotemplate** workflow template and wanted to update the replica at the **teamcentersite2** site, run the following utility command (the required logon information is omitted from the example):

```
data_sync -f=sync -site=teamcentersite2 -class=EPMTaskTemplate -update
```

Note

If you want to synchronize the template at more than one site, add more **-site** arguments to the command.

Results

The replicate template is updated at the specified sites.

6. Managing Task Behavior

Delete a task

Procedure

1. Open the template containing the task you wish to delete, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task that you want to delete.
4. Click **Remove** .
5. Click **End Edit**  to save your template.

The selected task and any attached links are deleted.

Modifying tasks

Modify tasks in Workflow Designer

Modify task behavior within a workflow process template using attributes, access control lists (ACLs), handlers, notifications, and properties.

Procedure

1. Select a template from the list.
2. Click **Start Edit** .
3. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
4. Select the relevant task in the workflow.
Modifications can be made to the tasks through the **Properties**, **Access**, **Handlers**, **Assignments**, **Notifications**, and **Forms** tabs.

Access Control Lists

Access control lists (ACLs) contain a list of privileges granted, denied, or not set for each accessor. In the Workflow Designer, you can create and apply **Workflow** ACLs or select and apply **System** ACLs to a task.

- A **Workflow** ACL is specific to a workflow. Objects can use a Workflow ACL to define access and determine what operations can be performed on it. For example, you could create a workflow ACL when you want to provide permission for a user to promote a task, or for approvers to gain write access to the target object. Workflow ACLs can stay in effect for the task until the same workflow sets another ACL later in the process or the workflow is completed.
- A **System** ACL is one that has been already defined outside Workflow Designer. It can be applied to the task and all subsequent tasks in the workflow unless it is changed by another instance of the **EPM-set-rule-based-protection** handler or the process is completed. You can select and apply a **System** ACL in the Workflow Designer, but information about existing System ACLs is read-only.

To create a System ACL, from the **Launcher**, click **Access Manager**.

If this option is unavailable, search for it. You can pin it from the search results for future access.

How ACLs work with Access Manager

In Workflow Designer, the **Access** tab lets you create and apply a Workflow ACL or configure a task with a specific System ACL for Access Manager-enabled systems.

ACLs work in parallel with the **EPM-set-rule-based-protection** handler, which indicates that an ACL will be passed to Access Manager. Access Manager will then apply access as defined by the ACL. Although **EPM-set-rule-based-protection** will indicate an ACL, it does not apply the ACL. Access Manager picks up the ACL and applies and enforces it. The ACL set by **EPM-set-rule-based-protection** is not exposed to Access Manager until a task's state has been set to **Started**. A task's state does not transition to Started until all the handlers on the Start action execute successfully. This means that other handlers that are located on the same Start action as **EPM-set-rule-based-protection** will not execute under the access indicated by **EPM-set-rule-based-protection**. In order for other handlers to adhere to the access indicated by **EPM-set-rule-based-protection**, they can either be placed on the Complete action of the current task or the Start action of a successor task. It is important to understand this concept because some handlers rely on access, and therefore a proper configuration is required to ensure the intended access is being applied when the handlers execute.

Example

For example, a desired configuration may be to have all the following applied on a single task:

EPM-set-rule-based-protection indicates an ACL.

Access Manager then picks up the ACL and applies the access.

Finally, **EPM-attach-related-objects** executes based on the access of the ACL that was indicated by **EPM-set-rule-based-protection**.

Since the ACL will not be applied until the task starts, to achieve the desired behavior, the **EPM-attach-related-objects** should be placed on the Complete action.

In this example, the configuration should look like this:

- Task Start handler: **EPM-set-rule-based-protection**
- Task Complete handler: **EPM-attach-related-objects**

With this configuration the processing will execute as follows:

1. Current task state is Pending.
2. Task is triggered to start.
3. Handlers on the Start action will execute, which in this example is the **EPM-set-rule-based-protection** handler, and **EPM-set-rule-based-protection** will indicate an ACL.
4. After the handlers on the Start action execute successfully, the task state is set to Started. The indicated ACL will now be applied (and this access will remain until a different access is set).
5. Handlers on the Complete action will execute, which in this example is the **EPM-attach-related-objects** handler.
6. The **EPM-attach-related-objects** executes under the intended ACL, which is the ACL indicated by **EPM-set-rule-based-protection** in step 3.

In Workflow Designer, the **Access** tab lets you create and apply a Workflow ACL or configure a task with a specific System ACL for Access Manager-enabled systems.

Create a Workflow ACL

A Workflow ACL is specific to a workflow. It can stay in effect for the task until the same workflow sets another ACL later in the process or the workflow is completed. In Workflow Designer, use the **Access** tab to create a Workflow ACL.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the **Access** tab.
4. Under **Access Type**, click **Workflow**.
5. Under **Named ACL**, click **Manage ACL** . The **Manage ACL** panel opens.

6. Under **Create New ACL**, enter the new **ACL Name** and click **Create**.
The new workflow ACL instance is created but accessors and privileges are not assigned yet.
7. Ensure the correct task is selected in the workflow for the ACL. If no task is selected, the ACL is applied to the root task.
8. Select the new ACL from the **ACL Name** drop-down list.
9. Under **Object Access Control List**, click **Add** .
A new row of access control entries (ACEs) are available to define the privileges for the new workflow ACL.
10. Select the **Accessor Type** from the drop-down list.
11. (Optional) Select the **Accessor** from the drop-down list. Accessors are collections of users who share certain common traits, such as membership in the group that owns the object or membership in the project team. Some Accessor Types do not require an Accessor.

For example, if you chose the **Accessor Type** of **Participant**, the list of **Accessors** would display every type of Dynamic Participant available.
12. Set privileges by clicking the cell corresponding to the privilege you want to set, and choose to grant privileges  or deny privileges . Hover over the cell to identify the icons.

Click **Allow All**  to grant access for all entries or **Deny All**  to restrict access to all entries. **Reset**  returns the selected row to the last saved values for the privileges, while **Clear All**  clears all the cells for the selected row.

At least one privilege must be granted or denied for the **Object Access Control List** table.

Note

Whenever possible, do not explicitly set privileges. Leaving privileges unset allows rules to accomplish focused objectives by allowing objects and accessors to filter through rules that do not apply to them.

13. Click **Save**.
The ACL is applied to the selected task.

Workflow

Workflow Designer interface showing the **Access** tab. The workflow diagram consists of the following steps: Start → Complete the work → Review the work (with a count of 2) → Acknowledge the work (with a count of 2) → Finish.

TASK TYPES

- Acknowledge Task
- Add Status Task
- Condition Task
- Do Task
- Or Task
- Review Task
- Route Task
- Task
- Validate Task

Properties Access Handlers Notifications Assignments List Arrangements

OBJECT ACCESS CONTROL LIST

Accessor Type	Accessor	View	Edit	Delete	Share	Permissions	Actions	Tools
Responsible Party		☒	☒	☐	☒	☐	☐	☐

Save

The Workflow Designer automatically creates and updates the **EPM-set-rule-based-protection** handler with the new ACL once the edits are saved.

Properties Access Handlers Notifications Assignments List Arrangements

Name: Complete the work

Trigger	Arguments
Assigned	
Started	
EPM-set-rule-based-protection... -acl=New ACL	
Performed	
Completed	
Promoted	
Suspended	
Resumed	
Aborted	
Undo	

Note

If you create or update the **EPM-set-rule-based-protection** handler from the **Handlers** tab, the **Access** tab automatically reflects those changes.

Modify a Workflow ACL

A **Workflow** ACL is specific to a workflow. Once you create a **Workflow** ACL, you can modify its settings.

See [Create a Workflow ACL](#) for more information about how to create an ACL.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Access** tab.
5. Modify privileges by clicking the cell corresponding to the privilege you want to edit, and choose to grant privileges  or deny privileges . Hover over the cell to identify the icons where necessary.

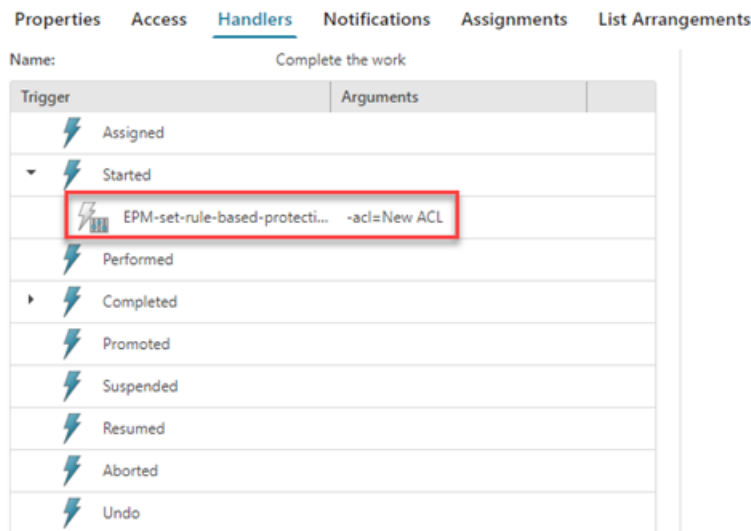
Click **Allow All**  to grant access for all entries or **Deny All**  to restrict access to all entries. **Reset**  returns the selected row to the last saved values for the privileges, while **Clear All**  clears all the cells for the selected row.

At least one privilege must be granted or denied for the **Object Access Control List** table.

Note

Whenever possible, do not explicitly set privileges. Leaving privileges unset allows rules to accomplish focused objectives by allowing objects and accessors to filter through rules that do not apply to them.

6. Click **Save** and the **Workflow** ACL is updated.
The Workflow Designer automatically creates and updates the **EPM-set-rule-based-protection** handler with the new ACL once the edits are saved.



Delete a Workflow ACL

When you delete a Workflow ACL from a task in a template, the Workflow Designer automatically deletes the associated **EPM-set-rule-based-protection** handler from the task once the edits are saved.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task that contains the Workflow ACL to delete.
4. Click the **Access** tab.
5. Under **Named ACL**, click **Manage ACL** . The **Manage ACL** panel opens.
6. Under **ACL**, select the ACL to delete from the list and click **Delete** .
7. At the prompt, click **Delete**.
A message confirms that the ACL was deleted. The associated **EPM-set-rule-based-protection** handler for the task is also removed from the **Handlers** tab. Any other tasks that use the deleted ACL will have the name removed from the **Access** tab, but the handler remains.

Import a template that includes an ACL on a task

When you import a template that includes an ACL on a task(s), the ACL name, or instance, imports but its associated accessors and privileges do not. The associated **EPM-set-rule-based-protection** handler remains on each task in the imported template, but you must create and redefine the ACL details for the task on the **Access** tab for the ACL to apply to all tasks. Once you create and redefine the privileges for one imported ACL, the other tasks in the template that included the ACL are automatically populated with the same details.

Procedure

1. **Import the template** that contains the ACL.
2. Select a task that contains an ACL and click the **Access** tab.
Note that the ACL is not selected on the task once it imports.
3. Under **Named ACL**, click **Manage ACL** .
The **Manage ACL** panel opens.
4. Under **Create New ACL**, enter the **ACL Name** and click **Create**. Use the same ACL name as that in the imported handler.
The new workflow ACL instance is created but accessors and privileges are not assigned yet.
5. Select the newly-created ACL from the **ACL Name** drop-down list.
6. Under **Object Access Control List**, click **Add** .
A new row of access control entries (ACEs) are available to define the privileges for the new workflow ACL.
7. Select the **Accessor Type** from the drop-down list.
8. Select the **Accessor** from the drop-down list.
9. Set privileges by clicking the cell corresponding to the privilege you want to set, and choose to grant privileges  or deny privileges . Hover over the cell to identify the icons.
10. Click **Save**.
The ACL is applied to the selected task and all subsequent tasks that the ACL applied to in the imported template.

Specify an existing ACL for a task

In Workflow Designer, use the **Access** tab to configure a task with a specific ACL for Access Manager-enabled systems. You can specify to use either a **Workflow** or **System** ACL.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:

- Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to apply the ACL to.
 4. Click the **Access** tab.
 5. Click the **type of ACL** to apply:
 - **System**
 - **Workflow**
 6. Under **Named ACL**, click the **ACL Name** drop-down list to select a specific ACL.
 7. Click **Save** to save the changes to the workflow.

The Workflow Designer automatically creates and updates the **EPM-set-rule-based-protection** handler once the edits are saved.

Work with task handlers

Handlers are the lowest-level building blocks in a workflow. They are small ITK programs used to extend and customize tasks. There are two kinds of handlers:

- Action handlers extend and customize task actions. They perform such actions as displaying information; retrieving the results of previous tasks (inherit); notifying users; assigning users, reviewers, dynamic participants; setting object properties and protections; and adding release status and trigger system commands.
- Rule handlers integrate workflow business rules into workflow processes at the task level. They attach conditions to a task action. Rule handlers confirm that a defined rule has been satisfied. If the rule is met, the handler returns the **EPM_go** command, allowing the task to continue. If the rule is not met, it returns the **EPM_nogo** command, preventing the task from continuing. If there are multiple targets for a single rule handler, all targets must satisfy the rule for **EPM_go** to be returned (**AND** condition).

Rule Quorums

Many conditions defined by a rule handler are binary (that is, they are either true or false). However, some conditions are neither true nor false. EPM allows two or more rule handlers to be combined using logical **AND/OR** conditions. When several rule handlers are combined using a logical **Or** condition, rule quorums specify the number of rule handlers that must return **EPM_go** for the action to complete.

Action and rule handlers in the **Handlers** panel can be copied:

- From one action to another action in a task.
- From one task to another task in the same template.
- From a task in one template to a task in another template.

A full list of action and rule handlers and their definitions is available in the Workflow Handlers Guide.

Add task handlers

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Handlers** tab.

In the left pane, the table lists any handlers assigned to the selected task. Handlers are listed under their specific trigger group to represent the execution order of action and rule handlers. To see any handlers currently assigned under a specific trigger, click expand ▼.

Name Review the work

Trigger	Arguments
 Promoted	
 Suspended	
 Resumed	
 Aborted	
 Undo	
 EPM-demote	

5. Select a trigger group to add the handler to.
6. On the **New Handler** panel on the right, click the handler type you want to create: **Action** or **Rule**.
7. All the applicable handlers for the trigger group are available in the drop-down list. The handler list is not case-sensitive. This list may contain custom handlers as well as handlers that are included with other solutions.

Select a handler.

The mandatory arguments display based on the handler selection.

The screenshot shows the 'Handlers' tab in the Workflow Designer. On the left, a table lists triggers: Assigned, Started, Performed, Completed (selected), Promoted, Suspended, Resumed, Aborted, and Undo. On the right, the 'New Handler' panel is open. It shows 'Handler Type' set to 'Action' and 'Rule'. The 'Name' is 'EPM-notify-signoffs'. Below, 'Add Arguments' is shown with an 'Argument' of '-comment' and a 'Value(s)' of 'Complete'. A 'Delete' button is next to the argument. An 'Add' button is at the bottom right of the panel.

Click **Help** to launch the handler documentation.

8. Add a new argument row by clicking **Add Arguments** .
9. Select the new argument from the drop-down list.
10. Enter any corresponding values in the **Value(s)** field.

Hints are provided for the handler arguments and their respective values to help you identify mandatory and optional parameters and avoid runtime errors.

11. Click **Add**.
Expand the trigger group to view the new handler in the table. Once added, you can highlight the new handler and click **Cut** , **Copy** , or **Delete** . To make any additional changes to the argument, highlight the row and edit the handler in the right-side **Information** panel.

Remove task handlers

Procedure

1. Open the template that contains the tasks you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Handlers** tab.
5. In the table on the left, highlight the action or rule handler that you want to delete under the trigger group and click **Delete** .

Update task handlers

Procedure

1. Open the template that contains the tasks you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Handlers** tab.
5. In the table on the left, highlight the action or rule handler that you want to edit.

To make any additional changes to the argument(s), edit the handler in the **Information** panel on the right. You can add additional arguments, or enter a different value for the argument in the **Value(s)** field, and click **Save**.

Other available editing options include:

Cut

Removes the handler from the current location and saves it to the clipboard so you can paste it elsewhere.

Copy

Creates another instance of the same handler in a new location, so you have two different instances of the same handler. Templates do not need to be in edit mode to copy a handler.

A handler can be copied:

- From one action to another action in a task.
- From one task to another task in the same template.
- From a task in one template to a task in another template.

Paste

Places the copied handler in the selected location.

Move Up

Moves the selected handler up within the table. This icon only appears for handlers within the same trigger group.

Move Down

Moves the selected handler down within the table. This icon only appears for handlers within the same trigger group.

Delete

Removes the handler from the table.

View task handlers

You can view the task handlers that make up each task in a workflow.

Procedure

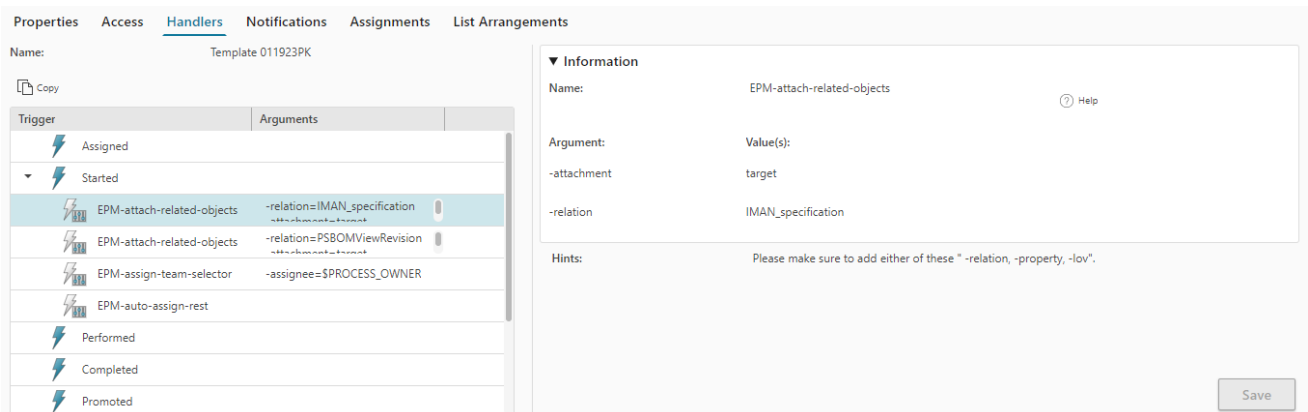
- 1. Open the template that contains the tasks you want to view.
- 2. Click the task that contains the handlers you want to view.
- 3. Click the **Handlers** tab to view the **Rule** and **Action** handlers assigned to the task.

In the left pane, the table lists any handlers assigned to the selected task. Handlers are listed under their specific trigger group to represent the execution order of action and rule handlers.

Click the expand icon to see any handlers currently assigned under a specific trigger group ▼.

To view handler information for the root task, click the initial **Start** task. 

The **Information** panel on the right displays the handler details.



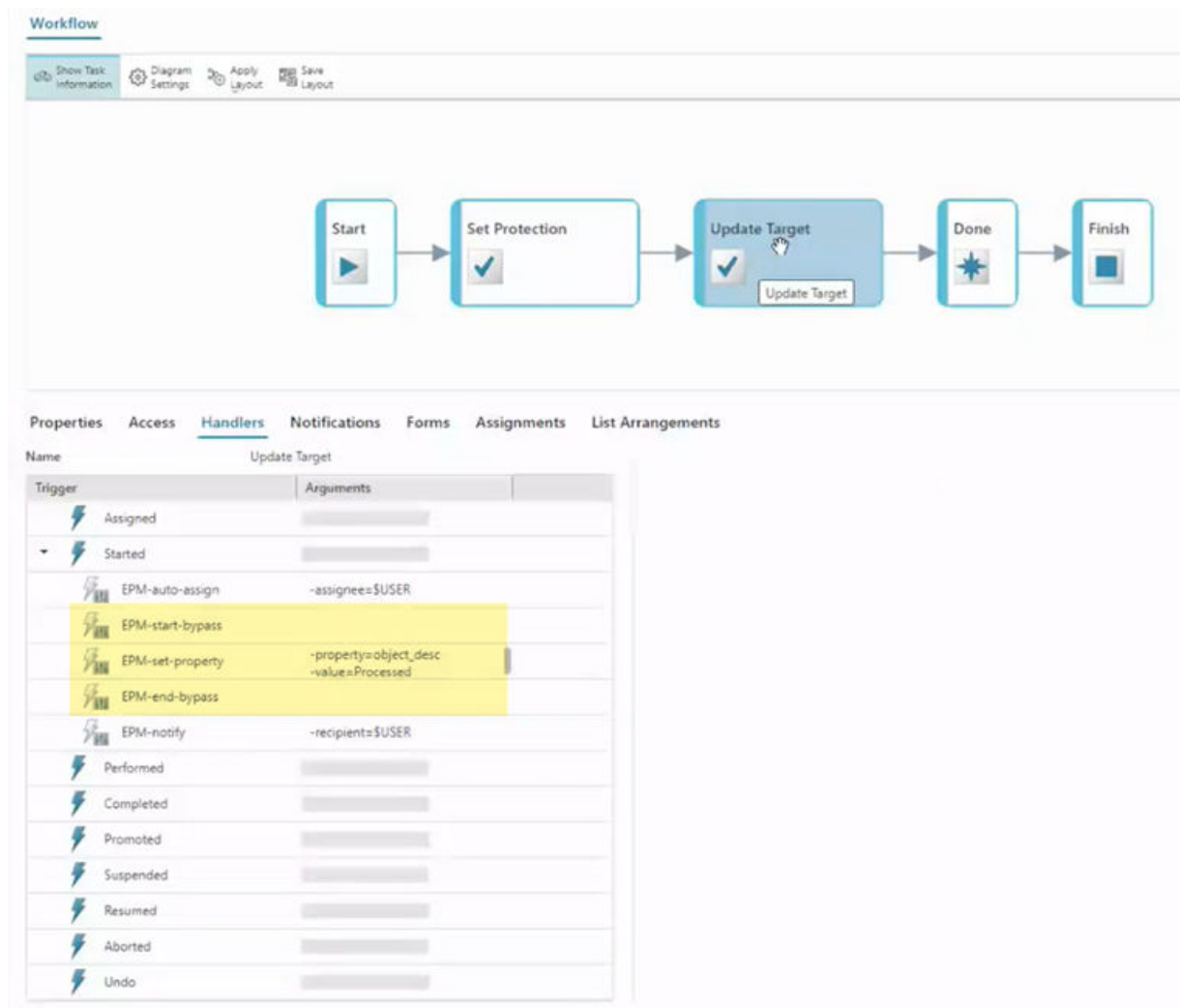
Bypass handlers

Bypass handlers (**EPM-start-bypass** and **EPM-end-bypass**) prevent user access errors by letting a select set of handlers run with system access. Administrators can define system authority for any handlers wrapped in between **EPM-start-bypass** and **EPM-end-bypass**, providing an alternative to running all handlers with either all or no system access.

EPM-start-bypass starts the bypass process, while **EPM-end-bypass** ends bypass. Any handlers in between grant users the access they need to make necessary updates.

Any number of handlers can be included between the bypass handlers. However, these handlers are only operative within a given action, although they can be configured throughout multiple actions. Here, the

EPM-set-property handler is provided system access in the **Started** action, as it is included in between the two bypass handlers.



Consider the following when using the bypass handlers:

- If bypass is enabled, then handlers run with system authority and **EPM-start-bypass/EPM-end-bypass** have no effect.
- The **WRKFLW_access_level_for_handlers_execution** preference has precedence over bypass subset handlers when it is set to **system** access. When system authority is enabled for all handlers, the **EPM-start-bypass/EPM-end-bypass** pair has no effect.
- A subworkflow inherits bypass from its parent workflow when the **EPM-create-sub-process** handler is executed within the bypass handlers' scope.
- Both handlers must be included and work as a pair when executing the bypass process.
- If the **EPM-end-bypass** handler is configured but the **EPM-start-bypass** is not configured, then bypass is not enabled and the **EPM-end-bypass** handler has no effect.

- If the **EPM-start-bypass** handler is configured more than once before the **EPM-end-bypass** handler is configured, then during execution, the first occurrence of the **EPM-start-bypass** handler starts bypass and the other **EPM-start-bypass** handlers have no effect. Likewise, if the **EPM-end-bypass** handler is configured more than once following an **EPM-start-bypass** handler, then the first occurrence of the **EPM-end-bypass** handler ends bypass and the other **EPM-end-bypass** handlers have no effect.

See the Handlers guide for more information about the **EPM-start-bypass** and **EPM-end-bypass** handlers.

Assign users to tasks

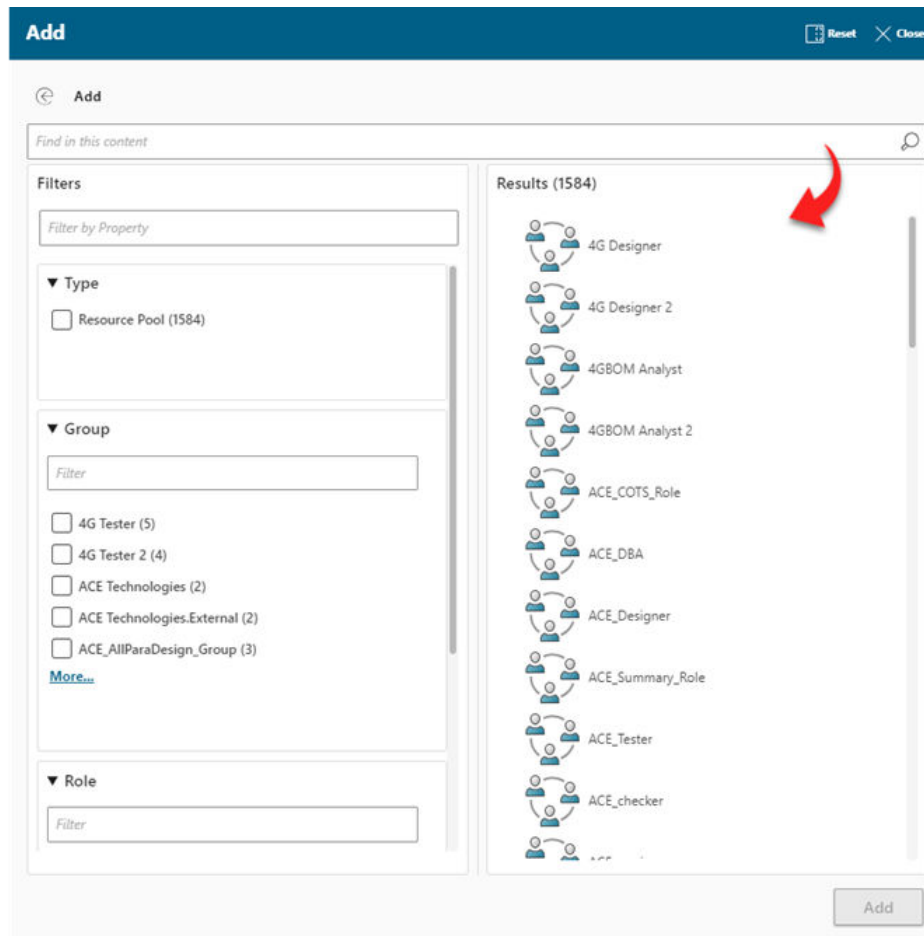
Use the **Assignments** tab to assign a user to a task.

The type of task determines the number of users that can be assigned. Most tasks require a single user, but multiple users can be assigned to a Review, Acknowledge, or a Route task (see the following section).

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to assign a user to it.
4. Click the **Assignments** tab.
5. Click **Add**  under **Assignments** to view a list of assignees.
6. Select either **Key Roles**, **Teams**, or an individual **User/Group Member** from the drop-down list, then click **Add** .
 - **Key Roles**
 Specifies the key roles defined for workflows, dynamic participants, and project management.
 To add a key role, search for and select the key role to add for the task, then click **Add**.
 To remove your selection, click the key role and click **Remove** .
 - To add **Teams** (predefined teams to choose from) or a **User/Group Member** (available participants in a specific group or role):
 - a. Type the name of the team or user to assign and press **Enter** or click **Search**  to see results. You can enter a partial name or wildcard characters.
 - b. For additional search filters, click the **Filter** icon .
 You can also display the filters and increase the size of the **Add** panel by dragging the left side of the panel to the left.

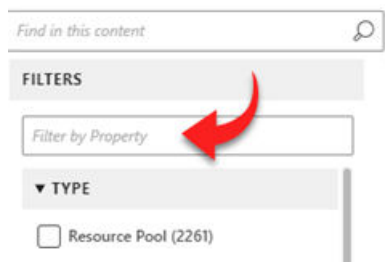
Results now display on the right.



Use the keywords **userid**, **group**, or **role** to narrow your search. Combine keywords by including a space in between (such as **role:checker userid:ed**), and for any term that contains a space, use quotations (**group:"Workflow Reports"**).

Once a search is performed and the results display, this top-most search box can be used again to additionally narrow those results.

- c. Filters are defaulted to **Type**, **Group**, or **Role**. If the user is assigned to a project, the **Project** filter is available. A search box narrows your search to a specific property.



You can also search within each property.

- d. Make your selection and click **Add**.

To remove your selection, highlight the user/team and click **Remove** .

7. Click **Add**.

The new task assignment is listed in the table. To make any changes, highlight the assignment and click **Edit** .

Work with Review tasks

Review tasks route workflow targets (documents, parts, designs, and so on) for review and require additional information.

Caution

Do not add or delete subtasks from the **Review** task. It may cause an error that prevents the task from being executed.

The Review task includes two subtasks:

- The **select-signoff-team** subtask requires the workflow process initiator to select the users performing the review (the signoff team). You can configure this subtask with predefined group or role profiles. The workflow process initiator must select or allow the workflow process initiator to select users of their choice in an ad hoc manner. Each member of the signoff team is responsible for reviewing and signing off on the attached target objects.

This subtask allows the selector to search by group, role, user and to select signoff members individually or by project teams.

- The **perform-signoffs** subtask is then distributed to the selected signoff team, prompting them to review the target objects and sign off. When a Review task is performed in a workflow process, the **perform-signoffs** task displays these options to each signoff team member: **Approve**, **Reject**. Selecting either **Approve** or **Reject** performs the task.

Configure an approval quorum

You can specify an approval quorum for the task. An *approval quorum* is the minimum number of signoff members required for a Review task to proceed.

Procedure

1. Open the template that contains the Review task you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the Review task that you want to modify.

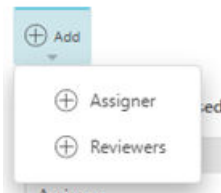
4. Click the **Assignments** tab.
5. Under **Participation**, specify the type of approval quorum: **Percent** or **Numeric**.
6. Accordingly, enter one of the following in the text box:
 - A percentage value required to progress the task. For example, if it is set to 51%, then 51% of signoff members must approve for the task to move ahead.
 - A numeric value to specify the number of signoff members needed to move the task ahead. For example, if it is set to 5, then 5 members must approve for the task to move ahead. This value cannot exceed the number of reviewers added.
7. Click **Require full participation** if you want all users to review and comment before the task progresses.

Add assigners and reviewers to a Review task

Multiple users can be assigned to a Review, Acknowledge, or a Route task.

Procedure

1. Open the template that contains the Review task you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task that you want to modify.
4. Click the **Assignments** tab.
5. Under **Assignments**, click **Add**  and select either **Assigners** or **Reviewers**.



For both the **Assigner** and **Reviewer** role, you can add users from the list by **Key Roles**, **Teams**, or (as individual) **User/Group Members**.

Key Roles

Key roles specify the key roles defined for workflows, dynamic participants, and project management.

Teams

Predefined teams to choose from.

User/Group Member

Available participants in a specific group or role.

6. For the **Assigners**:

- a. Select either **Key Roles**, **Teams**, or an individual **User/Group Member** from the drop-down list.
- b. Click **Add** ⊕.
- c. To add a **Key Role**, search for and select the key role to add for the task, then click **Add**.

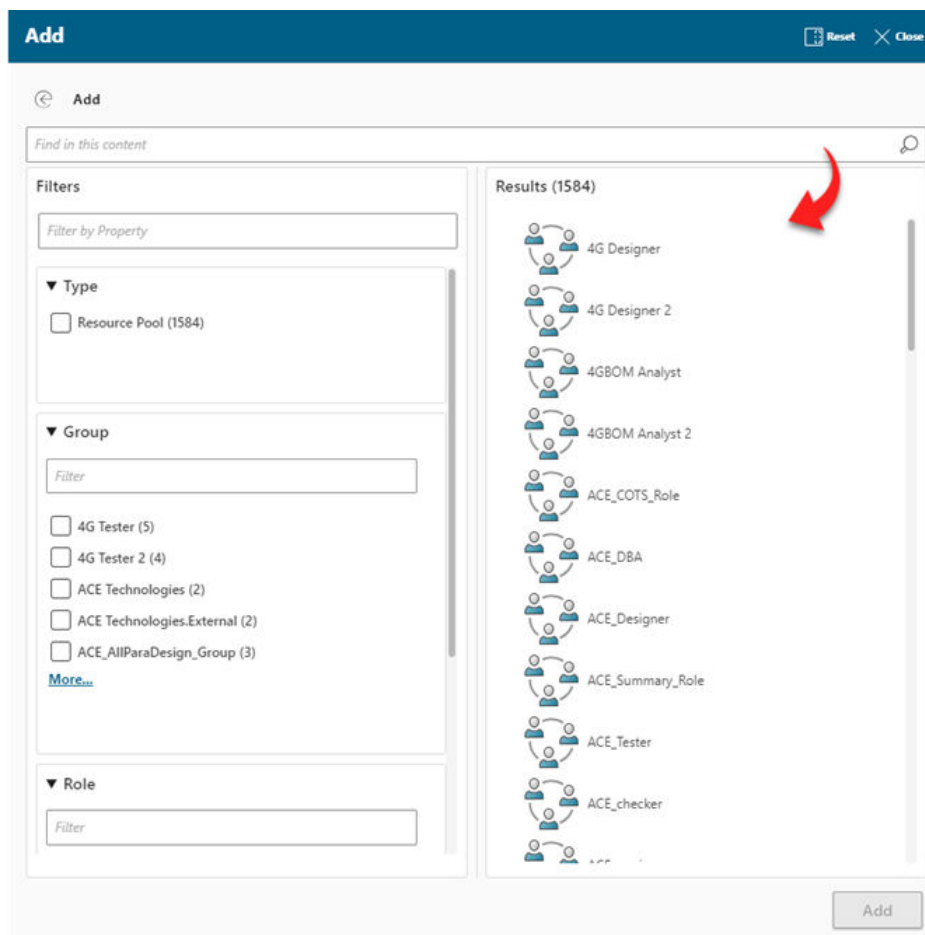
To remove your selection, click the key role and click **Remove** ⊖

d. If you select **Teams** or **User/Group Member**:

- i. Click **Add** ⊕.
- ii. Type the name of the team or user to assign and press **Enter** or click **Search** 🔍 to see results. You can enter a partial name or wildcard characters.
- iii. For additional search filters, click the **Filter** icon 🏠.

You can also display the filters and increase the size of the **Add** panel by dragging the left side of the panel to the left.

Results now display on the right.

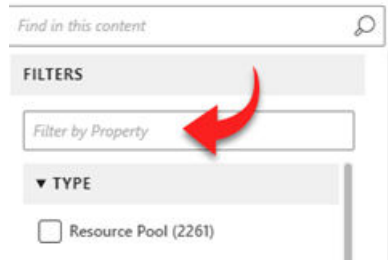


You can click **Reset**  to reset the panel to its default location and size.

Use the keywords **userid**, **group**, or **role** to narrow your search. Combine keywords by including a space in between (such as **role:checker userid:ed**), and for any term that contains a space, use quotations (**group:"Workflow Reports"**).

Once a search is performed and the results display, this top-most search box can be used again to additionally narrow those results.

- iv. Filters are defaulted to **Type**, **Group**, or **Role**. If the user is assigned to a project, the **Project** filter is available. A search box narrows your search to a specific property.



You can also search within each property.

- v. Make your selection and click **Add**.

To remove your selection, highlight the user/team and click **Remove** .

7. For the **Reviewers**:

- a. Select either **Key Roles**, **Teams**, or an individual **User/Group Member** from the drop-down list.
- b. Click **Add** .
- c. Filters act the same as **Assignee**; make your selection and click **Add** .
 - For **Key Roles** and **User/Group Member**, the option **Allow reassign outside selected group and role** lets the task owner reassign the workflow to another group and role outside of what is selected. If you do not select this check box, the task owner can only reassign the workflow to someone in the same group and role.
 - For **Teams**:
 - i. Enter the **Number of Reviewers** to review the task.
 - ii. Under **Assignment Option**, click one of the following:
 - **Team Assignment to be selected by assigner**
This option adds the team assignment to the select-signoff-team.
 - **Team Assignment to be claimed by assignee**
This option creates resource pool assignments for the group or role that was specified, such as Engineering or Designer.
- d. Click **All Reviewers must sign off** if you want all your reviewers to sign off on the review without the provision of being removed from the list.

8. Once you select your assignees/reviewers, click **Add**.

The new task assignment and its details are listed in the table.

9. Once you select your reviewers, click **Interactively paused for adding more reviewers** to be able to pause the workflow to add more reviewers to the select-signoff-team subtask, if necessary.

If you do not select this check box, the select-signoff-team task ends automatically with no additional reviewers, and continues to the perform-signoff subtask. The associated handler under the **Handlers** tab shows **-auto_complete**.

Add notifications to tasks

Add notifications to tasks in the **Notifications** tab.

Procedure

1. Open the template that contains the tasks you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to add the notifications to.
4. Click the **Notifications** tab.
5. To create an email notification for a task, click **Add** .
6. In the **Notify When Task** list, select when to notify the recipient during the task's lifecycle.
7. Enter a **Subject** for your notification email.
8. Enter the **Message** text for the recipient(s).
9. (Optional) Choose to include any information in the email about **Process Info**, **Targets**, and **References**.
10. Click **Add**  next to **Recipients** to identify whom to notify.
 - a. Select either **Key Roles**, **Teams**, or an individual **User/Group Member** from the drop-down list.
 - **Key Roles**
Specifies the key roles defined for workflows, dynamic participants, and project management.
 - **Teams (Resource Pools)**
Specifies predefined teams to choose from.
 - **Users**
Specifies participants for a specific group or role.
 - b. Click **Add** .

- c. To add a **Key Role**, search for and select the key role to add for the task, then click **Add**.

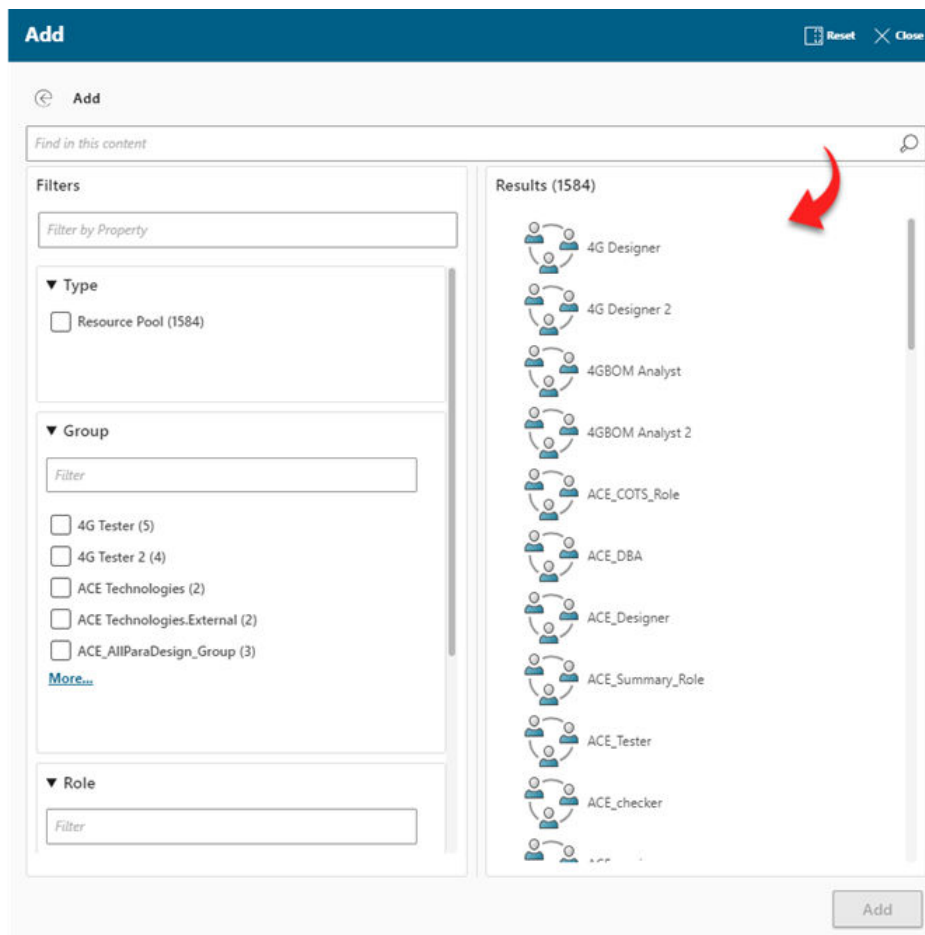
To remove your selection, click the key role and click **Remove** ⊖

- d. If you select **Teams** or **User/Group Member**:

- i. Click **Add** ⊕.
- ii. Type the name of the team or user to assign and press **Enter** or click **Search** 🔍 to see results. You can enter a partial name or wildcard characters.
- iii. For additional search filters, click the **Filter** icon 🏠.

You can also display the filters and increase the size of the **Add** panel by dragging the left side of the panel to the left.

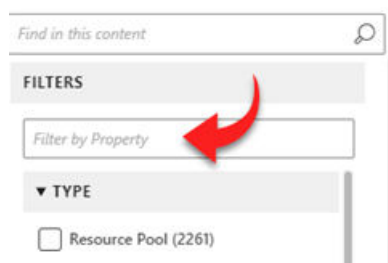
Results now display on the right.



You can click **Reset** 🏠 to return the panel to its default location and size.

Once a search is performed and the results display, this top-most search box can be used again to additionally narrow those results.

- iv. Filters are defaulted to **Type**, **Group**, or **Role**. If the user is assigned to a project, the **Project** filter is available. A search box narrows your search to a specific property.



You can also search within each property.

- v. Make your selection and click **Add**.

To remove your selection, highlight the user/team and click **Remove** ⊖

11. Once you've selected the recipient(s) to be notified, click **Add**.
The notification and its associated information is added to the notifications table. To remove a notification, highlight the notification and click **Remove** ⊖. To edit the notification, click **Edit** ✎ to display the Edit dialog box. To show all the information in a cell, click the gear icon ⚙ to **Wrap Text**.

Retain approvals

Retain approvals for a task during rework

Workflows often need to be routed for rework or restarted for various reasons. A loop back path allows the workflow to return to a previous stage for re-evaluation and adjustments, enabling corrections and retaining prior decisions without restarting the entire process. In such cases, resetting all the prior decisions may not be necessary. The ability to retain certain decisions during rework on a Review or Acknowledge task allows you to keep previous approved decisions intact while only resetting the reject decisions. Users who have approved will not receive the restarted task in their Inbox. This reduces cycle time by eliminating the need for all reviewers to re-approve.

There are three ways to configure the retention of decisions in a workflow:

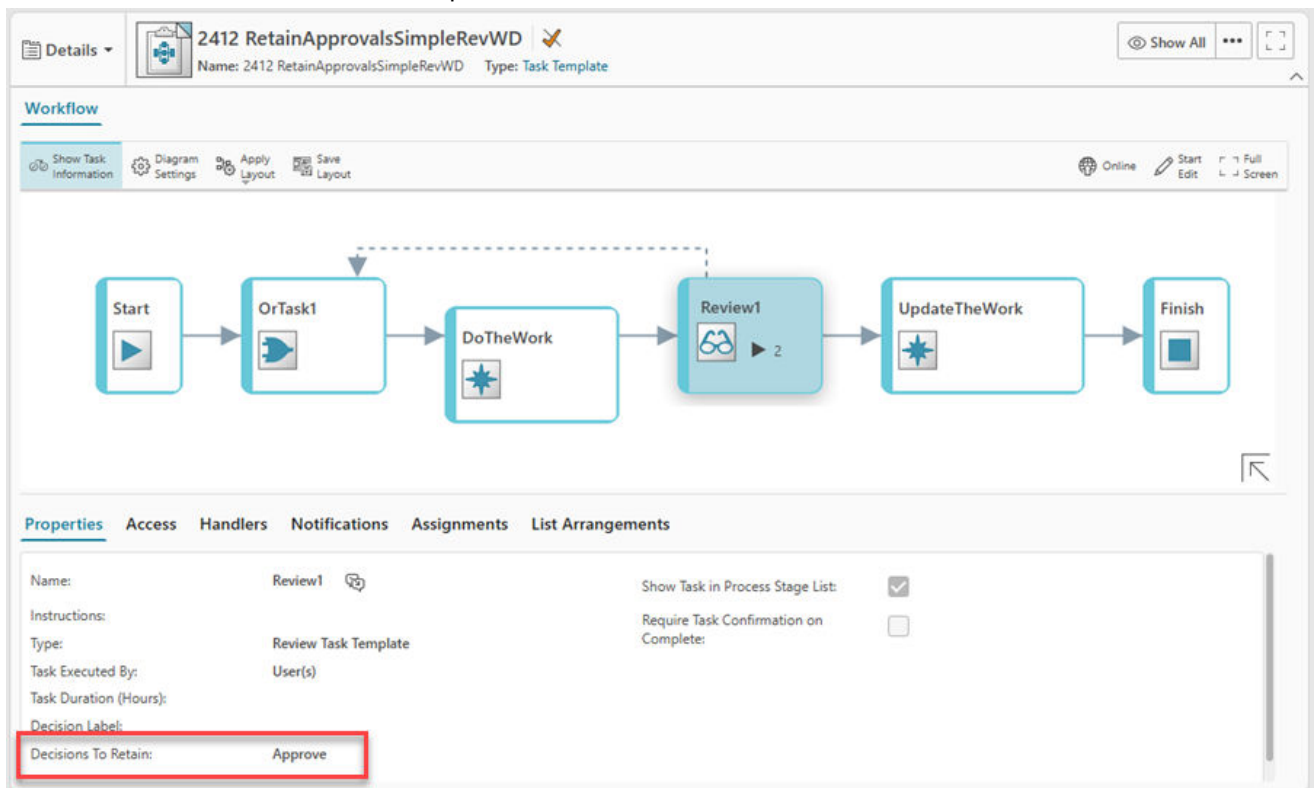
- Set it as a default on a Review/Acknowledge task when creating a workflow template.
- Use the EPM-set-property handler to set the property on Review/Acknowledge tasks.
- Allow a user with sufficient authority to decide whether to retain certain decisions for a Review/Acknowledge task during a workflow execution.

Retain approvals by default in a template

Workflow designers can configure a task to keep approved decisions intact by default, while only resetting rejected ones for a Review or Acknowledge task in a template. A loop back path allows the workflow to return to a previous stage for re-evaluation and adjustments, enabling corrections and retaining prior decisions without restarting the entire process.

Procedure

1. Open the template that contains the tasks you want to modify, and then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the Review or Acknowledge task to modify.
4. On the **Properties** tab, select **Approve** for a Review task or **Acknowledge** for an Acknowledge task under the **Decisions to Retain** drop-down list.

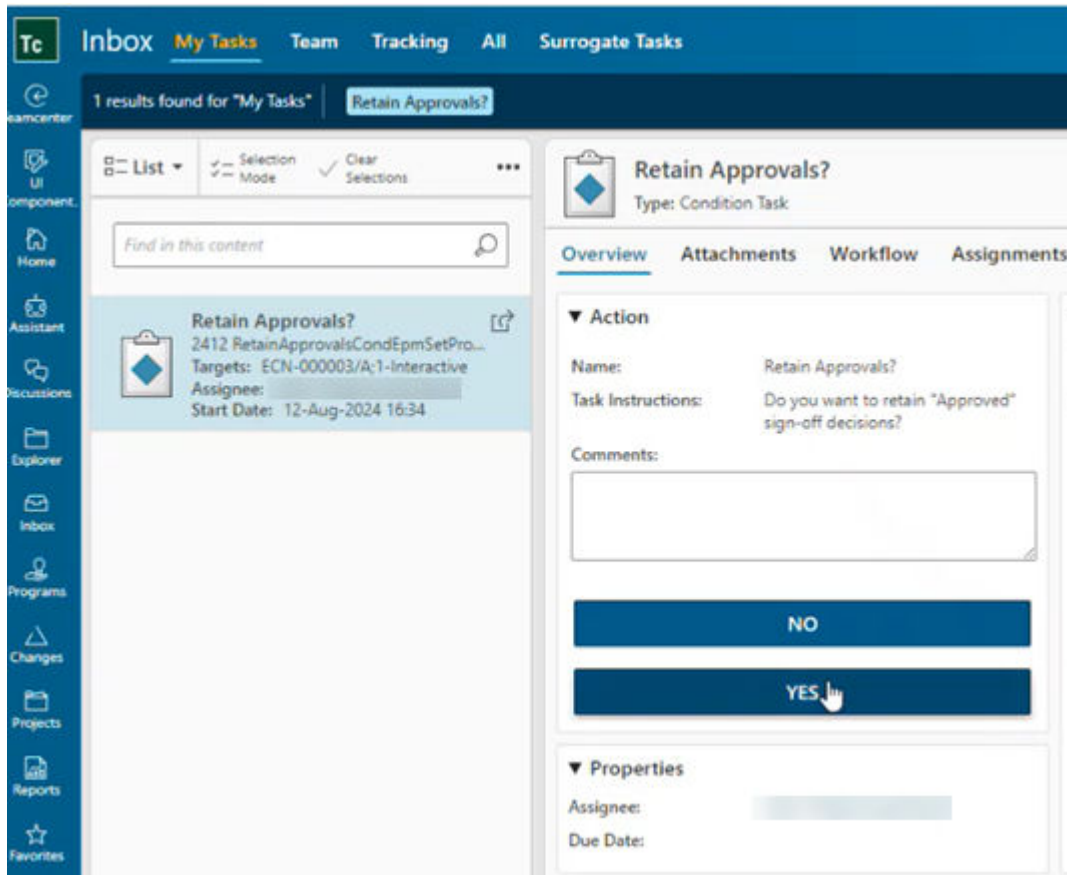


The selected task is modified to retain approvals by default whenever the template is used.

Use the epm-set-property handler on a Review/Acknowledge task for users to retain decisions

Adding an interactive task, such as a Condition task, to a template lets the workflow owner decide whether to retain decisions on a Review/Acknowledge task during a restart.

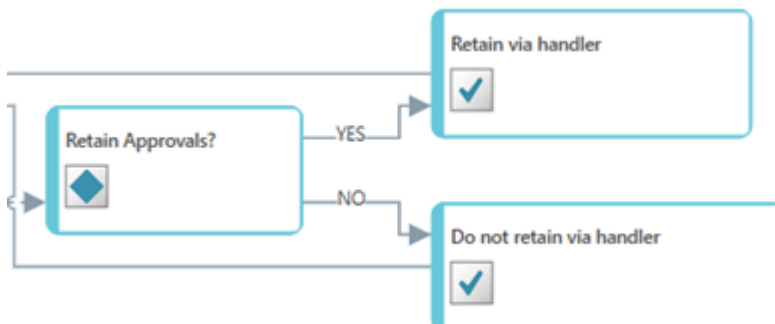
If the Review/Acknowledge task is rejected, the workflow routes to a Condition task that asks the workflow owner if they want to retain approved decisions.



If yes, the workflow reroutes with approved decisions retained. If the workflow owner decides not to retain approved decisions, the workflow reroutes back to all the assigned reviewers to review the task again.

Prerequisites

During the workflow creation process, include a non-interactive task, such as a Condition task, to provide the user with options. Place non-interactive tasks after the Condition task, ensuring they include the necessary handler settings to retain approvals.



The following section describes a sample configuration that uses a Condition task and the EPM-set-property handler to retain approvals for a Review task.

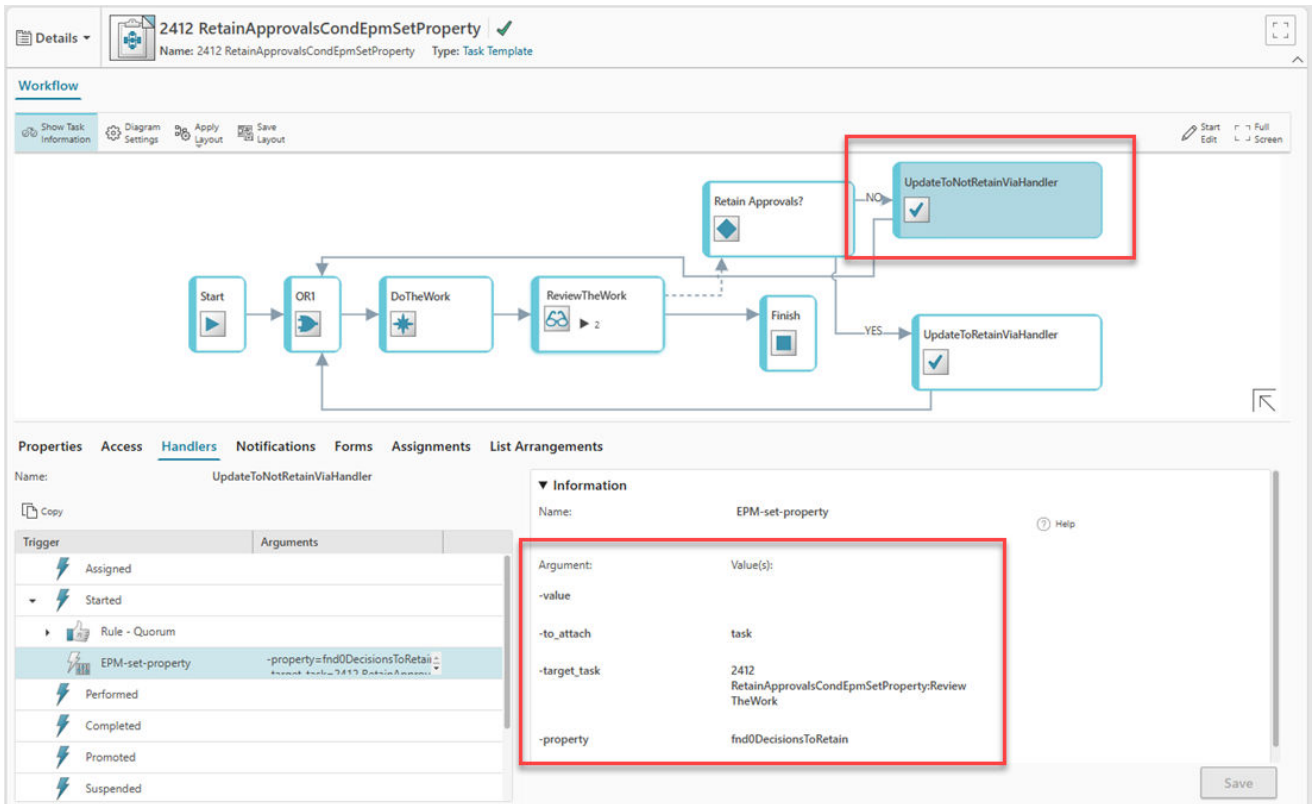
Procedure

1. Select the task where you want to set the handler value to determine if users will retain approvals. This is a task using the Task template.
2. On the **Handlers** tab, select the EPM-set-property handler for the **Started** trigger group.
3. Set the following:
 - a. To retain all previously-acquired decisions, set the handler's **-value** to **89** (approvals kept).
 - b. Set **-to_attach** to **task**.
 - c. Enter the full path to the Review/Acknowledge task where the property will be applied under **-target_task**. This would be the name of the task to retain approvals for; for example, a Review task.
 - d. **fn00DecisionsToRetain** is the **-property** value.

The screenshot displays the Workflow Designer interface for a task named '2412 RetainApprovalsCondEpmSetProperty'. The workflow diagram shows a sequence of tasks: Start, ORI, DoTheWork, ReviewTheWork, and Finish. A decision diamond 'Retain Approvals?' follows 'ReviewTheWork'. If the answer is 'NO', it leads to 'UpdateToNotRetainViaHandler'. If 'YES', it leads to 'UpdateToRetainViaHandler', which is highlighted with a red box. Below the diagram, the 'Handlers' tab is active, showing a list of triggers on the left and configuration details on the right. The 'EPM-set-property' handler is selected for the 'Started' trigger. The configuration details for 'UpdateToRetainViaHandler' are as follows:

Argument	Value(s)
-value	89
-to_attach	task
-target_task	2412 RetainApprovalsCondEpmSetProperty:ReviewTheWork
-property	fn00DecisionsToRetain

- To optionally provide a non-interactive task to discard previous approvals and restart the review process, add the interactive task with the EPM-set-property handler on the **Started** trigger group, but keep the **-value** argument blank.

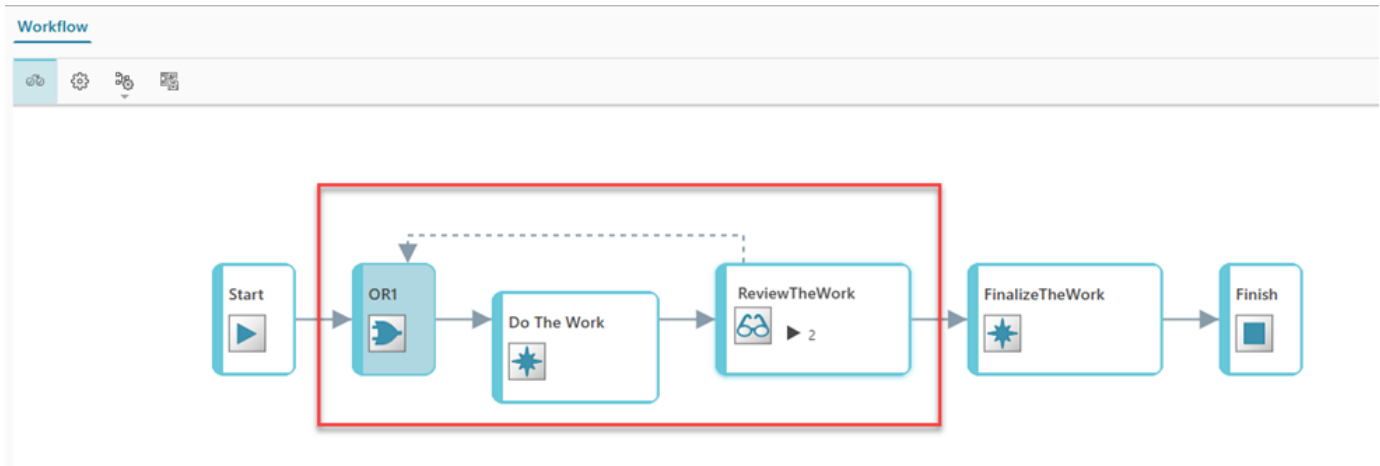


The user will receive a Condition task in their Inbox to decide whether to retain approvals on the task.

Allow an authorized user to decide whether to retain task approvals

In this scenario, a template is created where, if a Review/Acknowledge task is rejected, a user with appropriate privilege can decide whether to retain approvals before the review restarts. Privilege can be configured using the Fnd0EPMTaskModifyPrivilege condition.

For example, if a rejection occurred due to minor changes, a user might choose to retain all approvals. This decision can speed up the process and prevent the entire set of reviewers from receiving another Review task for a minor change. If a user decides not to retain approvals, such as when a large-scale change requires further assessment, the reviewers will receive the task in the Inbox again.



The task assignee indicates the decision via the **Information** panel on the Review/Acknowledge task.

Work with forms

Add forms to tasks in the **Forms** tab. The **Forms** tab is displayed when you include a **Task** task in your workflow.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Once you drag a **Task** task to the Workflow Designer from the **Task Palette**, the **Forms** tab is displayed.
4. On the **Forms** tab, click **Add**  to create a new form for the task.
5. In the **Add** dialog box, enter a **Form Type** or select one from the list.
6. Enter a **Form Name**.
7. Enter a **Description** for the form.
8. Click the **Attach As** list to select whether to add the form as a target or a reference.
9. Click **Add**.
The new form is added to the **Forms** table and the **EPM-create-form**, **EPM-display-form**, and **EPM-hold** handlers are added to the **Handlers** tab. To make any changes to the form, click **Edit** , or click **Delete**  to remove the form and its associated handlers from the task.

Modify task properties

You can modify task properties in the **Properties** pane.

Procedure

1. Open the template that contains the tasks you want to modify, then click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Properties** tab.
5. Edit the **Name** if needed.
6. Enter any **Instructions** for the task.
7. The **Type** list identifies the task type.
8. Enter the duration for the task under **Task Duration (Hours)** in hours. An associated task handler (EPM-set-duration) is created in the **Handlers** table.
9. Set the **Decision Label** for a Review and Acknowledge task.
10. Click **Show Task in Process Stage List** to display the task in the **Process State List** property for the target object. Tasks in the **Process Stage List** are used to determine the ACL for the target objects.
11. Click **Process in Background** to configure the task (and any subtasks) to allow for background processing.

This option appears if Workflow Designer has been configured for background processing. If so, all of those tasks' actions (except **Perform** and **Assign**) are processed asynchronously.

Values for **EPM_task_execution_mode** include the following:

FOREGROUND	All tasks run in the foreground. This is the default.
BACKGROUND	All tasks run in the background. This option displays the Dispatcher Request Priority list on the Properties tab for a task.
CONFIGURABLE	• If the value is On, and you select the Process in Background check box, the task runs in the background. When selected, the Dispatcher Request Priority list is displayed as well. • If the value is Off, the task runs in the foreground.

12. If the task is running in the background, the **Dispatcher Request Priority** list is displayed. This list lets you define the execution priority for background tasks and is specifically used with multiple instances of Dispatcher running. Click the list to select the level of priority for the task.

13. Click **Select Participants from Workflow** to select dynamic participants from the workflow instead of target objects. This option is only applicable for the root task.
14. Click **Require Task Confirmation on Complete** to require users to confirm that a selected interactive task is completed.
Selecting a root task requires completion confirmation on all child tasks.
15. Click **Require admin privilege for modification** to ensure templates are non-modifiable and can only be edited by a **dba** user.

Add a list of values to Review and Acknowledge tasks

The **Review** and **Acknowledge** tasks have optional properties for task behavior. Adding a decision label to a **Review** or **Acknowledge** task provides a list of values (LOV) to the task. LOV examples of decisions include choices such as **Approve** and **Reject**.

Prerequisites

Use the Business Modeler IDE to add a **decision label** for the **Review** or **Acknowledge** task. For more information, see Add custom decision labels.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the task to modify.
4. Click the **Properties** tab.
5. Select a value from the **Decision Label** list.
The selection adds decision buttons to the workflow task.

Add a release status to an Add Status task

Add Status tasks create and add a release status to the target objects of the workflow. A release status is a visual milestone in a workflow.

Status tasks have optional properties for task behavior.

Procedure

1. Open the template that contains the tasks you want to modify, and click **Start Edit** .

2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the status task to modify.
4. Click the **Properties** tab.
5. Select a value from the **Release Status** list.

The values in the list can include the default values or custom statuses defined by your organization.

Adding a release status adds a release status indicator icon to a workflow task.

Work with Condition Tasks

Use the **Condition Task** template to branch your workflow process according to defined criteria. Because this task template is used to branch workflow process flow, you must always create at least two paths branching off from the task. The paths can be either success paths, failure paths, or a combination of the two.

- Success paths can be either true paths, false paths, or paths with a customized result.
- Failure paths can only be generated from manual **Condition** tasks. They allow an alternate course when a specified task is rejected, when a user determines the path cannot be completed, or when an error occurs.

Tip

If you use a **Condition** task to branch your workflow process, you can use one or more **Or** tasks later in the workflow process to resolve the paths into a single path.

The system determines which of the branches flowing from a **Condition** task to perform based on the *task result*. The task result is stored in the **Condition** task. The successor tasks have a handler configured with a value that may match the task result. After the task result is set, the successor tasks are examined and any successor tasks containing a value matching the task result are started. Use any of the following methods to set the task results:

- Create a query against the target (automatic only).
- Create a query against the task (automatic only).
- Create a query against subworkflow (also known as subprocesses) (automatic only).

If there are multiple subworkflows, a query runs on the associated subworkflows and the results are used to branch accordingly. The query is typically configured to look at the root task's **result** attribute for all the subworkflows.

If there is only one subworkflow, and it is configured to set the result on the **Condition** task, no query is needed, and the workflow follows the branch based on the result.

A **Condition** task can be configured to run either automatically or manually. You must determine the configuration that is best suited for the workflow process template you are defining. Typically, if a handler can determine the criteria, it is best to configure the task as automatic.

Task	Description
Automatic Condition task	Add an action handler that sets the task's result to true, false, or a customized value. The simplest way to achieve this is to define a condition query or BMIDE condition filter for the task at design time; this automatically inserts the action handler. Alternatively, you can write custom code or use the EPM-set-task-result-to-property handler.
Manual Condition task	During design, you do not define a query or add an action handler to the task template. Because no query is defined and no action handler is configured to set the task result, when the workflow process is run, the end user must manually indicate a value using an interactive dialog box. The value chosen by the end user is used to set the task result. Custom conditions can also appear as manual condition options and appear as buttons in the Condition dialog box.

Caution

To ensure desired results, condition tasks that run queries in workflows should always have at least one target object when a condition query is run against workflow targets.

- When a condition task runs a condition query against workflow targets, the system searches the database for that query class and filters the results based on the workflow target objects.
- Because handlers can move objects between targets and references in a workflow, the workflow may have objects in the references folder, but no objects in the targets folder. The condition query will not search in the database if the workflow does not have any targets. This will set a false path of the condition task.

To configure a **Condition** task, do the following:

Procedure

1. Open the template that contains the tasks you want to modify, and click **Edit**  > **Start Edit**.
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Select the **Condition** task and view the **Properties**.
4. (Optional) **Instructions** are not required for tasks, but may guide users through the task.
5. Continue to the following sections to add condition queries to the task, or configure a task with a BMIDE condition.

Add a BMIDE condition to a Condition task

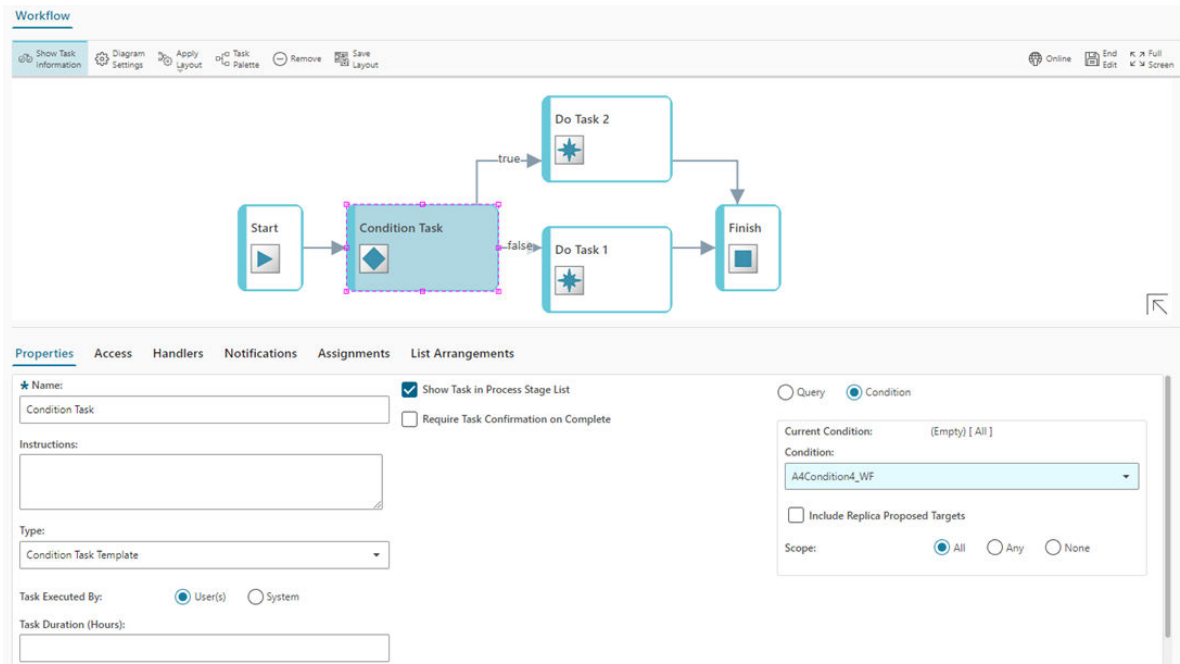
You can configure a Configure task with a BMIDE condition to automate the condition task result.

Prerequisites

A pre-existing BMIDE condition must be available before applying it to a Condition task. There are predefined generic conditions available in the BMIDE that you can reuse, or you can also create custom conditions if needed using the new condition wizard.

Procedure

1. In the **Properties** tab of the **Condition** task, click **Condition**.
2. Click the **Condition** drop-down list and select a condition to apply to the task, or enter a search string to locate one.
3. Click the **Include Replica Proposed Targets** check box to include targets for the remote workflow task in the search.
4. Click a scope to determine if the task satisfies the condition for **All** targets, **Any** targets, or **None**. The condition is added to the task.

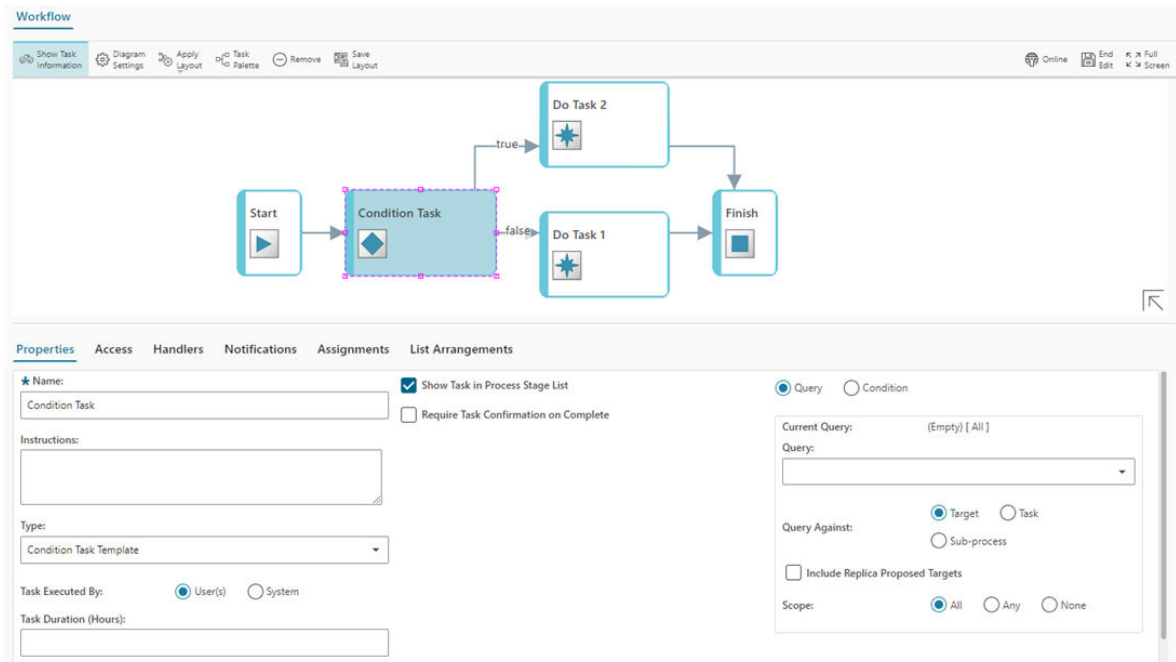


Add condition queries to a Condition task

Condition tasks have optional properties for task behavior. Condition queries control the specific functions of a **Condition** task. Once applied, conditions check for specific parameters that you set. These conditions work in conjunction with the **Success** path from the **Condition** task to its task or target.

Procedure

1. In the **Properties** tab of the **Condition** task, click **Query**.
2. Select a query from the **Query** list. Queries are created in the rich client.
Your organization may have their own configured queries in this list.
3. Select whether the query should run against a **Target**, **Task**, or a **Sub-process**.
4. When **Target** is selected, **Include Replica Proposed Targets** is active. Select this check box to include targets for the remote workflow task in the search.
5. If a target or subworkflow is selected, apply a **Scope** to the query. Options are **All**, **Any**, or **None**. The query is added to the task.



Configure how tasks display in a workflow template

Set the visibility of tasks

Workflow Designer provides the option to make tasks visible or non-visible to end users in a workflow template. Administrators can display a user-interactive task in a workflow, or hide tasks from users that are non-interactive, without specified assignments, or are automatically completed by a handler.

Filtering the visibility of tasks is convenient if you have a very large template with numerous tasks. Hiding non-interactive (or automated) tasks can tidy up a template's appearance and create less graphical clutter for the end user, as task nodes are minimized on the template's graphical interface and display only as icons. This task filtering then applies to various task lists, such as when submitting an object to a workflow, identifying task assignments or viewing upcoming tasks in the Inbox, and when viewing a process assignment list (PAL).

Note

The Review, Acknowledge, and Route tasks always display in a template and their visibility settings cannot be configured.

Procedure

1. Open the template that contains the task(s) you want to modify, and click **Edit**  > **Start Edit**.
2. Select one of the following:
 - Keep the template online during the editing process.

- Keep the template offline during the editing process.
- 3. Click the specific task to modify.
- 4. Click the **Properties** tab.
- 5. Under **Task Executed By**:
 - Click **System** to specify that it is a non-interactive, automated task that is not visible to the user.
 - Click **User** to specify that the task is interactive and will be visible to the user.

Workflow

Show Task Information | Diagram Settings | Apply Layout | Task Palette | Remove | Save Layout | Online | End Edit | Full Screen

Properties | Access | Handlers | Notifications | Assignments | List Arrangements

Type: Add Status Task Template

Task Executed By: ☐ User(s) ☒ System

Task Duration (Hours):

Release Status: Translation Submitted

Once online, the template reflects the task visibility updates in the following areas:

- When submitting a workflow, in the **Submit to Workflow** panel
- In the Inbox:
 - The Assignment tab
 - The Upcoming Tasks table on the Workflow tab
 - The graphical view on the Workflow tab; non-interactive tasks only display their **icon**.



- In an Assignment List

Related Tasks

Set the task order

Set the task order

Workflow Designer provides the option to configure the task order in a workflow template that is then reflected in other task lists.

For example, if you **add a new task to a template** from the **Task Palette**, that task is automatically displayed at the bottom of any workflow task list unless you manually change its order in the Workflow Designer.

Note

The reordering of child tasks in the Review, Acknowledge, and Route tasks cannot be configured.

Procedure

1. Open the template that contains the task(s) you want to modify, and click **Start Edit** .
2. Select one of the following:
 - Keep the template online during the editing process.
 - Keep the template offline during the editing process.
3. Click the **Properties** tab.
4. Click the **List Assignments** tab.
Tasks are listed in the order that they are in the template.
5. Change the task order by selecting a task row and clicking **Move Up**  or **Move Down**  (on top of the table) to move the task up or down, respectively.

Or, drag and drop the tasks to the correct order in the table.

This will represent the order that the tasks will execute in the template.

Once online, the updated task order is reflected in the following task lists:

- When submitting a workflow, in the **Submit to Workflow** panel
- In an Assignment List
- In the Inbox:
 - The Assignment tab
 - The Upcoming Tasks table on the Workflow tab

Related Tasks

Set the visibility of tasks

Check for errors with a Validate task

Find error codes

Most workflow errors are displayed within the Enterprise Process Modeling (EPM) module. All error codes are documented in the *Integration Toolkit Function Reference*. Here, error codes are grouped by module. For example, Application Encapsulation (AE) errors are listed within the AE module, Appearances errors are listed within the Appearances module, and so forth.

Although using workflow (EPM) error codes with the **Validate** task may be the most common usage, the task works with any error code. You can add error codes from any module, or custom error codes, to the **Results List**.

This procedure describes how to display a list of error messages.

Procedure

1. Open the *ITK Function Reference* on Support Center, under **Teamcenter Documentation, Teamcenter x Developer References** (where x is the release).
2. At the top of the page, select the **Modules** header.
3. In the **Modules** page, scroll down to the appropriate module.

For example, to see all Enterprise Process Modeling (EPM) errors, which contain the majority of workflow errors, scroll to **Enterprise Process Modeling (EPM)** and click the link.

4. Click **Errors**.

The error page displays all errors for that module. Error numbers are defined as `<module base value> + <error code>`.

For example, the **EPM_internal_error** error has an error code of **EMH_EPM_error_base + 1**.

Add error codes

In the context of the **Validate** task, a *workflow error* means any error generated by a workflow handler. After drawing a failure path between the **Validate** task and a successor task, you must specify how you want the failure path to respond to workflow errors. You configure a failure path by **pairing** a workflow handler and an error code.

The failure path can be configured to activate when:

- Any error occurs by selecting **All Errors**.

This option automatically configures the failure path to activate upon any error. No additional steps are required.

- *Specific errors occur by selecting **Selected Errors**, as shown in the following procedure.*

Procedure

1. Click the path you want to configure as a failure path.
The **Properties** for the path display.
2. Under **Results**, click **Selected Errors** to specify which error codes you want the Validate task to check.
The **Add Error Code** text box displays.
3. In the **Add Error Code** box, type an EPM error code. For example, type **32009** (**RES_OBJECT_IS_RESERVED**) to ensure the failure path is followed whenever a target is not checked in.
4. Click **Add** to add this error to the **Results List**.

Workflow

Show Task Information | Diagram Settings | Apply Layout | Task Palette | Remove | Save Layout

```

graph LR
    Start[Start] --> DoTask[Do Task]
    DoTask --> ValidateTask[Validate Task]
    ValidateTask -- Success --> Success[Success]
    ValidateTask -. Failure .-> Failure[Failure]
    Success --> OrTask[Or Task]
    Failure --> OrTask
    OrTask --> Finish[Finish]
  
```

Properties

Predecessor: Validate Task
Successor: Failure

Results: ☐ All Errors ☒ Selected Errors

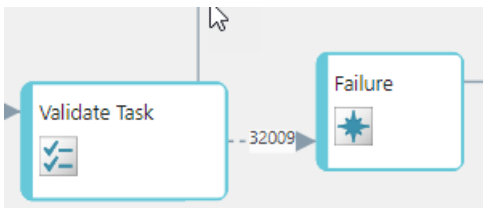
Add Error Code:

32009 **Add**

Results List:

ANY

5. Continue adding errors to the **Results List** until you have specified all the errors you want to cause the workflow to follow the failure path.
The specified errors display on the failure path.



Validate task behavior

The **Validate** task's behavior depends upon how its failure path is configured and what errors are received.

Failure criteria you specified	Error thrown (if any)	Task behavior
Fail if any error	Any error	Failure path is followed.
Fail if error on error list occurs	Error on error list	Failure path is followed.
Fail if error on error list occurs	Error <i>not</i> on error list	Workflow process halts. Task remains in Started state and an error appears.
No failure path configured	Any error	Workflow process stops. Task remains in Started state and an error appears.
Regardless of whether failure path was configured, and whether errors occurred	No errors occur	Success path followed. If no success path was configured, workflow process stops.

7. Using an assignment matrix

What is an assignment matrix?

An *assignment matrix* is a responsibility charting tool that identifies responsibilities for everyone on the team. It includes specific duties and how people participate in the project. You can use an assignment matrix to automate the assignment of workflow participants. This eliminates the need for end users to decide who participates in tasks during a workflow or when designing templates. By constructing an assignment matrix, a project manager can make sure that the team members maintain their specific roles and assignments. This can help to avoid confusion as the project progresses.

The assignment matrix creates an automated mapping between the task activities and the end users for easier and faster workflow execution.

You can predefine assignees for various roles (or **Assignment Types**) based on identified property values (or **Decisive Properties**). You can also base assignments on pre-configured Business Modeler Integrated Development Environment conditions for the target object. Assignees can include:

- Group members
- Resource pools
- Dynamic participants from the workflow target object
- Dynamic participants from the assignment source
- Key roles from other sources, such as the workflow owner or user

Example

An assignment matrix can automatically assign users to workflow tasks when the following criteria are met:

- **Workflow Target Type:** The workflow includes a specific target type, such as Change Notice Revision.
- **Decisive Properties:** The target object meets specified property values, such as:
 - Description = New
 - Disposition = Approved
 - Closure = On Hold

When these criteria are met and the assignment matrix handler is configured on the task to retrieve assignments from the assignment matrix, the system automatically assigns users to the workflow task. For example:

- A user from the Change Analyst role is assigned as the Assignee
 - Multiple users are assigned as Reviewers
- These assignments apply to all affected workflows that meet the specified criteria.

Note

An assignment matrix is not the same as a process assignment list (PAL). A PAL uses a human decision when determining the participants for an object in a workflow. An assignment matrix is configured before the workflow runs, and assignments are automated based on the data submitted to the workflow.

The assignment matrix process

The following is a high-level overview of the steps involved when creating and using an assignment matrix.

1. Administrative tasks

Before you use assignment matrices, an administrator must configure the environment. For example, by updating object style sheets so assignment matrices are visible and by extending the available options in key dropdown lists used when creating assignment matrices.

2. Create an assignment matrix definition

Access **Assignment Matrix** to begin. This is where you will create an assignment matrix definition that includes the details that will apply to an assignment matrix.

From the Launcher, click **Assignment Matrix**.

If this option is unavailable, search for it. You can pin it from the search results for future access.

3. Create and populate an assignment matrix based on the assignment matrix definition

Once created, the assignment matrix uses the pre-existing definition to determine user participation once it is applied to a specific business object.

4. Add and modify the AMX handler

A task must include the AMX handler to use an assignment matrix. The AMX handler sets participants based on target properties (which are specified in the assignment matrix definitions). The handler reads various fields from the assignment matrix and identifies an assignment type specified in its configuration. Then the workflow is automatically routed accordingly. Use this handler to set dynamic participants on the target object as well.

Understanding assignment matrices using a mail-sorting example

This example uses a mail-sorting analogy to explain how Teamcenter finds the right assignment matrix, chooses assignees, and handles dynamic participants.

The mail-sorting analogy

An assignment matrix works like an automated mail-sorting facility. Packages arrive, the system reads their labels, and routes them to the right people. No one has to stand there and sort by hand.

How it works

The workflow is your incoming mail: Documents (change requests, engineering specs, design reviews) enter your workflow like packages arriving at a mail room. Each one needs to reach the right people.

The assignment matrix is your sorting rulebook: You create rules once that say:

- Engineering Drawing + Urgent + Aerospace → Send to Lead Engineer, copy Quality Team
- Change Request + Cooling System + Elaborating → Send to Mechanical Engineering team

The system applies these rules automatically every time.

Setup versus runtime

You do the setup once. After that, Teamcenter runs the rules every time a workflow executes.

Setup (before workflows run):

An administrator sets up the mail-sorting facility. They write rulebooks that say, for example, “aerospace packages go to Building 3; automotive packages go to Building 5.” Then they program the sorting machine with those rules. The machine can now route every package automatically.

In Teamcenter, this means:

- Creating assignment matrix definitions
- Adding entries with properties/conditions and assignees
- Adding the assignment matrix handler to workflow tasks

Runtime (what happens every time):

When a package arrives, the system reads the label, finds the right rulebook, identifies the recipient, and delivers it instantly. No one has to make a decision in the moment.

In Teamcenter, when a document enters a workflow, the following happens:

- Teamcenter reads its properties
- Finds the right assignment matrix (discovery)
- Identifies the users (assignee retrieval)
- Assigns tasks immediately

The key difference: Setup is manual work done once by an administrator. Runtime is automatic processing that happens every time without human intervention.

Two key operations

The assignment matrix system performs two key operations: discovery and assignee retrieval.

Finding the right rulebook through discovery

Think of discovery as choosing the right rulebook for each piece of mail. Your campus might have different rulebooks for different mail types (standard documents, engineering documents, aerospace engineering documents).

When the **AMX_discover_assignment_matrix_per_target_object** preference is set to **true** (default), the system uses the careful approach. It checks each package individually for the most specific rulebook.

- Is there an Aerospace Engineering rulebook? If yes, use it.
- If not, check broader categories (Engineering, then Documents). This provides the most accurate routing.

When the **AMX_discover_assignment_matrix_per_target_object** preference is set to **false** (legacy), the system uses the quicker approach. It grabs the first matching rulebook for the group. This may be faster, but it can miss more specialized rules.

Following the rules for assignee retrieval

Once the system finds the right rulebook (discovery), it follows the instructions to determine who actually gets the work (assignee retrieval).

- The rulebook might say: **Send to Change Specialist**
- The system then resolves that role to real people, for example:
 - Julie for aerospace
 - Connor for automotive
- The system routes accordingly.

Dynamic participants and role-based routing

Instead of writing, "send to Julie" (which might create a problem when Julie leaves), you write "send to whoever has the Change Specialist role."

The system figures out who that is, based on:

- **The package itself:** Who has the Change Specialist role on this change request?
- **The project:** Who is the Project Manager for the Cooling System project?

This keeps your routing rules reliable even as people and teams change. For example, if no change specialist is configured on the change notice or request, the rule is silently ignored rather than treated as an error.

Why it matters

Without assignment matrices:

- Someone makes manual decisions for every workflow
- Routing can be inconsistent
- Work can be delayed or stuck in bottlenecks

With assignment matrices:

- You define the rules once and reuse them
- Routing is consistent and immediate
- You remove human bottlenecks

Example of multiple assignment sources

Imagine there is one package that must go to multiple departments (Engineering, Quality, and Manufacturing). Each department has its own assignment source object, and each assignment source has an associated assignment matrix that qualifies one assignee.

When the **AMX_discover_assignment_matrix_per_target_object** preference is set to **true**:

- Teamcenter checks all three assignment sources.
- Each assignment matrix qualifies its corresponding assignee: Yuvraj (Engineering), Shailesh (Quality), and Sriman (Manufacturing).
- All three assignees receive copies.

When the **AMX_discover_assignment_matrix_per_target_object** preference is set to **false**:

- Teamcenter checks the first assignment source.
- The first assignment source provides its assignment matrix.
- The assignment matrix qualifies Yuvraj (Engineering).
- The process stops.
- Only Yuvraj receives the package; Shailesh and Sriman are not assigned.

The bottom line

Assignment matrices automate workflow routing. You define rules like, "when you see these labels, send to these people," and Teamcenter does the rest. The preference controls whether it carefully checks for specialized rules or quickly applies general rules.

Administrative tasks

Using an assignment matrix in a workflow requires the following administrative tasks.

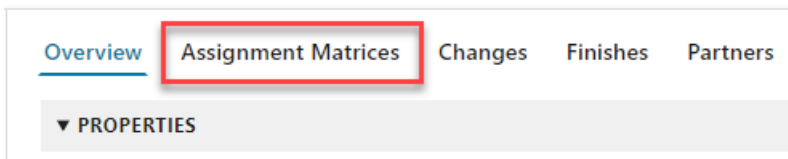
Modify the object style sheet

You can control the layout for certain portions of the declarative interface by using XML rendering templates (XRT), also called *style sheets*. These XML files are stored in the Teamcenter database and are read as needed. For this reason, changes made to these style sheets are reflected in the UI without the need to build or deploy an application file. To view the assignment matrix details for an object, you must modify the object's or project's style sheet to include the **Assignment Matrices** tab. This tab can apply to both objects and projects.

1. From the **XRT editor**, open the style sheet for the object or project that you want to contain the assignment matrix.
2. Enter the following line:

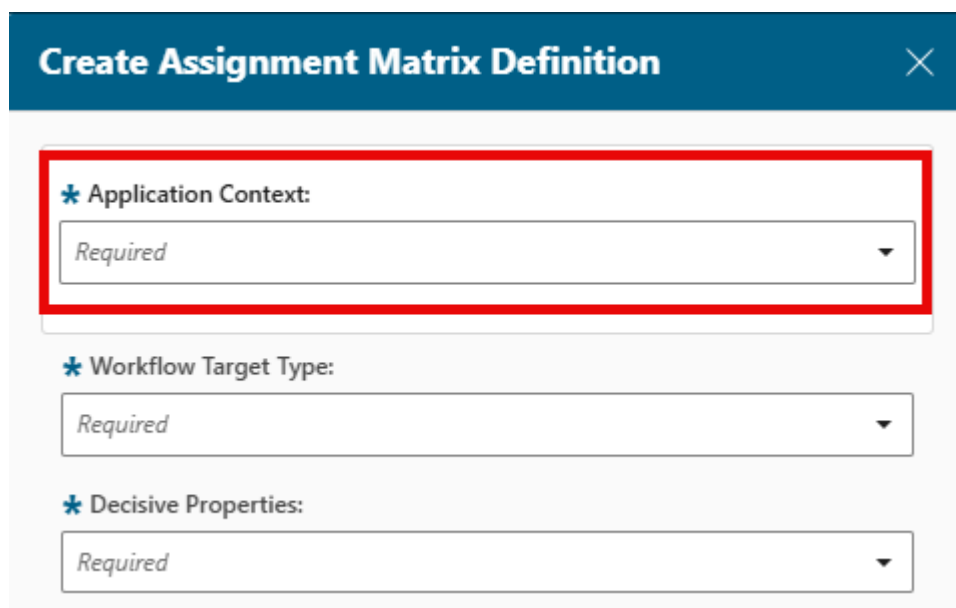
```
<inject type="dataset" src="Amx1AssignmentMatrixesPage"/>
```

The **Assignment Matrices** tab is displayed.



Add additional LOV selections for the assignment matrix

- The **Application Context** dropdown list identifies the environments that the assignment matrix will be used in.



Create Assignment Matrix Definition ✕

* **Application Context:**
Required ▼

* **Workflow Target Type:**
Required ▼

* **Decisive Properties:**
Required ▼

General is provided out-of-the-box, but additional selections can be added in BMIDE by modifying the **Amx0ApplicationContext** LOV.

- The **Assignment Type** dropdown list is used when adding entries to the assignment matrix once it has been created.

Add Entry

* Criteria Type:

☒ Decisive Properties
 ☐ Condition

▼ Criteria

Property Name	Operator	Property Value	
Description	Is Any		
Technical Review Priority	Is Any		
Recurring Cost	Is Any		

* Assignment Type:

Required

▼ Users

+

Add

▼ Teams

+

Add

▼ Key Roles

+

Add

Additional selections can be added in BMIDE by modifying the **Amx0AssignmentType** LOV.

Add BMIDE conditions to determine task assignments

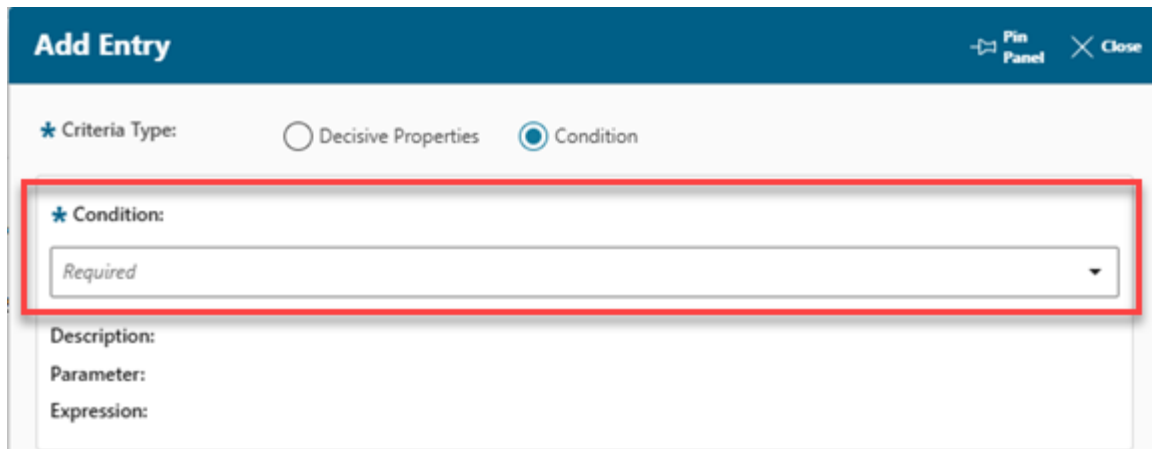
You can establish conditions in BMIDE as a prerequisite for effectively populating the assignment matrix. While optional, these conditions can include operators like "or" and others not supported by decisive properties to evaluate whether specific attributes meet the criteria for task assignment.

Note the following about BMIDE conditions:

- **Condition Naming:** Ensure the condition name includes **AMX**.
- **Condition Syntax:** Use exactly two parameters:
 - **Workflow target type**
 - **User Session**
- **Condition Signature:** The condition must take **User Session** and **AMX** target type as inputs. The AMX target type must match exactly.

- **Targets:** All targets are considered when evaluating AMX rows, regardless of conditions or decisive properties.

These conditions appear in the **Add Entry** panel's **Condition** dropdown list.



The screenshot shows the 'Add Entry' panel with a blue header bar containing 'Pin Panel' and 'Close' icons. Below the header, there are two radio buttons for 'Criteria Type': 'Decisive Properties' (unselected) and 'Condition' (selected). A red rectangular box highlights the 'Condition' dropdown menu, which currently displays 'Required'. Below the dropdown, there are labels for 'Description:', 'Parameter:', and 'Expression:'.

Assignment matrix discovery process for workflow objects

An administrator can control the discovery mode using the **AMX_discover_assignment_matrix_per_target_object** preference setting. This preference affects both assignment matrix discovery and assignee retrieval operations.

Teamcenter supports two approaches to discover and apply the assignment matrix:

True (the default)

Teamcenter discovers assignment matrices individually for each target object and evaluates type hierarchies. This is more accurate but requires more processing time.

False (legacy behavior)

Teamcenter discovers one assignment matrix for a group of target objects and applies it to all objects in that group. This is faster but less precise.

See [The assignment matrix discovery process](#) for more detailed information and examples.

Configure property display format for Amx0PropNameValueDouble objects

The site preference **Amx0PropNameValueDouble.amx0PropertyValue.FORMATSTRING** defines how the property **amx0PropertyValue** is displayed for objects of type **Amx0PropNameValueDouble**. This preference uses a format string, with the default value set to **##.##**. In this format, the **#** symbol represents a digit, which will be shown if it enhances the number's accuracy. However, it will not display leading zeros or zeros at the end of a decimal. This preference is enabled by default.

The assignment matrix discovery process

The **AMX_discover_assignment_matrix_per_target_object** preference controls how Teamcenter discovers assignment matrices and assigns tasks in workflows. Understanding how the current and legacy behaviors differ helps you set up workflows that manage the trade-off between precision and efficiency.

When you submit an object to a workflow, Teamcenter automatically searches for and discovers the appropriate assignment matrix to use for that object. Teamcenter looks at your object's type and properties, finds a matching assignment matrix, and then uses that matrix to assign the right people to workflow tasks. The **AMX_discover_assignment_matrix_per_target_object** preference controls how this search happens.

When you submit an object to a workflow, Teamcenter automatically searches for and discovers the appropriate assignment matrix to use for that object. Teamcenter looks at your object's type and properties, finds a matching assignment matrix, and then uses that matrix to assign the right people to workflow tasks. The **AMX_discover_assignment_matrix_per_target_object** preference controls how this search happens.

Two aspects of assignment matrix processing

When a workflow processes target objects, Teamcenter performs two separate operations:

- **Assignment matrix discovery**

Teamcenter finds which assignment matrix to use for the target objects.

- **Assignee retrieval**

Teamcenter uses the discovered assignment matrix to identify the actual users who should be assigned to tasks.

The **AMX_discover_assignment_matrix_per_target_object** preference, set by an administrator, affects both operations. Consider it this way: assignment matrix discovery is like finding the right instruction manual on a shelf, and assignee retrieval is like following that manual to assign people to tasks.

How the preference works

The **AMX_discover_assignment_matrix_per_target_object** preference has two values:

Preference = true (the default)

When the preference is set to **true**, Teamcenter uses an individual object-based discovery approach.

Assignment matrices can be hosted in different locations:

- Directly on the target object itself
- On an assignment source object (for example, a project that contains multiple item revisions)

- Added as a workflow reference object

When a target object has a **-relation/-property/-relationS2P** configured on an assignment source object, Teamcenter searches the assignment source object for assignment matrices. It does not search the target object itself. This provides flexibility in managing assignment matrices across multiple objects that share common assignment requirements.

Here, the example shows what happens when assignment matrices are hosted directly on the target objects:

1. Each workflow target object is evaluated independently.
2. For each workflow target object, Teamcenter retrieves its actual type.
3. Teamcenter searches for an assignment matrix associated with that specific type.
4. If no assignment matrix is found, Teamcenter moves up the type hierarchy to parent types until an assignment matrix is discovered or the **WorkspaceObject** type is reached.
5. Each target object can use a different assignment matrix based on its own type or its parent type hierarchy. For example, if the target object is of **Document Revision** type, you can have an assignment matrix of **Document Revision** type as well as an assignment matrix of **Item Revision** type (its parent type). However, you cannot have two assignment matrices of the same type (for example, two **Document Revision** assignment matrices) on a target object.

If assignment matrices are hosted on multiple assignment source objects:

1. With the preference set to true, Teamcenter uses a two-step process to resolve assignees:
 - a. First, Teamcenter iterates through all target objects.
 - b. Then, for each target object, Teamcenter iterates through all assignment source objects.
2. Teamcenter checks each workflow target object and its corresponding assignment source object for an assignment matrix, and then resolves assignees accordingly
3. The discovery does not stop at the first assignment matrix. All relevant sources are considered.

Preference = false (legacy behavior)

When the preference is set to **false**, Teamcenter uses a group-based discovery approach:

1. The assignment matrix handler identifies all candidate target objects based on the **-target_type** argument.
2. Teamcenter loops through the candidate target objects (or assignment source objects) to discover an assignment matrix. When the first assignment matrix is found on a specific object, the discovery process stops.
3. The discovered assignment matrix is then applied to all target objects in the group, regardless of their individual characteristics.

For applications with multiple assignment source objects, the same logic applies. With the preference set to **false**, Teamcenter executes a loop over all the assignment source objects and stops as soon as it finds an assignment matrix on one of them.

Note

To check the current preference setting in your system, contact your administrator. The **AMX_discover_assignment_matrix_per_target_object** preference is set at the system level and requires administrator access to view or modify.

Performance considerations

The assignment matrix handler functionality (when the preference is set to **true**) performs a more comprehensive assignment matrix discovery by evaluating each target object individually and searching through the type hierarchy. This thorough discovery process generally requires more execution time compared to the legacy (**false** preference) behavior. Consider this performance trade-off when deciding which mode to use for your workflows.

When to use each mode

Use **true** when you want the system to look carefully at each item on its own, including its type and specific details. This gives you the most accurate task assignments, but it takes more time to process.

Use **false** (legacy behavior) when all your items are basically the same type and you want things to run faster. In this mode, Teamcenter figures out the assignments once and reuses them for all items in the group, which is quicker but less precise for each individual item.

See [Discovery scenarios for assignment matrices](#) and [Assigning from multiple assignment sources](#) for more information.

What you will notice in your workflows

The preference setting affects your workflow results in these situations:

- When your workflow processes objects of different types (for example, both **DocumentRevisions** and **PartRevisions**), the **true** setting ensures that Teamcenter searches for an assignment matrix for each target object specifically. If it finds an assignment matrix, it applies it only to that target object.

With the **false** setting, target objects are segregated into buckets based on their type (for example, all **DocumentRevisions** in a "**DocumentRevisions**" bucket and all **PartRevisions** in a "**PartRevisions**" bucket). For each bucket, Teamcenter loops through the target objects to find an assignment matrix. When an assignment matrix is found on any target object in that bucket (for example, the second **DocumentRevision** or the fifth **PartRevision**), the discovery process stops. That assignment matrix is applied to all target objects in that bucket.

- When workflow target objects have multiple related assignment source objects, and each assignment source object has its own assignment matrix (for example, multiple specification documents), the **true**

setting processes all assignment source objects. It discovers all assignment matrices and uses all of them to determine the assignees.

With the **false** setting, Teamcenter loops over the assignment source objects and stops as soon as it finds the first assignment matrix. It then applies that discovered assignment matrix to all target objects in the bucket.

- If a single target object has multiple assignment matrices configured which fall into the parent hierarchy (for example, **Document Revision**, **ItemRevision**), then your objects are configured in the assignment matrix handler, both preference values (**true** and **false**) walk up the type hierarchy. The difference is where the walk-up starts and when it stops:
 - With the **true** setting, the walk-up starts from the actual target object type name of the current target object being processed. Teamcenter walks up the type hierarchy from that type until the **WorkspaceObject** type is reached, and collects all applicable assignment matrices for that target object. As Teamcenter walks up the hierarchy, the type name used to search for assignment matrices changes at each level.

For example, if the target object is of **Document Revision** type, Teamcenter first searches for an assignment matrix of **Document Revision** type. If found, it collects that assignment matrix and continues walking up to the parent type (**Item Revision**), then searches for an assignment matrix of **Item Revision** type, and so on until it reaches **WorkspaceObject**. Note that the discovery process does not stop at the first assignment matrix.

- With the **false** setting, the walk-up starts from the target type name configured in the assignment matrix handler (**target_type** argument). Teamcenter starts the assignment matrix discovery process with this **target_type** name. If no assignment matrix is found, the process stops. Otherwise, Teamcenter walks up the type hierarchy from that configured type until the **WorkspaceObject** type is reached to discover the assignment matrix.

As Teamcenter walks up the hierarchy, it changes the type name used to search for assignment matrices at each level. For example, if the assignment matrix **-target_type** is **Document Revision**, and the workflow target object has assignment matrices of both **Document Revision** and **Item Revision**, Teamcenter first searches for an assignment matrix of **Document Revision** type. Since Teamcenter finds the assignment matrix with the **Document Revision** type, the discovery process stops (and the walk-up does not happen). Note that with the true setting, Teamcenter continues the walk-up through the type hierarchy until it reaches **WorkspaceObject**, even after it finds an assignment matrix. Note that the discovery process stops as soon as assignment matrix is found.

Discovery scenarios for assignment matrices

These examples illustrate how the **AMX_discover_assignment_matrix_per_target_object** preference changes the way Teamcenter finds assignment matrices and assigns users.

Scenario 1

This example shows how the **AMX_discover_assignment_matrix_per_target_object** preference affects whether Teamcenter can discover an assignment matrix when the AMX-auto-assign-task-assignees-from-assignment-matrix handler uses a parent type but only the specific workflow target object type has an assignment matrix.

Situation

You are organizing a document review workflow. You have an instruction manual specifically for Document Revisions (**DocumentRevision_AMX_1**), but your workflow handler is configured to look for Item Revisions (the parent type).

Your workflow target object is **DocRev_1**, which is a Document Revision. Document Revision is a specialized type that inherits from Item Revision.

Setup details

- The handler argument **target_type** = **ItemRevision**
- Workflow target object = **DocRev_1** (DocumentRevision type)
- Assignment matrix available: **DocumentRevision_AMX_1** (associated with DocumentRevision type only)

When the preference is set to true:

This is the default setting. Teamcenter looks at the actual workflow target object type and finds the specific manual:

1. Teamcenter evaluates **DocRev_1** individually.
2. Teamcenter retrieves the actual type of **DocRev_1:DocumentRevision**.
3. Teamcenter searches for an assignment matrix for DocumentRevision type.
4. Teamcenter discovers **DocumentRevision_AMX_1** and uses it to assign assignees.
5. Teamcenter continues up the type hierarchy to the parent type (ItemRevision).
6. No assignment matrix is found for **ItemRevision** type.
7. Teamcenter stops when it reaches WorkspaceObject (the top of the hierarchy).

Result: The workflow successfully discovers the **DocumentRevision_AMX_1** matrix because Teamcenter evaluates the actual object type (DocumentRevision) rather than just the handler's target type (ItemRevision).

When the preference is set to false:

This setting represents the legacy behavior. Teamcenter looks for a manual based on the handler's **-target_type** and cannot find one:

1. The handler specifies ItemRevision as the workflow target type.
2. Teamcenter searches for an assignment matrix associated with ItemRevision type.
3. The only available matrix is **DocumentRevision_AMX_1**, which is associated with DocumentRevision type.

4. The type names do not match exactly (ItemRevision does not match DocumentRevision).
5. No assignment matrix is discovered.
6. No assignees can be returned because no assignment matrix was found.

Result: The workflow cannot find an assignment matrix because the handler's target type (ItemRevision) does not exactly match the assignment matrix's workflow target type (DocumentRevision).

Key difference

When the preference is set to true, Teamcenter first checks the actual type of the object at runtime. It sees that **DocRev_1** is a DocumentRevision, then looks for matrices for DocumentRevision and finds **DocumentRevision_AMX_1**. This approach works better when workflow target objects use more specific types than the **General** application context type configured in the handler.

When the preference is set to false (the legacy behavior), Teamcenter cannot find the assignment matrix. It only looks for an exact text match between the handler's **-target_type** and the assignment matrix type. In this example, the handler uses ItemRevision, but the only assignment matrix available is **DocumentRevision_AMX_1**. Since the names do not match (ItemRevision vs. DocumentRevision), Teamcenter does not find an assignment matrix.

Scenario 2

This example shows how the **AMX_discover_assignment_matrix_per_target_object** preference affects assignment matrix discovery when the handler uses a parent type and only the parent type has an assignment matrix.

Situation

You are organizing a document review workflow. You have an instruction manual for Item Revisions (**ItemRev_AMX_1**), but not for Document Revisions specifically.

Your workflow target object is **DocRev_1**, which is a Document Revision. Document Revision is a specialized type that inherits from Item Revision. The handler is configured to look for Item Revisions.

Setup details

- Handler **target_type** = ItemRevision
- Workflow target object = **DocRev_1** (DocumentRevision type)
- Assignment matrix available: **ItemRev_AMX_1** (associated with ItemRevision type only)

When the preference is set to true:

This is the default setting. Teamcenter looks at the actual object type first, then traverses up the type hierarchy:

1. Teamcenter evaluates **DocRev_1** individually.
2. Teamcenter retrieves the actual object type of **DocRev_1: DocumentRevision**.
3. Teamcenter searches for an assignment matrix for DocumentRevision type.
4. No assignment matrix is found for DocumentRevision.
5. Teamcenter continues up the type hierarchy to the parent type (ItemRevision).
6. Teamcenter discovers **ItemRev_AMX_1** (the assignment matrix for ItemRevision).
7. Teamcenter uses **ItemRev_AMX_1** to return assignees.
8. Teamcenter stops when it reaches WorkspaceObject (the top of the hierarchy).

Result: The workflow successfully discovers **ItemRev_AMX_1** by searching the type hierarchy. Even though no assignment matrix exists for the specific DocumentRevision type, Teamcenter finds the parent type's assignment matrix.

When the preference is set to false:

This setting represents the legacy behavior. Teamcenter looks for a manual based on the handler's target type and finds it immediately:

1. The handler specifies ItemRevision as the target type.
2. Teamcenter searches for an assignment matrix associated with ItemRevision type.
3. Teamcenter discovers **ItemRev_AMX_1** (the assignment matrix for ItemRevision).
4. Teamcenter uses **ItemRev_AMX_1** to return assignees.
5. The process completes.

Result: The workflow successfully discovers **ItemRev_AMX_1** because the handler's **-target_type** argument (ItemRevision) matches the assignment matrix type exactly. The assignment matrix is found and used immediately.

Key difference

Both methods successfully discover the assignment matrix in this scenario. When the preference is set to true, Teamcenter finds it after checking the specific object type first and then searching up to the parent type. When the preference is set to false, the legacy behavior finds it immediately by matching the handler's **-target_type**. The end result is the same, but the new behavior performs additional steps to check for more specific matrices first.

Scenario 3

This example shows how assignment matrix discovery works when the assignment source object is of type **TC_Project** and demonstrates the supported project-related keywords.

Situation

You are organizing a project workflow where assignments need to be made based on project roles. Your handler configuration uses **project_list** as the property to retrieve the assignment source object. The assignment matrix uses project-related keywords to identify appropriate assignees from the project team structure.

Setup details

- Handler property = **project_list**
- Assignment source object = **Project_1** (TC_Project type)
- Assignment matrix available: **Project_AMX_1** (uses project-related keywords)

Supported keywords for TC_Project assignment source objects

When the assignment source is of type **TC_Project**, only the following project-related keywords are supported in the assignment matrix:

- **\$PROJECT_ADMINISTRATOR**
- **\$PROJECT_TEAM_ADMINISTRATOR**
- **\$PROJECT_AUTHOR**
- **\$PROJECT_MEMBER**

How it works

1. The handler configuration includes **-property = project_list**.
2. Teamcenter uses the **project_list** property to retrieve the assignment source **TC_Project** object.
3. Teamcenter uses the **TC_Project** object to fetch the assignment matrix.
4. The assignment matrix rows can specify assignments from the workflow target object or use project-related keywords from the assignment source object (the **TC_Project**).
5. Teamcenter evaluates the project-related keywords in the assignment matrix row (such as **ASSIGNMENT SOURCE: \$PROJECT_ADMINISTRATOR**).
6. Teamcenter retrieves the Project Administrator assigned to **Project_1** and adds the user as a reviewer.

Result: The workflow successfully discovers **Project_AMX_1** and assigns relevant users based on the configuration. The project-related keywords enable dynamic assignment based on the project team structure.

Keyword support based on assignment source type

The type of assignment source object determines which participant type names are supported:

- If the assignment source is of type **WorkspaceObject**, all participant type names are supported.

- If the assignment source is of type **TC_Project**, only the four project-related keywords (**PROJECT_ADMINISTRATOR**, **PROJECT_TEAM_ADMINISTRATOR**, **PROJECT_AUTHOR**, and **PROJECT_MEMBER**) are supported.
- If the assignment source is of type **POM_application_object** but not of type **TC_Project**, no participant type names are supported. Teamcenter does not display an error or warning to the user when the assignment matrix row is configured this way.

Key difference

Using **TC_Project** as an assignment source enables project-based assignments through specialized keywords. This approach works when you need to assign workflow tasks based on project team membership rather than object ownership or other relationships. The handler retrieves the project object through the **-property** parameter, which can be configured as **owning_project** or **project_list**. The assignment matrix uses project-specific keywords to identify the appropriate assignees from the project team structure.

Assigning from multiple assignment sources

These examples show how the **AMX_discover_assignment_matrix_per_target_object** preference influences assignee selection. The preferences apply when multiple assignment source objects have their own assignment matrices.

Scenario 1

This example shows how the preference affects which users are assigned to tasks when two assignment source objects each have their own unique assignment matrix.

Situation

You are managing a review workflow for one item. The item is related to two assignment source objects via the **IMAN_specification** relation. Each assignment source object has its own assignment matrix with a reviewer defined. You want to determine which reviewers are assigned to the Review task.

Item 1 (**IR_1**) is related to:

- **assignment source 1** which has **assignment source 1 user 1** as a reviewer
- **assignment source 2** which has **assignment source 2 user 1** as a reviewer

Each **assignment_source** object has its own assignment matrix configured.

Setup details

- One workflow target object: **IR_1** (ItemRevision type)
- Assignment matrix handler arguments: **relation=IMAN_specification**

- Two assignment source objects: **assignment source 1**, **assignment source 2** (both ItemRevision type)
- Reviewers defined:
 - **assignment source 1 user 1** is a reviewer on **assignment source 1**
 - **assignment source 2 user 1** is a reviewer on **assignment source 2**
- Assignment matrices:
 - **assignment source 1** has **assignment source 1 AMX**
 - **assignment source 2** has **assignment source 2 AMX**
- Assignment matrix row configuration: Both have **participant type = assignment source:\$Reviewer** (meaning "assign the reviewer from the assignment source object")

When the preference is set to true:

This is the default setting. This setting represents the default Teamcenter processes all assignment source objects individually:

1. Teamcenter loops through the assignment source objects related to **IR_1**.
2. **assignment source 1** is found and has **assignment source 1 AMX** associated with it.
3. **assignment source 1 AMX** is used to determine the participant assignee.
4. The **assignment source:\$Reviewer** token designates **assignment source 1 user 1** as the assignee.
5. **assignment source 1 user 1** is added as a reviewer on the task.
6. Teamcenter continues processing the next assignment source object.
7. **assignment source 2** is found and has **assignment source 2 AMX** associated with it.
8. **assignment source 2 AMX** is used for discovering the participant assignee.
9. The **assignment source:\$Reviewer** token designates **assignment source 2 user 1** as the assignee.
10. **assignment source 2 user 1** is added as a reviewer on the task.

Result: Two reviewers are assigned (**assignment source 1 user 1** and **assignment source 2 user 1**) because Teamcenter processes each assignment source object individually and uses each assignment matrix found.

When the preference is set to false:

This setting represents the legacy behavior. Teamcenter stops at the first assignment matrix found:

1. Teamcenter loops through the assignment_source objects related to **IR_1**.
2. **assignment source 1** is found and has **assignment source 1 AMX** associated with it.
3. As soon as **assignment source 1 AMX** is found, further assignment source processing stops.
4. **assignment source 1 AMX** is used for participant assignee discovery.

5. The **assignment source:\$Reviewer** token designates **assignment source 1 user 1** as the assignee.
6. **assignment source 1 user 1** is added as a reviewer on the task.
7. **assignment source 2** is never processed.

Result: Only one reviewer is assigned (**assignment source 1 user 1**) because Teamcenter stops processing after finding the first assignment matrix.

Scenario 2

This example shows how the preference affects which users are assigned to tasks when one workflow target object is related to multiple assignment source objects, and only some assignment source objects have assignment matrices.

Situation

You are managing a specification review workflow for one item. The item is linked to four different specification documents (assignment source objects). Only two of those specifications have assignment matrices that define who should review them.

Item 1 (**IR_1**) is linked to four specifications:

- Specification A (**assignment source 1**) has **assignment source 1 AMX**
- Specification B (**assignment source 2**) has no assignment matrix
- Specification C (**assignment source 3**) has **assignment source 3 AMX**
- Specification D (**assignment source 4**) has no assignment matrix

You want to assign reviewers based on who is responsible for each specification.

Setup details

- One workflow target object: **IR 1** (ItemRevision type)
- The assignment matrix handler argument: **relation=IMAN specification**
- Four assignment source objects: **assignment source 1**, **assignment source 2**, **assignment source 3**, **assignment source 4** (all ItemRevision type)

Reviewers defined on assignment source objects include:

- **assignment source 1 user 1** is a reviewer on **assignment source 1**
- **assignment source 2 user 1** is a reviewer on **assignment source 2**
- **assignment source 3 user 1** is a reviewer on **assignment source 3**
- **assignment source 4 user 1** is a reviewer on **assignment source 4**

Assignment matrices include:

- **assignment source 1** has **assignment source 1 AMX**
- **assignment source 2** has no assignment matrix
- **assignment source 3** has **assignment source 3 AMX**
- **assignment source 4** has no assignment matrix

Assignment matrix row configuration:

Participant type = assignment source:\$Reviewer (meaning "assign the reviewer from the assignment source object")

When the preference is set to true:

This is the default setting. Teamcenter evaluates each `assignment_source` object individually:

1. Teamcenter loops through all `assignment_source` objects associated with IR 1.
2. **assignment source 1**: Teamcenter locates **assignment source 1 AMX** and uses it. The **assignment source:\$Reviewer** token specifies **assignment source 1 user 1**, and adds it as a reviewer on the task.
3. **assignment source 2**: No assignment matrix is associated with **assignment source 2**. This assignment source is skipped.
4. **assignment source 3**: Teamcenter locates **assignment source 3 AMX** and uses it. The **assignment source:\$Reviewer** token designates **assignment source 3 user 1** and adds it as a reviewer on the task.
5. **assignment source 4**: No assignment matrix is associated with **assignment source 4**. This assignment source is skipped.
6. Both **assignment source 1 user 1** and **assignment source 3 user 1** are added as reviewers on the task.

Result: Two reviewers are assigned (**assignment source 1 user 1** and **assignment source 3 user 1**) because Teamcenter processes all assignment source objects and uses each assignment matrix it locates. Assignment source objects without matrices are skipped.

When the preference is set to false:

This setting represents the legacy behavior. Teamcenter locates the first assignment matrix and stops looking.

1. Teamcenter loops through the assignment source objects (**assignment source 1**, **assignment source 2**, **assignment source 3**, **assignment source 4**) looking for any assignment matrix.
2. Teamcenter locates **assignment source 1 AMX** on **assignment source 1**.
3. Teamcenter stops looking immediately and uses **assignment source 1 AMX**.

4. The **assignment source:\$Reviewer** token designates **assignment source 1 user 1** (the reviewer on **assignment source 1**).
5. **assignment source 1 user 1** is added as a reviewer on the task.
6. **assignment source 2**, **assignment source 3**, and **assignment source 4** are never processed.

Result: Only one reviewer is assigned (**assignment source 1 user 1**) because Teamcenter stops processing as soon as it locates the first assignment matrix. The other assignment source objects and their assignment matrices are never evaluated.

Key difference

The legacy behavior stops processing assignment source objects as soon as the first assignment matrix is found, potentially missing other relevant matrices and assignees.

When the preference is set to true, Teamcenter processes all assignment source objects individually, discovering and using all available assignment matrices to identify all relevant assignees.

Scenario 3

This example shows how the preference affects which users are assigned to tasks when multiple assignment matrices are configured directly on the target object itself.

Situation

You are managing a review workflow for a document revision. The document revision has two assignment matrix instances configured on it: one of ItemRevision type and another of DocumentRevision type. The assignment matrix handler is configured to use the DocumentRevision type. You want to determine which reviewers are assigned to the Review task.

Document Revision 1 (**DR_1**) has two assignment matrices configured:

- **DR_1 ItemRevision AMX** (ItemRevision type) which has **ItemRev user 1** as a reviewer
- **DR_1 DocumentRevision AMX** (DocumentRevision type) which has **DocRev user 1** as a reviewer

Setup details

- One workflow target object: **DR_1** (DocumentRevision type)
- Assignment matrix handler argument: **target_type=DocumentRevision**
- Two assignment matrix instances on **DR_1**:
 - **DR_1 ItemRevision AMX** (ItemRevision type)
 - **DR_1 DocumentRevision AMX** (DocumentRevision type)
- Reviewers defined:
 - **ItemRev user 1** is a Reviewer in **DR_1 ItemRevision AMX**

- **DocRev user 1** is a Reviewer in **DR_1 DocumentRevision AMX**

When the preference is set to true:

This is the default setting. Teamcenter processes all assignment matrix instances by starting from the target object type and then walking up the parent type hierarchy:

1. Teamcenter evaluates **DR_1** for assignment matrices.
2. The handler argument **-target_type=DocumentRevision** filters the assignment matrices.
3. **DR_1 DocumentRevision AMX** is found and matches the DocumentRevision type.
4. **DR_1 DocumentRevision AMX** is used to determine the participant assignee.
5. The reviewer token designates **DocRev user 1** as the assignee.
6. **DocRev user 1** is added as a reviewer on the task.
7. Teamcenter continues the discovery process by walking up the parent hierarchy, from **DocumentRevision** to **ItemRevision**.
8. For **ItemRevision** type, **DR_1 ItemRevision AMX** is found.
9. The reviewer token designates **ItemRev user 1** as the assignee.
10. **ItemRev user 1** is added as a reviewer on the task.

Result: Two assignees are added as a reviewer: **DocRev user 1** and **ItemRev user 1**.

When the preference is set to false:

This setting represents the legacy behavior. Teamcenter stops at the first assignment matrix instance of the specified type:

1. Teamcenter evaluates **DR_1** for assignment matrices.
2. The handler argument **target_type=DocumentRevision** filters the assignment matrices.
3. **DR_1 DocumentRevision AMX** is found and matches the DocumentRevision type.
4. As soon as **DR_1 DocumentRevision AMX** is found, further assignment matrix processing stops, and Teamcenter does not walk up the parent hierarchy.
5. **DR_1 DocumentRevision AMX** is used for participant assignee discovery.
6. The reviewer token designates **DocRev user 1** as the assignee.
7. **DocRev user 1** is added as a reviewer on the task.

Result: One reviewer is assigned (**DocRev user 1**) because Teamcenter stops processing after finding the first assignment matrix instance of the specified type.

Key difference

- With the preference set to true, Teamcenter continues discovery after the first match: it walks up the parent hierarchy and processes additional assignment matrices, so both **DocRev user 1** and **ItemRev user 1** are assigned as reviewers.
- With the preference set to false, Teamcenter stops immediately after the first matching assignment matrix is found on the target object and does not walk up the hierarchy. Only **DocRev user 1** is assigned as a reviewer.

Create an assignment matrix definition

An assignment matrix definition includes the details to apply to an assignment matrix, based on the target object type. It acts as a container that holds all the rules for an object to proceed through a workflow without user interaction. The assignment matrix uses this definition to determine user participation once it is applied to a specific object.

Any user can create an assignment matrix definition by default. However, administrators can configure Access Manager rules to restrict who is allowed to create them.

Procedure

1. From the **Launcher**, click **Assignment Matrix**.

If this option is unavailable, search for it. You can pin it from the search results for future access.

The available assignment matrices display under the **Matrices** tab.

2. Click the **Definitions** tab.
Available assignment definitions display.
3. Go to **More Commands** *** > **New** ✱ > **Create Assignment Matrix Definition**  from the **Page level** toolbar.
4. Select an existing **Application Context** from the drop-down.

The **Application Context** identifies the environment that the assignment matrix will be used in. This must be a unique value.

5. Select the **Workflow Target Type** that contains the assignment matrix. This must be a unique value.
6. Select the **Decisive Properties** for the target from the drop-down menu.

Decisive properties are values that describe how to identify the selected target so that the assignment matrix can assign participants. See [Conditions and decisive properties](#) for more information.

Note

Runtime properties are not supported for decisive properties. In addition, properties that use interdependent lists of values (LOVs) are not supported.

7. Click **Create**.
The new definition is listed on the **Definitions** tab. Once selected, the properties are displayed for the definition. References can be added later as well.

Assignment Matrix

Matrices

Definitions

34 results found for "Definitions"

List

Selection Mode

Select All

Find in this content

AMX_Operator-AMX Item Revision

Kazukiewicz, Julie (jak)

Last Modifying User: Kazukiewicz, Julie (jak)

Date Modified: 06-Feb-2026

Operator4_96035705-CFx_Property_Support_ItemR...

AW_AMX_4 (aw_amx_4)

Last Modifying User: AW_AMX_4 (aw_amx_4)

Date Modified: 06-Feb-2026

Operator5_17096069-CFx_Property_Support_ItemR...

AW_AMX_5 (aw_amx_5)

Last Modifying User: AW_AMX_5 (aw_amx_5)

Date Modified: 06-Feb-2026

AMX_Multi_1_83303073-AMX Item Revision

AW_AMX_4 (aw_amx_4)

Last Modifying User: AW_AMX_4 (aw_amx_4)

Date Modified: 06-Feb-2026

Examples

The following example shows an assignment matrix definition for the **Industrial Systems** application context with a target type of **Change Request Revision**.

Industrial Systems-Change Request Revision

Industrial Systems-Change Request Revision

Owner: Admin, AMX1 (amxadmin1)

Creation Date: 03-Dec-2025

▼ Properties

Application Context: Industrial Systems

Target Type: Change Request Revision

Decisive Properties: Description Change Maturity Maturity

Owner: Admin, AMX1 (amxadmin1)

Group ID: dba

Creation Date: 03-Dec-2025

Workflow Designer on Active Workspace
Unpublished work. © 2026 Siemens

7-25
Software Version 2606

Properties:

- **Application Context:** Industrial Systems
- **Target Type:** Change Request Revision
- **Decisive Properties:** Description, Change Maturity, Maturity

This definition enables assignment matrix rules to evaluate these three properties when determining task assignments.

What to do next

Create and populate an assignment matrix.

Create and populate an assignment matrix

The assignment matrix contains the assignment details for the target object as defined in the assignment matrix definition. After you create the assignment matrix, you can populate it with specific properties. This ensures the correct assignees are identified for the object during a workflow.

Prerequisites

- An assignment matrix definition must be created.
- If you choose to use BMIDE conditions to define your task assignments, you must establish them before populating the assignment matrix. This ensures they appear in the list of conditions in the **Add Entry** panel.
- When using a non-workspace object (for example, **TC_project**) as an assignment source, add the **Amx0AssignmentMatrixRel** relation property to the non-workspace object type via BMIDE.
- When you add key roles to an assignment matrix, you can pick dynamic participants that fit the object type. The assignment matrix automatically assigns tasks based on those roles. See [Understanding dynamic participants in assignment matrices](#).

Procedure

1. Open the target object that you want to contain the assignment matrix.
2. Click the **Assignment Matrices** tab.

Note

The object's style sheet must be modified for this tab to appear.

3. Click **Add** ⊕ in the top-right portion of the **Assignment Matrices** table.
The **Add** dialog box opens.

4. Enter a **Name** for the assignment matrix.
5. Select the **Application Context** from the dropdown list.
The application context identifies the specific environment or scenario in which the assignment matrix operates.
6. Select the **Workflow Target Type**.
If an assignment matrix definition is identified for the target, the **Workflow Target Type** is automatically displayed.
7. Enter a **Description** for the assignment matrix.
The **Assignment Matrix Definition** is automatically selected based on the data entered, and its **Decisive Properties** are displayed.
8. Click **Open on Add** to open the assignment matrix immediately.
9. Click **Add**, and the new assignment matrix appears.

What to do next

After creating the assignment matrix, populate it with assignment entries. See [Add entries to an assignment matrix](#).

Add entries to an assignment matrix

After an assignment matrix is created, you can populate the matrix with decisive properties or conditions and assignees.

Prerequisites

- An assignment matrix is created.
- BMIDE conditions are established (if using conditions).

Procedure

1. Click **Add** above the **Entries** table.
The **Add Entry** panel opens.
2. Click the **Decisive Properties** or **Condition** button.
See [Conditions and decisive properties](#) to understand how decisive properties and conditions work with assignment matrices.
3. Configure the criteria for the entry:
 - a. If you selected **Decisive Properties**, select properties and enter values.
You can use operators such as equals, not equals, greater than, or less than. See [Assignment matrix property operators](#) for details.

- b. If you selected **Condition**, select the BMIDE condition from the dropdown list.
- 4. Select an **Assignment Type** from the dropdown list.

Assignment types include **Assignee**, **Reviewer**, **Acknowledger**, and **Notifyee**.

- 5. Add assignees to the entry.

The **Add Entry** panel is organized into three sections for adding different types of assignees: **Users**, **Teams**, and **Key Roles**.

Tip

Click the **Filter** icon  for additional search filters to specify a user's **Group** or **Role**. Click the **Filter** icon  again to see the results.

- a. (Optional) Add users as assignees.
 - i. In the **Add Entry** panel, locate the **Users** section.
 - ii. Click **Add** .
 - iii. Type the name of the user and press **Enter**, or click **Search** to display results.
 - iv. Select the user from the search results.
 - v. Click **Add** to include the user in the assignees list.

Tip

You can enter a partial name or wildcard characters in the search field.

- b. (Optional) Add teams as assignees.
 - i. In the **Add Entry** panel, locate the **Teams** (or **Resource Pools**) section.
 - ii. Click **Add** .
 - iii. Type the name of the Resource Pool and press **Enter**, or click **Search** to display results.
 - iv. Select the Resource Pool from the search results.
 - v. Click **Add** to include the team in the assignees list.

Tip

You can display filters and increase the size of the **Add Entry** panel by dragging the left edge of the panel to the left.

- c. (Optional) Add key roles as assignees.

Key roles are dynamic participants that the assignment matrix handler resolves at runtime based on workflow context. See to [Understanding dynamic participants in assignment matrices](#) to understand more about using dynamic participants in an assignment matrix.

- i. In the **Add Entry** panel, locate the **Key Roles** (or **Dynamic Participants**) section.

- ii. Click **Add** .
- iii. Select the dynamic participant source:
 - **From Workflow Target:** Type the name of the participant on the workflow target object that contains the assignee(s).
 - **From Assignment Source:** Type the name of the participant on the assignment source object that contains the assignee(s).
 - **From Others:** Select either **Process Owner** or **User Session**.
- iv. Click **Add** to include the key role in the assignees list.

Note

Key roles display with their handler keyword prefix. For example, workflow target:\$CHANGE_SPECIALIST.

6. Review the selected assignees in the **Add Entry** panel.

To remove an assignee before adding the entry, select the assignee and click **Remove** .

7. Click **Add** to add the assignment matrix entry.
The entry appears in the **Entries** table with the following columns:
 - **Criteria:** The decisive property values or condition name.
 - **Criteria Type:** Displays either **Decisive Properties** or **Condition**.
 - **Assignment Type:** The role type such as **Assignee**, **Reviewer**, **Acknowledger**, or **Notifyee**.
 - **Assignees - Users/Teams:** Group members and resource pools.
 - **Assignees - Key Roles:** Dynamic participants with handler keyword notation such as workflow target:\$CHANGESPECIALIST or workflow target:\$CHANGESPECIALISTORSOURCE:project_manager.
8. Continue adding entries to the assignment matrix as necessary.

Note

Decisive properties, their respective values, and the assignment type must be unique for each assignment matrix row.

Click **Table Settings**  > **Wrap Text** to view all assignees in each row. Wrapping text makes it easier to see all assignees when entries contain multiple users, teams, or key roles.

Results

Once created, all the available assignment matrices display under the **Matrices** tab. This tab is in the left panel of the **Assignment Matrix** tool. The **Matrices** tab contains the **Overview** and **References** tabs.

Overview tab

The **Overview** tab displays the properties and criteria for an assignment matrix. You can make [updates](#) to the assignment matrix here.

References tab

Details

AMX_4_27217635
Name: AMX_4_27217635 Type: Assignment Matrix

Edit

Overview

References

✂

📄

+

Add

⋮

✓

Select

⌵

Object	Type	Release Status	Date Released	Owner
033403/A1-AMX_Copy1_21652405	AMX Item Revision			AW_AMX_3 (aw_amx_3)
033404/A1-AMX_Copy2_74664844	AMX Item Revision			AW_AMX_3 (aw_amx_3)
AWTest_ValidatethatAMXcanbe_76546402	Folder			AW_AMX_2 (aw_amx_2)

The **References** tab shows which objects use a specific assignment matrix. For example, if three users create item revisions using the same matrix, you will see those objects listed when you select the matrix. This helps you see what depends on the matrix before you change it. You can also [add additional references](#) from this tab.

Examples

The following example shows an assignment matrix entry for a Change Request for a cooling system:

Properties

Access

Handlers

Notifications

Assignments

List Arrangements

Name:

Specialist completes Planning

Copy

Trigger	Arguments
Assigned	
Started	
EPM-set-rule-based-protect...	-acl=Task Single Assignee Write
AMX-auto-assign-task-assig...	-application_context=Industrial S...
Performed	

Information

Name: AMX-auto-assign-task-assignees-from-assignment-matrix

Argument:

Value(s):

-target_type

ChangeRequestRevision

-assignment_type

Responsible

-application_context

Industrial Systems

Criteria:

- **Description** = Change Request For Cooling System
- **Change Maturity** = Elaborating
- **Maturity** = Any
- **Criteria Type:** Decisive Properties
- **Assignment Type:** Assignee
- **Assignees - Key Roles:** workflow target : \$CHANGE_SPECIALIST

How it works: When a Change Request Revision is submitted to a workflow with these exact property values, the AMX-auto-assign-task-assignees-from-assignment-matrix handler automatically assigns all participants from the change specialist role on that target object to the task.

Understanding dynamic participants in assignment matrices

Dynamic participants enable assignment matrices to automatically assign workflow tasks based on roles defined in object relationships. You can configure dynamic participants from three sources: assignment sources, workflow target objects, and other system-defined roles.

How the assignment matrix uses dynamic participants

When you add key roles as assignees in an assignment matrix, you can select dynamic participants that are applicable to the specific object type. The assignment matrix uses these dynamic participants to automatically assign tasks based on the roles defined in the object relationships.

Sources for dynamic participants

You can configure dynamic participants from three sources:

Assignment source

An assignment source is a container that holds a set of assignment matrices. The assignment source can be a type or subtype of a Persistent Object Model (POM) application object. The assignment matrices are used in workflows that go through the assignment source.

To use dynamic participants from an assignment source, the assignment source must be a subtype of ItemRevision. Dynamic participants are not supported for other POM application object types.

When you select a dynamic participant from an assignment source, the workflow assigns tasks to users based on their roles in that source object.

Workflow target object

A workflow target object lists dynamic participants and special key roles specific to the target object type. These participants are defined by the object's properties and relationships.

For example, a Change Request Revision includes these dynamic participants:

- Proposed Responsible Party
- Proposed Approver
- Change Specialist

When you select a dynamic participant from the workflow target object, the workflow assigns tasks to the specific users defined in those roles on the target object.

Other system-defined roles

Other options for dynamic participants include system-defined roles that apply across workflows:

- **Process Owner:** The user responsible for managing and overseeing the workflow.
- **User Session:** Represents the active participation of the user who initiated the workflow.

Configuring dynamic participants

When using dynamic participants in assignment matrices, the system requires:

- Identification of which source provides the appropriate participants for your workflow.
- The assignment source or the workflow target object has the required roles defined.
- Selection of the dynamic participant when adding key roles to the assignment matrix entry.

The assignment matrix displays dynamic participants with a prefix that indicates their source:

- `workflow target:$CHANGE_SPECIALIST` indicates a participant from the workflow target object.
- `assignment source:$PROJECT_MANAGER` indicates a participant from the assignment source.

Example

A change request workflow uses an assignment matrix with the dynamic participant `workflow target:$CHANGE_SPECIALIST`. When the workflow runs, it automatically assigns the task to all users who have the Change Specialist role on that specific change request object.

Assignment matrix property operators

Property operators define conditions for task assignments based on specific property values. These operators help set criteria that determine how tasks are assigned to users in assignment matrices.

Supported property types

The operators support both singular and array (VLA) properties of the following types:

- PROP_double
- PROP_float
- PROP_int
- PROP_short
- PROP_char
- PROP_string
- PROP_date
- PROP_logical

For singular properties

The following operators apply to singular (non-VLA) properties:

- **Equal to (=)**: Checks if a property value matches a specified value.
- **Not equal to (!=)**: Checks if a property value does not match a specified value.
- **Greater than (>)**: Checks if a property value is greater than a specified value.
- **Greater than or equal to (>=)**: Checks if a property value is greater than or equal to a specified value.
- **Less than (<)**: Checks if a property value is less than a specified value.
- **Less than or equal to (<=)**: Checks if a property value is less than or equal to a specified value.
- **Is Blank**: Checks if a property value is blank or not set.
- **Is Not Blank**: Checks if a property value is not blank or is set.
- **Any** (the default)

Note

When using comparison operators with strings, the system performs a character-by-character comparison. Each character in the first string is compared sequentially with the corresponding character in the second string.

For VLA properties

The following operators apply to VLA (array) properties:

- **Equal to (=)**: Checks if a property value contains a specified value.
- **Not equal to (!=)**: Checks if a property value does not contain a specified value.
- **Greater than (>)**: Checks if a property value contains a value greater than a specified value.
- **Greater than or equal to (>=)**: Checks if a property value contains a value greater than or equal to a specified value.
- **Less than (<)**: Checks if a property value contains a value less than a specified value.
- **Less than or equal to (<=)**: Checks if a property value contains a value less than or equal to a specified value.
- **Is Blank**: Checks if a property value is blank or not set.
- **Is Not Blank**: Checks if a property value is not blank or is set.
- **Any** (the default)

Example

If you set the following criteria:

- **Property Name**: Test results attached
- **Operator**: !=
- **Property Value**: Yes

The condition **Test results attached != Yes** evaluates to true whenever the value of **Test results attached** is anything other than **Yes** (for example, **No**, or **blank**). This triggers specific actions or assignments in the workflow based on these criteria.

Conditions and decisive properties

Assignment matrices use conditions or decisive properties to match tasks with assignees. Conditions handle complex logic in BMIDE, while decisive properties provide simple matching in the assignment matrix.

What they are

You add entries to an assignment matrix. You choose between two ways to match tasks with assignees:

- **Conditions**: Complex rules you create in BMIDE that can use OR, AND, and other logic.

- **Decisive properties:** Simple property matching you configure directly in the assignment matrix. All properties must match (AND logic only).

Key differences

Feature	Conditions	Decisive properties
Where you set it up	BMIDE (before adding entries)	Assignment matrix (when adding entries)
Logic you can use	OR, AND, NOT, and other operators	AND only (all must match)
Reusability	Use in multiple matrices	Specific to one entry
BMIDE access needed	Yes	No

When to use each one

Use conditions when:

- You need OR logic. For example: Status = Approved OR Status = Pending Review.
- You want to reuse the same rule in multiple matrices.
- You need complex logic that decisive properties cannot handle.

Use decisive properties when:

- All your criteria must be true. For example: Priority = High AND Department = Engineering.
- You want quick setup without BMIDE.
- You want to use blank values as wildcards that match anything.

Examples

Condition example:

You create a BMIDE condition: (Impact Area = "Safety" OR Impact Area = "Regulatory")

When you add this condition to an entry, the task is assigned when the Impact Area is either Safety or Regulatory.

Decisive properties example:

You set these decisive properties in an entry:

- **Priority** = High
- **Department** = Engineering

- **Status** = (blank)

The task is assigned when **Priority** is High, **Department** is Engineering, and **Status** has any value.

Using blank values in decisive properties

When you leave a decisive property value blank, it matches any value on the target object. This works like a wildcard.

For example, if you set Department = Engineering and leave Priority blank, the entry matches all Engineering objects regardless of their priority.

Update an assignment matrix

After creating an assignment matrix entry, you can modify its properties and assignees on the **Overview** tab. You can also track which objects use it on the **References** tab. The **References** tab also enables you to quickly create new objects that automatically inherit the selected assignment matrix configuration.

Overview tab

After an assignment matrix entry is created, you can modify it in the following ways by clicking **Edit**  on the **Overview** tab.

Properties

Name and Description

Update the assignment matrix **Name** and **Description** under the **Properties** section.

Entries

Update the assignment matrix details under the **Entries** section.

- **Save As**

Creates a new entry (row) based on the selected entry.

- **Delete**

Deletes the selected entry.

- **Cut**

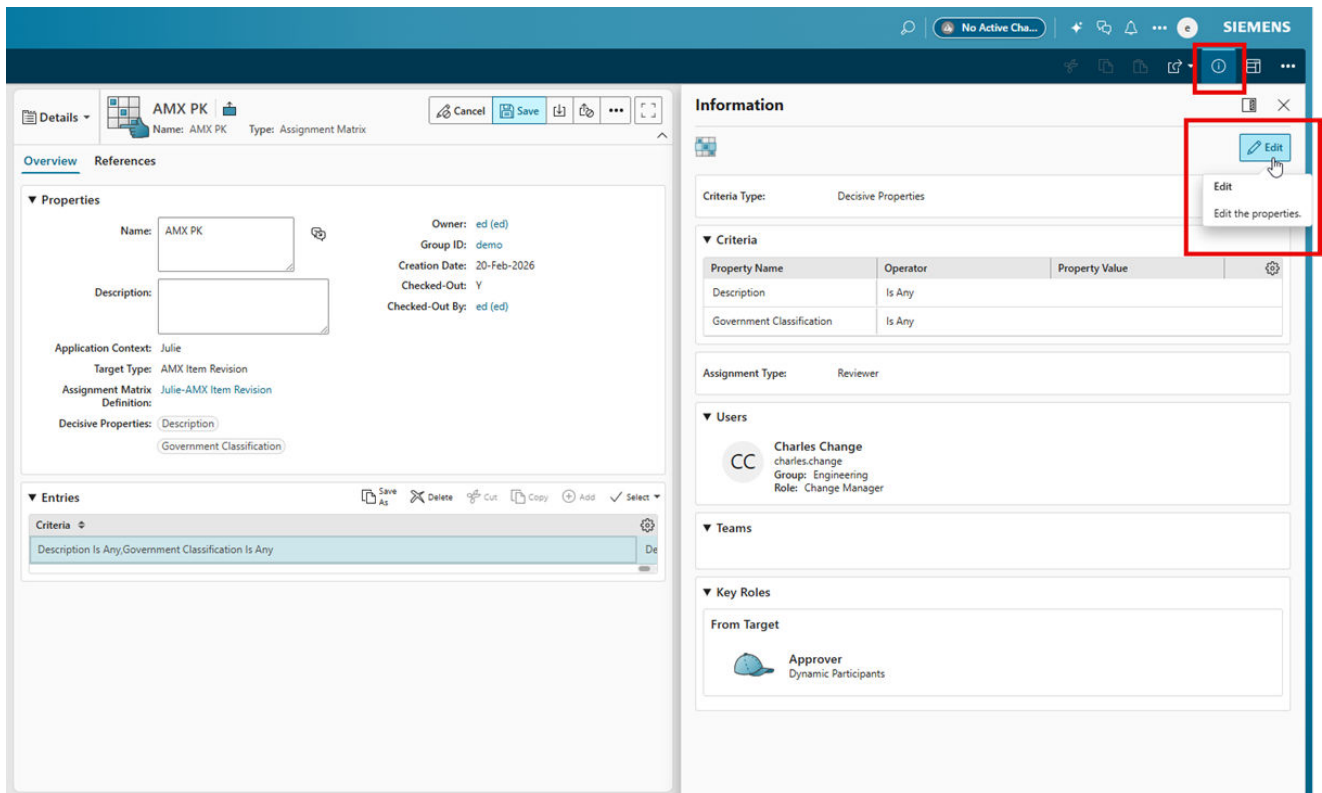
Removes the assignment matrix and places it on the clipboard.

- **Copy**

Duplicates selected assignment matrix and places it on the clipboard without removing it from the **Entries** table.

- **Information**

To update additional properties as well as any users, click a row in the **Entries** table, then click the **Information** icon  to display the **Information** panel. Here, you can edit entry criteria as well as **Users**, **Teams**, or **Key Roles**.



To edit any of the information, click **Edit** , make any changes, and then **Save**  after any updates.

References

The **References** tab displays the objects that use a selected assignment matrix. When you find an assignment matrix with the configuration you want to use, you can create a new object that uses that assignment matrix directly from this tab. This approach is faster than creating a new object and then searching for the assignment matrix to apply.

Click **Add**  to create a new object, then select the object **Type** from the **Add** panel. The system creates a new object and automatically applies the selected assignment matrix to it. The new object inherits all the properties and assignees defined in the assignment matrix.

After the object is created, it appears in the **References** tab list along with any other objects that use the same assignment matrix. This view helps you track which objects share the same assignment matrix configuration. If multiple users create objects using the same assignment matrix, all those objects appear in the **References** tab when you select that matrix. For example, if three users create separate item revision objects and all three use the same assignment matrix, the **References** tab displays three entries.

Add and modify the AMX handler

Add the **AMX-auto-assign-task-assignees-from-assignment-matrix** (AMX) handler to a workflow task to automatically assign task assignees from an assignment matrix. The handler evaluates the target object's decisive properties against assignment matrix rows and applies matching assignments during the workflow process.

The AMX handler uses arguments to identify various assignment combinations for an object. Arguments can consider assignees being merged or overridden, partial matches for objects, and dynamic participants. Any workflow template can add this handler to use an assignment matrix.

For example, if an assignment matrix rule specifies **workflow target:\$CHANGE_SPECIALIST** in the **Assignees - Key Roles** column, the handler retrieves all participants assigned to the Change Specialist role on the target object and automatically assigns them to the workflow task when the decisive property criteria are satisfied.

See the handler documentation for detailed examples.

Procedure

1. Open the template that contains the task that you want to contain an assignment matrix, and click **Edit**  > **Start Edit**.
2. Click the task to include the assignment matrix.
3. Click the **Handlers** tab.

In the left pane, the table lists any handlers assigned to the selected task. Handlers are listed under their specific trigger group in order to represent the running order of action and rule handlers. To see any handlers currently assigned under a specific trigger, click expand ▼.

4. Select a trigger group to add the handler to.
5. In the **New Handler** panel on the right, click **Action** for the handler type.
6. All the applicable handlers for the trigger group are available in the drop-down list. The handler list is not case-sensitive, and it may contain custom handlers as well as handlers that are included with other solutions.

Select the **AMX-auto-assign-task-assignees-from-assignment-matrix** handler.

The mandatory arguments are displayed.

Handler Type: ☒ Action ☐ Rule

★ Name: AMX-auto-assign-task-assignees-from-assignment-matrix Help

⊕ Add Arguments

Argument: Value(s):

-target_type Click to add a value

-application_context Type in required value(s)

-assignment_type Type in required value(s) Type in required value(s)

Add

7. Enter the corresponding values in the **Value(s)** field.

Note

Argument descriptions are listed in the handler documentation.

8. To add additional arguments, click **Add Arguments** ⊕ and select the new argument from the drop-down list.
9. Enter any corresponding values in the **Value(s)** field.

Hints are provided for the handler arguments and their respective values to help you identify mandatory and optional parameters while avoiding runtime errors.

10. Click **Add**.

After completing this step, an assignee will be automatically assigned to the task if the corresponding assignment matrix matches these specific properties.

Note

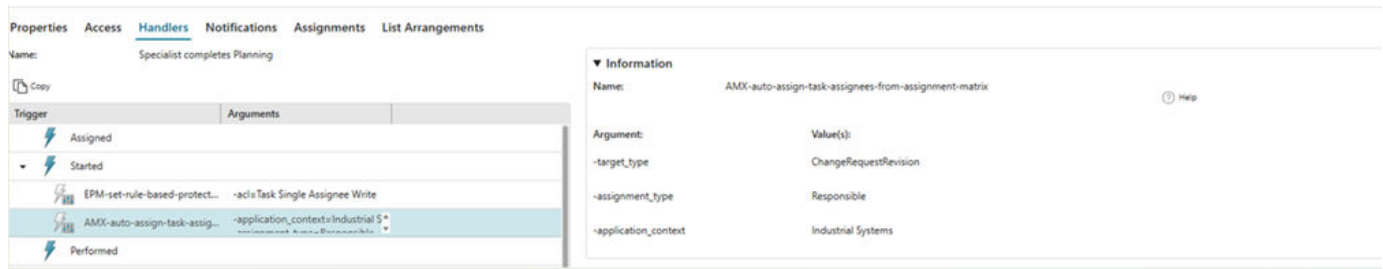
Multiple **AMX-auto-assign-task-assignees-from-assignment-matrix** handler entries can be added for a single task definition.

Expand the trigger group to view the new AMX handler in the table. Once added, you can highlight the handler and click **Cut** ✂, **Copy** 📄, or **Delete** ⊖. To make any additional changes to the argument, highlight the row and edit the handler in the **Information** panel on the right.

Examples

Example 1

The following example shows the assignment matrix handler configuration for the **Specialist completes Planning** task in the Industrial Systems Change Request Process workflow:



- Handler Name: **AMX-auto-assign-task-assignees-from-assignment-matrix**
- Trigger: **Started**
- Mandatory Arguments:
 - **-target_type**: ChangeRequestRevision
 - **-assignment_type**: Responsible
 - **-application_context**: Industrial Systems

When this task starts, the handler evaluates all assignment matrix entries for the Industrial Systems application context where the target type is **Change Request Revision** and the assignment type is **Responsible**. If the target object's decisive properties match an entry in the assignment matrix, the handler automatically assigns the specified users, teams, and dynamic participants to the task.

Example 2

The handler supports optional arguments that determine the assignment source:

- **-property**: Specifies a property to retrieve the assignment source object. When specified, the handler uses the assignment source for participant resolution.
- **-relation**: Specifies a relation to retrieve the assignment source object. When specified, the handler uses the assignment source for participant resolution.
 - **-secondary_type**: Dependent optional argument used with type-based filtering.
- **relationS2P**: Specifies the relation which is used to retrieve the primary objects using the target object.
 - **-primary_type**: Dependent optional argument used with type-based filtering.

When you specify one of the optional arguments (**-property**, **-relation**, and **-relationS2P**), Teamcenter locates the assignment matrix based on the assignment source objects.

If you do not specify either optional argument, Teamcenter retrieves the assignment matrix from the workflow target object. If the workflow target object does not contain an assignment matrix, Teamcenter then uses the workflow reference objects to locate an assignment matrix.

The following example shows the assignment matrix handler configuration that uses an assignment source to retrieve assignees:

- Trigger: **Started**
- Arguments: -**target_type:ItemRevision**
- assignment_type: **Reviewer**
- application_context: **General**
- relation: **IMAN_specification** (optional - retrieves assignment source objects)

When this task starts, the handler evaluates the workflow target object (**ItemRevision**). Because the relation argument is specified, the handler retrieves related objects via the **IMAN_specification** relation and uses those assignment source objects to find the assignment matrix. The handler then assigns reviewers based on the assignment matrix entries associated with the assignment source objects.